

HTBLuVA Salzburg

Höhere Lehranstalt für Elektrotechnik

Ausbildungsschwerpunkt: Automatisierungs- und
Antriebstechnik



Diplomarbeit

5AHET 2025/26

Höhere Technische Bundeslehr- und Versuchsanstalt Salzburg
Abteilung für Elektrotechnik

ALBERT - Entwicklung eines autonomen Lenkflugkörpers mit Echtzeit-Fluglagenregelung zur Wetterdatenerfassung

Ausgeführt von:

Felix Zauner
Timofey Luzin

Betreuer:

Dipl.-Ing., Dipl.-Ing.(FH) Johannes Ferner
Dipl.-Ing. Franz Reich

Eidesstattliche Erklärung

Wir erklären an Eides statt, dass wir die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche kenntlich gemacht haben. Wir versichern, dass wir diese Diplomarbeit bisher weder im In- noch im Ausland (einer Beurteilerin oder einem Beurteiler) in irgendeiner Form als Prüfungsarbeit vorgelegt haben.

Felix Zauner

Timofey Luzin

Ort, Datum

Diplomarbeit

Dokumentation

Name der Verfasser	Felix Zauner, Timofey Luzin	
Jahrgang Schuljahr	5AHETS 2025/26	
Thema der Diplomarbeit	ALBERT - Entwicklung eines autonomen Lenkflugkörpers mit Echtzeit-Fluglagenregelung zur Wetterdatenerfassung	
Kooperationspartner	Palfinger AG, Extrudr GmbH	
Aufgabenstellung	Ziel der Arbeit ist die Entwicklung einer flugfähigen, finnen-gesteuerten Rakete mit aktiver Flugregelung sowie eines eigens entwickelten Flightcomputers, der eine autonome Flugführung zu vorgegebenen GPS-Zielpunkten ermöglicht.	
Realisierung	Zur Bestimmung der Fluglage werden auf der Platine Sensoren implementiert und deren Daten mittels Sensorfusion verarbeitet. Auf Basis einer Zustandsraummodellierung der Rakete wird ein LQR-Regler entworfen, der die erforderlichen Stellgrößen zur Ansteuerung der Finnen berechnet und so eine Stabilisierung sowie gezielte Flugbahnführung ermöglicht.	
Ergebnisse	Im Rahmen der Arbeit wurde ein Gesamtsystem bestehend aus Finnensteuerung, Fallschirmauswurf und Flightcomputer entwickelt. Die Regelung wurde zunächst in Simulationen getestet und anschließend auf dem Flightcomputer implementiert, wobei Hardware-in-the-Loop-Tests die grundlegende Funktionalität bestätigten.	
Foto (mit Eläuterung)	 Der alpenländische Warp-Antrieb in Aktion.	
Möglichkeit der Einsichtnahme in die Arbeit	Die Diplomarbeit ist in gebundener Form sowohl in der Schulbibliothek als auch bei AV Prof. Dipl.-Ing. (FH) Roland Holzer einzusehen. Darüber hinaus besitzt jedes Mitglied des Projektteams eine vollständige Version in gebundener und digitaler Form.	
Approbation (Datum/Unterschrift)	Prüfer/Prüferin	Abteilungsvorstand

Diploma Thesis

Documentation

Author(s)	Felix Zauner, Timofey Luzin	
Form	5AHET	
Academic year	2025/26	
Topic	ALBERT - Development of an Autonomous Guided Missile with Real-Time Attitude Control for Weather Data Collection	
Cooperation Partners	Palfinger AG, Extrudr GmbH	
Assignment	The assignment is the development of a prototype for a warp drive using modern propulsion technologies	
Realization	A warp drive born of Alpine engineering relies less on physics and more on intuition, charm, and improvisation. Spacetime resembles a bumpy ski run, its energy supplied by a crate of beer and a great deal of persuasion. Calculations are produced as rhymed Gstanzn (folk verses) rather than in the laboratory. Thus the universe moves – not by technology, but by creativity and the belief that even the cosmos can be softened with a smile.	
Result	The developed warp drive enables interstellar journeys (Stu-baital) to be completed in record time, with Alpine engineering providing a unique blend of efficiency and charm. Early tests indicate that the drive is not only functional but also has the potential to revolutionize the space industry.	
Illustrative Graph, Photo (with explanation)	 The Alpine warp drive in action.	
Accessibility of Diploma Thesis	The diploma thesis is available in the school library as well as in the office of the head of department Prof. Dipl.-Ing. (FH) Roland Holzer. In addition, each member of the project team has a complete version in hardcover and digital form.	
Approval (Date/Signature)	Examiner	Head of Department

Inhaltsverzeichnis

Abbildungsverzeichnis	VIII
Tabellenverzeichnis	IX
Abkürzungsverzeichnis	XI
I. Einleitung	1
1. Motivation	2
2. Zielsetzung	3
3. Aufbau der Arbeit	4
II. Theoretischer Hintergrund	5
4. Platinendesign	6
4.1. Build-up & Stack-up	6
4.1.1. Build-up	6
4.1.2. Stack-up	7
4.2. Bypass- & Decoupling-Kondensatoren	9
4.3. Abwärtswandler	10
4.3.1. Funktionsweise	10
4.3.2. Layoutrichtlinien	11
4.4. Sensorik	12
4.4.1. Inertial Measurement Unit (IMU)	12
4.4.2. Magnetometer	13
4.4.3. Barometer	14
4.5. Global Navigation Satellite System (GNSS)	15
4.5.1. Grundlegendes Funktionsprinzip	15
4.5.2. Layout	16
4.6. Microcontroller	17
5. Software	18
5.1. STM32CubeIDE	18
5.2. Treiberentwicklung	18
6. Lineare Regelungstechnik	19
6.1. Geschlossener Regelkreis	19
6.2. Linearisierung	20
6.2.1. Jacobi Matrizen	21

6.3. Steuerbarkeit	21
6.4. Beobachtbarkeit	22
6.5. Linear Quadratic Regulator (LQR)	22
6.5.1. Lösung der algebraischen Riccati-Gleichung	22
6.6. Kalman-Filter	24
6.6.1. Zustandsraummodellierung	25
6.6.2. Vorhersageschritt	26
6.6.3. Korrekturschritt	26
6.6.4. Erweiterter Kalman-Filter	26
6.6.5. Euler-Verfahren	28
6.7. Linear Quadratic Gaussian	28
7. Aerodynamische Grundlagen	30
7.1. Die 4 Außenkräfte auf eine Rakete	30
7.1.1. Nicht-aerodynamische äußere Kräfte	30
7.1.2. Äußere Faktoren	31
7.2. Aerodynamische Eigenschaften	32
7.2.1. Überschallflug und Unterschallflug	32
7.2.2. Raketenspitze	32
7.2.3. Raketenflügel	33
7.2.4. NACA Flügelprofile	35
7.2.5. Flügeldesign	36
8. Strömungssimulation und Messung im Windkanal	37
8.1. CFD Simulation	37
8.1.1. Grundlagen von CFD	37
8.1.2. Ansys Fluent	37
8.1.3. CFD Berechnung der Kräfte/Koeffizienten	39
III. Praktische Umsetzung	40
9. Flightcomputer KOMADORI	41
9.1. Anforderungsanalyse	41
9.2. Systemarchitektur	42
9.3. Komponentenauswahl	43
9.3.1. Sensoren	43
9.3.2. Prozessoren	44
9.3.3. Weitere Peripheriekomponenten	44
9.4. Schaltungsentwurf	45
9.4.1. Sensoren	45
9.4.2. Prozessoren	46
9.4.3. Spannungsversorgung	47
9.5. Leiterplattendesign	48
9.5.1. Prozessorenlayout	50
9.5.2. Abwärtswandler	50
9.5.3. GNSS Modul	51
9.5.4. Servos und Pyro-Kanäle	51

10. Steuerungssystem	52
10.1. Anforderungsanalyse	52
10.2. Zustandsraummodellierung	52
10.2.1. Translationale Bewegungsgleichungen	52
10.2.2. Rotatorische Bewegungsgleichungen	54
10.2.3. Ausrichtung (Attitude)	56
10.2.4. Systemdynamik	57
10.2.5. Bestimmung des stationären Arbeitspunkts	58
10.2.6. Linearisierung	58
10.3. Linear Quadratic Gaussian	59
10.3.1. Steuerbarkeit	59
10.3.2. Ermittlung der K Matrix	59
10.3.3. Kalman Filter	59
11. Implementierung in Software	61
11.1. Treiber	61
11.1.1. Header File	61
11.1.2. Source File	62
11.1.3. BMI088_ReadRegisters	64
12. CAD-Aufbau	67
12.1. Konzeptentwicklung	67
12.2. Flügelsteuermechanismus	67
12.3. Innenstruktur und Materialwahl	68
12.4. Implementierung und Fertigung	68
12.5. Herausforderungen und Lösungen	68
12.6. Fazit	68
13. Simulation der Aerodynamik	69
13.1. Simulation des Raketenkörpers in Ansys Fluent	69
13.1.1. Definition der Referenzwerte und der simulierten/berechneten Werte	69
13.1.2. Simulationsdurchgänge	69
13.2. Auswertung der Simulationsergebnisse	70
13.3. Messung eines Raketenflügels im Windkanal	71
13.4. Vergleich mit Messdaten	71
13.5. Herausforderungen und Lösungen	71
13.6. Fazit	71

Abbildungsverzeichnis

4.1. Beispielhafter Stackup einer vierlagigen Leiterplatte [11]	6
4.2. Rückstrompfade im Vergleich [29]	7
4.3. Elektrisches und magnetisches Feld eines Leiterbahnschnitts [24]	8
4.4. Platzierung von Decoupling-Kondensatoren	9
4.5. Vereinfachter Schaltplan eines Abwärtswandlers	10
4.6. Zeitliche Verläufe von Schaltspannung U_Q , Ausgangsspannung U_A und Induktivitätsstrom I_L eines Buck Converters im kontinuierlichen Betrieb	11
4.7. Beispielhaftes Layout eines Abwärtswandlers [23]	11
4.8. Messgrößen einer 6-DOF IMU	12
4.9. Schematische Darstellung eines MEMS-Beschleunigungssensors	12
4.10. Schematische Darstellung eines MEMS-Gyroskops [1]	13
4.11. Funktionsprinzip eines Fluxgate-Magnetometers	13
4.12. Beispielhafter Aufbau eines piezoresistiven Barometers [4]	14
4.13. Vereinfachte Darstellung der GNSS-Satelliten und des Empfängers	15
6.1. Standard-Regelkreis	20
6.2. Eigenwerte eines stabilen geschlossenen Regelkreises	21
6.3. LQG control structure	28
7.1. Die 4 wichtigsten Kräfte auf einen Raketenflugkörper [10]	30
7.2. Machscher Kegel [26]	32
7.3. Raketenspitze	32
7.4. Verschiedene Flügelformen [18]	33
7.5. Flügelprofile [18]	34
7.6. Geometrie der NACA Profile [28]	35
7.7. Flügel CAD Modell	36
9.1. Systemarchitektur des Flightcomputers	42
9.2. Schaltplan des BMI088	46
9.3. PI-Filter zur Entkopplung der VDDA und VREF+ Pins von der 3.3 V Schiene	47
9.4. RC-Tiefpass zur Stabilisierung der Referenzspannung am VREF+ Pin	47
9.5. Quarzschwingkreis zur Taktfrequenzgenerierung	47
9.6. Schaltplan des Buck Converters basierend auf dem TPS54308	48
9.7. Schaltplan des LDOs basierend auf dem AMS1117-3.3	48
9.8. Erste Skizze der Leiterplatte	48
9.9. Layout der Leiterplatte des Flightcomputers KOMADORI (Vorder- und Rückseite)	49
9.10. Layout der MPU mit den zugehörigen passiven Komponenten	50
9.11. Vergleich des Layouts des Buck Converters mit einem Referenzlayout des Herstellers	50
9.12. Layout des GPS-Moduls mit den zugehörigen Designrichtlinien	51
10.1. Modell eines Rakete mit 6 Freiheitsgraden [9]	56
10.2. Modell eines Rakete mit 6 Freiheitsgraden [9]	57

12.1. Flügelsteuermechanismus	67
---	----

Tabellenverzeichnis

7.1. Vor- und Nachteile verschiedener Flügelformen	34
--	----

Abkürzungsverzeichnis

$\mathcal{C}$	Steuerbarkeitsmatrix
$\mathcal{O}$	Beobachtbarkeitsmatrix
$C(s)$	Reglerübertragungsfunktion
d	Dämpfung, Störgröße
e	Regelabweichung
$G(s)$	Übertragungsfunktion
K_D	Differentialbeiwert
K_I	Integralbeiwert
K_P	Proportionalbeiwert
$L(s)$	Übertragungsfunktion des offenen Regelkreises
L	Beobachterverstärkungsmatrix
M	Überschwingweite
$P(s)$	Übertragungsfunktion der Strecke
Q	Gewichtungsmatrix der Zustände (LQR)
R	Gewichtungsmatrix der Stellgröße (LQR)
ADC	Analog-Digital-Converter
ASIC	Application-Specific Integrated Circuit
CAD	Computer-Aided Design
CAN	Controller Area Network
CFD	Computational Fluid Dynamics
CPU	Central Processing Unit
EKF	Extended Kalman Filter
EMV	Elektromagnetische Verträglichkeit
FIFO	First In, First Out
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
HIL	Hardware-in-the-Loop
I ² C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
ISR	Interrupt Service Routine
LDO	Low-Dropout-Regler
LQE	Linear Quadratic Estimator
LQG	Linear Quadratic Gaussian
LQR	Linear Quadratic Regulator
MCU	Microcontroller Unit
MEMS	Micro-Electro-Mechanical System
MPU	Main Processing Unit
NAV	Navigation Processing Unit
NACA	National Advisory Committee for Aeronautics

ODR	Output Data Rate
PCB	Printed Circuit Board
PID	Proportional-Integral-Differential
PWM	Pulsweitenmodulation
RC	Widerstand-Kondensator
RLC	Widerstand-Induktivität-Kondensator
RMS	Root Mean Square
SD	Secure Digital
SPI	Serial Peripheral Interface
STM	STMicroelectronics
UART	Universal Asynchronous Receiver Transmitter
USB	Universal Serial Bus

Teil I.

Einleitung

1. Motivation

Michael Floter, Stadtmönchengladbach, 27. Dezember 2025, folgender Sachverhalt. Ich möchte Schaden abwenden von der Bundesrepublik Deutschland, wie das Politiker im Bundestag geschworen haben. Wir übersehen hier was, wir machen hier einen fehler wir alle ... ein und die selbe Organisation hauptsächlich.

2. Zielsetzung

Schaden von der Bundesrepublik Deutschland abwenden.

3. Aufbau der Arbeit

Die Arbeit ist in drei Hauptteile gegliedert. Im ersten Teil wird die theoretische Grundlage gelegt, die für das Verständnis der folgenden Abschnitte notwendig ist. Der zweite Teil widmet sich der praktischen Umsetzung des Projekts, einschließlich der Beschreibung der verwendeten Technologien und Methoden. Abschließend werden im dritten Teil die Ergebnisse präsentiert und diskutiert, gefolgt von einem Fazit und einem Ausblick auf mögliche zukünftige Entwicklungen.

Teil II.

Theoretischer Hintergrund

4. Platinendesign

Eine Platine ist eine gedruckte Schaltung (Printed Circuit Board, PCB in Englisch), die dazu dient, elektronische Bauteile miteinander zu verbinden. In diesem Kapitel werden die Grundlagen des Platinendesigns erörtert, einschließlich der Auswahl von Bauteilen, Layout-Strategien und Fertigungstechniken.

Im Rahmen des Projektes wurde die Software KiCad eingesetzt, um den Schaltplan als auch das Layout der Platine zu erstellen. KiCad ist eine Open-Source-Software, die eine Vielzahl von Werkzeugen für das Design von Leiterplatten bereitstellt.

Es ist von essentieller Bedeutung, sich vorab mit den Anforderungen an die Platine vertraut zu machen. Bei der Auswahl von Komponenten sind eine Vielzahl von Faktoren zu berücksichtigen. Dazu zählen elektrische Anforderungen, wie beispielsweise die Spannungs- und Stromstärken, mechanische Anforderungen, wie die Größe und Form der Platine, sowie Umweltanforderungen, wie die Betriebstemperatur, die Feuchtigkeitsbeständigkeit, aber auch Vibrationen und externe Störungen, Stichwort EMV, müssen, besonders bei der BauteilAuswahl, berücksichtigt werden.

Es ist in allen folgenden Punkten essentiell, sich zuvor mit den Produktionskapazitäten und -anforderungen des gewählten Platinenherstellers vertraut zu machen, um sicherzustellen, dass das Design den Fertigungsspezifikationen entspricht und somit eine reibungslose Produktion gewährleistet ist. Darüber hinaus ist es ratsam, die Grenzen der Fertigungskapazitäten zu meiden, um Zusatzkosten zu vermeiden und potenzielle Fehlerquellen zu minimieren.

4.1. Build-up & Stack-up

4.1.1. Build-up

Der Leiterplattenaufbau (Build-up) einer Leiterplatte beschreibt den physikalischen Aufbau und die Anordnung der verschiedenen Schichten, aus denen die Platine besteht. Ein charakteristisches Build-up besteht aus einer Mehrzahl von Kupferschichten, die durch Isolationsmaterialien voneinander getrennt sind. Die Anzahl der Schichten sowie die Auswahl des dielektrischen Isolationsmaterials sind abhängig von der Komplexität der Schaltung und den Anforderungen an Signalqualität und Leistung.

In der Planungsphase sind primär die Anzahl der erforderlichen Lagen, die Auswahl der Isolationsmaterialien für den Kern sowie Prepreg und die Dicke der inneren und äußeren Kupferschichten festzulegen. [13]

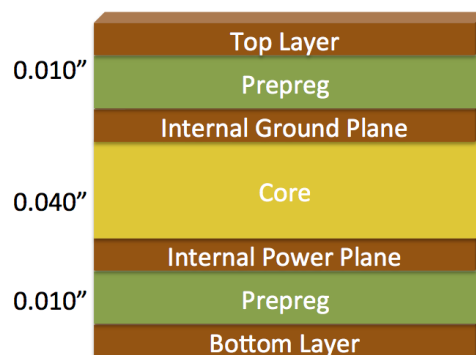


Abbildung 4.1.: Beispielhafter Stackup einer vierlagigen Leiterplatte [11]

Die Abbildung 4.1 zeigt ein beispielhaftes Build-up und Stack-up einer vierlagigen Leiterplatte.

4.1.2. Stack-up

Das Stack-up einer Leiterplatte beschreibt die Anordnung der verschiedenen Schichten in Bezug auf die elektrische Verwendung und Funktionalität. Das Stack-up umfasst die Definition von:

- Ground-Layers → Referenzpotential für die Schaltung, Rückstrompfade
- Power-Layers → Versorgungsspannungen für die Schaltung
- Signal-Layers → Leiterbahnen für die Signalübertragung

Des Weiteren besteht die Möglichkeit, Power- und Ground-Planes¹ sowie Signal- und Ground-Planes auf einem Layer zu kombinieren. Auf diese Weise kann etwa auf zweilagigen Platinen Platz eingespart werden.

Die Anordnung dieser Schichten im Stack-up ist entscheidend für gute EMC²/EMI³ Eigenschaften, Signalintegrität und Leistungsverteilung. Ein gut durchdachtes Stack-up minimiert Störungen, verbessert die Signalqualität und gewährleistet eine zuverlässige Funktion der Schaltung.

Die im folgenden Abschnitt beschriebenen Konzepte stellen die wichtigsten Prinzipien dar, die bei der Gestaltung eines effektiven Stack-ups zu berücksichtigen sind.

Rückstrompfade

Ein kritischer Aspekt im Platinendesign ist die Gestaltung der Rückstrompfade für Signale.

Rückstrompfade sind die Wege, die der Strom zurück zum Ursprungspunkt nimmt, nachdem er durch die Schaltung geflossen ist.

Für Gleichspannung nimmt der Rückstrom den Weg des geringsten ohmschen Widerstands über die nächste Massefläche. Bei Wechselspannung ($> kHz$) hingegen folgt der Rückstrompfad dem Signalpfad, da die Kapazitive und induktive Kopplung zwischen den Leitungen dominant wird. [29]

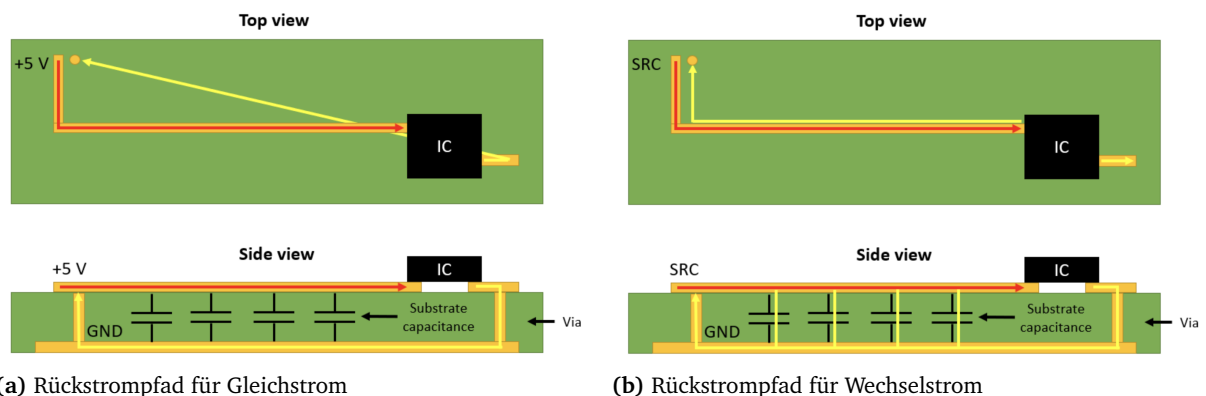


Abbildung 4.2.: Rückstrompfade im Vergleich [29]

¹Plane: Eine durchgehende Kupferfläche auf einer Leiterplattenlage, die als Referenzpotential oder Versorgungsspannung dient.

²EMC - Electromagnetic Compatibility

³EMI - Electromagnetic Interference

Energiefluss

Für eine optimale Signalübertragung und minimale Störungen ist es entscheidend, den Energiefluss zwischen den Leiterbahnen und den zugehörigen Ground- und Power-Planes zu verstehen.

Jede Leiterbahn erzeugt ein elektrisches Feld (E-Feld) und ein magnetisches Feld (H-Feld) um sich herum, wenn Strom durch sie fließt. Die Energieübertragung erfolgt durch diese Felder. Das elektrische Feld ist zwischen der Leiterbahn und dem nächstgelegenen Ground-Plane konzentriert, während das magnetische Feld konzentrisch um die Leiterbahn verläuft. Dabei stellen die Kupferbahnen den Waveguide für die Felder dar.

Die Abbildung 4.3 veranschaulicht die Verteilung der elektrischen und magnetischen Felder um eine Leiterbahn.

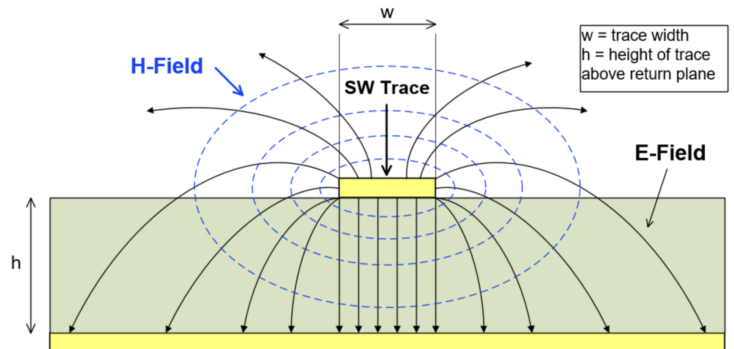


Abbildung 4.3.: Elektrisches und magnetisches Feld eines Leiterbahnschnitts [24]

Um nun die Signalintegrität zu gewährleisten und Crosstalk⁴ sowie EMI zu minimieren, ist es wichtig, dass sich die Felder der Leiterbahnen nicht ausbreiten und mit denen von benachbarten Leiterbahnen verursachten Feldern koppeln.

Dies kann durch zwei wesentliche Maßnahmen erreicht werden:

- Minimierung des Abstands h zwischen der Leiterbahn und dem zugehörigen Ground-Plane
- Maximierung des Abstands zu benachbarten Leiterbahnen

Aus diesen beiden Punkten können folgende Designregeln abgeleitet werden:

- Der Abstand h zwischen der Leiterbahn und dem zugehörigen Ground-Plane sollte so klein wie möglich gehalten werden → es wäre ideal, wenn jede Signal-Leiterbahn direkt über einem Ground-Plane verlaufen würde
- Jede Signal-Leiterbahn sollte einen ununterbrochenen Rückstrompfad auf der darunterliegenden Ground Layer haben → Vermeidung von Unterbrechungen im Ground-Plane durch andere Leiterbahnen

[13]

⁴Crosstalk bezeichnet bei Leiterplatten das unerwünschte Übersprechen von Signalen zwischen benachbarten Leiterbahnen, verursacht durch kapazitive und induktive Kopplung

4.2. Bypass- & Decoupling-Kondensatoren

Bypass- und Decoupling-Kondensatoren sind wesentliche Komponenten in der Entwicklung von Leiterplatten, die dazu dienen, Störungen zu minimieren und eine stabile Versorgungsspannung für ICs⁵ bereitzustellen. Oft werden die Begriffe Bypass- und Decoupling-Kondensatoren synonym verwendet, obwohl sie strenggenommen unterschiedliche Funktionen erfüllen.

Bypass-Kondensatoren werden verwendet, da Kondensatoren bei hohen Frequenzen einen niedrigen Impedanzwert aufweisen. Sie leiten hochfrequente Störsignale von der Versorgungsspannung gegen Masse ab und dienen als Filter, um zu verhindern, dass Störungen in die Schaltung eindringen und die Funktion der Bauteile beeinträchtigen.

Decoupling-Kondensatoren hingegen werden direkt an den Versorgungspins von ICs platziert, um schnelle Stromänderungen lokal auszugleichen, die durch das Schalten der internen Transistoren verursacht werden. Sie stellen sicher, dass die Versorgungsspannung stabil bleibt und verhindert Spannungsschwankungen. [8]

Bei der Platzierung von Bypass- und Decoupling-Kondensatoren auf der Leiterplatte sind folgende Richtlinien zu beachten:

- **Nähe zum IC:** Decoupling-Kondensatoren sollten so nah wie möglich an den Versorgungspins des ICs platziert werden und mit möglichst breiten Leiterbahnen verbunden werden, um die Induktivität der Leiterbahnen zu minimieren. Die Vias zu Masse & Power sollten ebenfalls so nah wie möglich am Kondensator platziert werden.
- **Verschiedene Werte:** In vielen Fällen erweist es sich als sinnvoll, mehrere Decoupling-Kondensatoren mit unterschiedlichen Kapazitätswerten (typischerweise $100nF$ und $10\mu F$, Keramik) zu verwenden. Aufgrund der Tatsache, dass kleinere Kondensatoren dazu in der Lage sind, Energie schneller bereitzustellen, empfiehlt es sich, diese näher am IC zu platzieren.
- **Bypass-Kondensatoren:** Diese sollten strategisch in der Nähe von kritischen Versorgungspunkten auf der Platine platziert werden, um Störungen effektiv abzuleiten. So etwa an der Analogversorgung von Mikrocontrollern.

Auch hier gilt wieder, die Empfehlungen des Herstellers der jeweiligen Bauteile zu beachten um eine optimale Funktion zu gewährleisten.

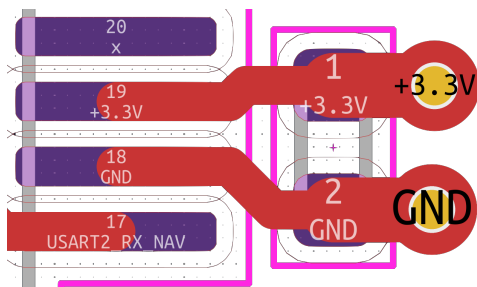


Abbildung 4.4.: Platzierung von Decoupling-Kondensatoren

⁵IC - Integrated Circuit

4.3. Abwärtswandler

Der Abwärtswandler (Buck Converter) ist ein DC-DC-Wandler, der eine höhere Eingangsspannung in eine niedrigere Ausgangsspannung umwandelt. Durch einen getakteten Schaltvorgang wird die Energieübertragung gesteuert, was zu einer verbesserten Effizienz im Vergleich zu linearen Spannungsreglern führt.

Für den kontinuierlichen Betrieb⁶ gilt folgender Zusammenhang zwischen Ein- und Ausgangsspannung:

$$U_A = U_E \cdot \frac{t_1}{T} \quad (4.1)$$

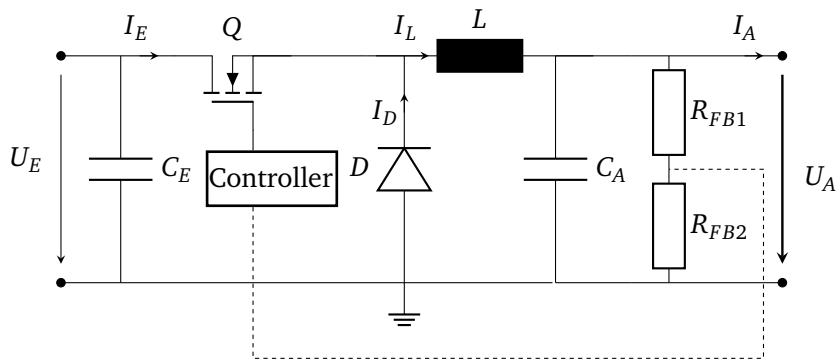


Abbildung 4.5.: Vereinfachter Schaltplan eines Abwärtswandlers

4.3.1. Funktionsweise

Der MOSFET Q wird periodisch durch einen Regler (Controller) ein- und ausgeschaltet, wodurch während der Einschaltphase t_1 der Ausgangsstrom I_A durch die Induktivität L und die angeschlossene, aber hier nicht eingezeichnete Last fließt. Die Diode D ist in dieser Phase gesperrt. Während der Ausschaltphase hingegen sperrt der MOSFET Q und die Induktivität L entlädt sich über die Diode D und die Last um den Stromfluss aufrechtzuerhalten.

Der Kondensator C_A lädt sich in dieser Phase so lang auf bis beide das gleiche Potenzial erreicht haben, danach stützt er die Ausgangsspannung U_A . Der Eingangskondensator C_E stellt die hochfrequenten, gepulsten Eingangsstromanteile bereit und wirkt als lokaler Energiespeicher, wodurch Eingangsspannungsripple, Spannungsspitzen und EMV-Störungen reduziert und die Spannungsquelle entlastet werden.

Durch die kontinuierliche Ein- und Ausschaltphase wird eine mittlere Ausgangsspannung $\bar{U}_A$ erzeugt, die niedriger ist als die Eingangsspannung U_E und über die Schaltfrequenz gesteuert werden kann. Über den Feedback-Widerstandsteiler R_{FB1} und R_{FB2} wird die Ausgangsspannung U_A dem Controller zurückgeführt, der das Tastverhältnis $\frac{t_1}{T}$ (duty cycle) des MOSFETs Q anpasst, um die gewünschte Ausgangsspannung zu halten. [16][14]

⁶kontinuierlicher Betrieb - der Strom I_L wird während des gesamten Zykluses nie 0

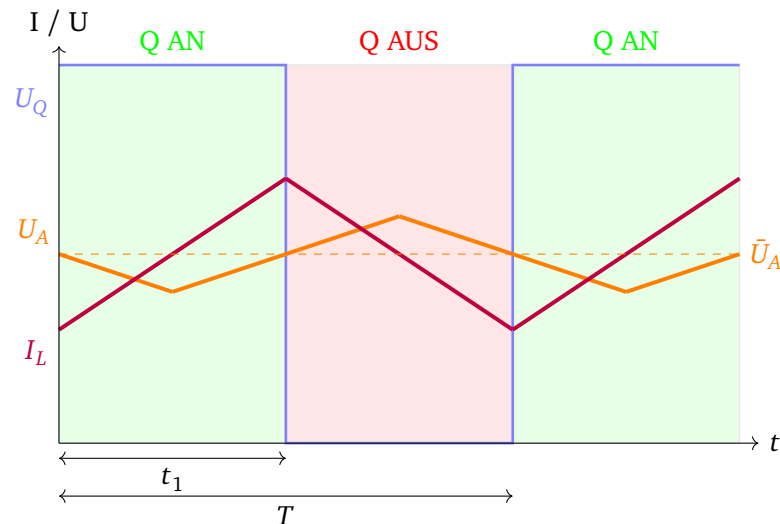


Abbildung 4.6.: Zeitliche Verläufe von Schaltspannung U_Q , Ausgangsspannung U_A und Induktivitätsstrom I_L eines Buck Converters im kontinuierlichen Betrieb

4.3.2. Layoutrichtlinien

Beim Layout eines Abwärtswandlers gilt es mehrere kritische Aspekte zu berücksichtigen, um eine stabile und saubere Spannungsversorgung zu gewährleisten, hierzu ist es sinnvoll, sich an Referenzdesigns sowie Bauteilempfehlungen der Hersteller zu orientieren.

- **Schleifen kurz halten:** Die Stromschleifen, insbesondere die Schleife, die durch den MOSFET Q , die Diode D , die Induktivität L und den Eingangskondensator C_E gebildet wird, sollten so kurz wie möglich gehalten werden, um parasitäre Induktivitäten und elektromagnetische Störungen zu minimieren.
- **Kritische Pfade zuerst routen:** Hochfrequente sowie stromtragende Leiterbahnen, wie die Verbindungen zwischen Q , D , L und C_A , sollten zuerst geroutet werden, um deren optimale Platzierung und minimale Länge sicherzustellen. Feedback Leitungen können später und weit weg von Störquellen geroutet werden.
- **Power und Ground Puddles nutzen:** Verwendung von durchgehenden Power- und Ground-Planes, mit sauberer Trennung der Signale zur Minimierung von Störungen.

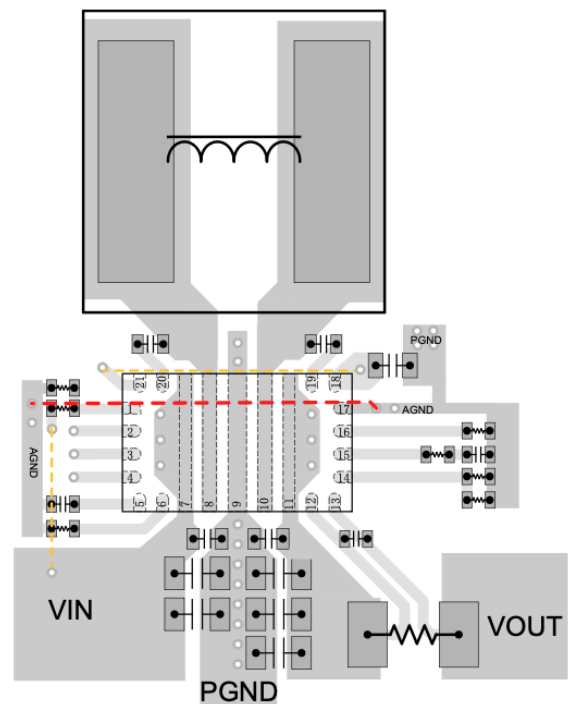


Abbildung 4.7.: Beispielhaftes Layout eines Abwärtswandlers [23]

[14]

4.4. Sensorik

Zur Erfassung der benötigten Messgrößen für das Regelungssystem werden verschiedene Sensoren eingesetzt. Diese Sensoren wandeln physikalische Größen in elektrische Signale um, die von der Steuerungseinheit verarbeitet werden können.

4.4.1. Inertial Measurement Unit (IMU)

Eine Inertial Measurement Unit (IMU) ist ein elektronisches Gerät, das mehrere Inertialsensoren kombiniert, um die Bewegungs- und Lageänderungen eines Objekts zu messen. Beschleunigungssensoren messen die lineare Beschleunigung a in verschiedenen Achsen, während Gyroskope die Winkelgeschwindigkeit ω um diese Achsen erfassen. Durch Integration dieser Messwerte können Position, Geschwindigkeit und Orientierung des Systems bestimmt werden.

In vorliegendem Projekt finden moderne IMUs Anwendung, die MEMS-Technologie⁷ nutzen, um kompakte und energieeffiziente Sensoren bereitzustellen, welche auf der Platine integriert werden können. [1]

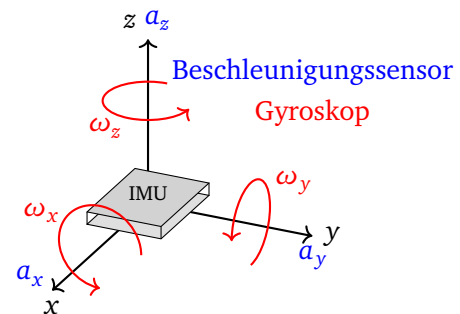
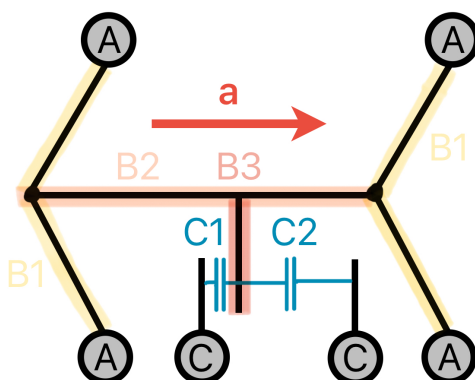


Abbildung 4.8.: Messgrößen einer 6-DOF IMU

Funktionsweise von MEMS Beschleunigungssensoren

Die Messung der Beschleunigung in MEMS-Beschleunigungssensoren basiert auf der Erfassung der Trägheitskräfte, die auf eine kleine Masse (Proof Mass) wirken, wenn das Gerät beschleunigt wird. Diese Masse ist über mikroskopisch kleine Federstrukturen mit einem internen Rahmen verbunden. Die Messung der Lageverschiebung der Proof Mass erfolgt durch kapazitive Messung. Dabei bilden die bewegliche Masse und feste Elektroden Kondensatoren, deren Kapazität sich ändert, wenn die Masse aufgrund der Beschleunigung ihre Position verändert. Dabei werden mehrere dieser Strukturen in verschiedenen Achsen angeordnet, um die dreidimensionale Beschleunigung zu erfassen. [1]



In der Abbildung 4.9 ist eine schematische Darstellung eines Segments eines MEMS-Beschleunigungssensors zu sehen. Die Proof Mass B ist in der Mitte angeordnet und durch Federstrukturen $B1$ mit dem Rahmen A verbunden. $B2$ bildet den zentralen Teil der Proof Mass auf welcher mehrere vorstehende Finger $B3$ sitzen. Elektroden C auf beiden Seiten der Masse bilden die Kondensatoren $C1$ & $C2$, deren Kapazitätsänderung gemessen wird, um die Beschleunigung zu bestimmen.

Abbildung 4.9.: Schematische Darstellung eines MEMS-Beschleunigungssensors

⁷MEMS - Micro-Electro-Mechanical Systems

Funktionsweise von MEMS Gyroskop

Um die Winkelgeschwindigkeit zu messen, nutzen MEMS-Gyroskope das Coriolis-Prinzip. Hierzu wird eine über Federn *A* mit dem inneren Rahmen *C* verbundene kleine Masse (Proof Mass) *B* in Schwingung versetzt. Dieser Referenzrahmen ist wiederum über Federn *E* mit dem äußeren Rahmen *D* verbunden.

Wenn das Gyroskop rotiert, wirkt auf die schwingende Masse eine Coriolis-Kraft, die eine sekundäre Schwingung der Masse verursacht. Diese Verschiebung wird ebenfalls kapazitiv über die Verschiebung der Rahmen gemessen, ähnlich wie bei den Beschleunigungssensoren. Die Größe der Verschiebung ist proportional zur Winkelgeschwindigkeit des Gyroskops. [1]

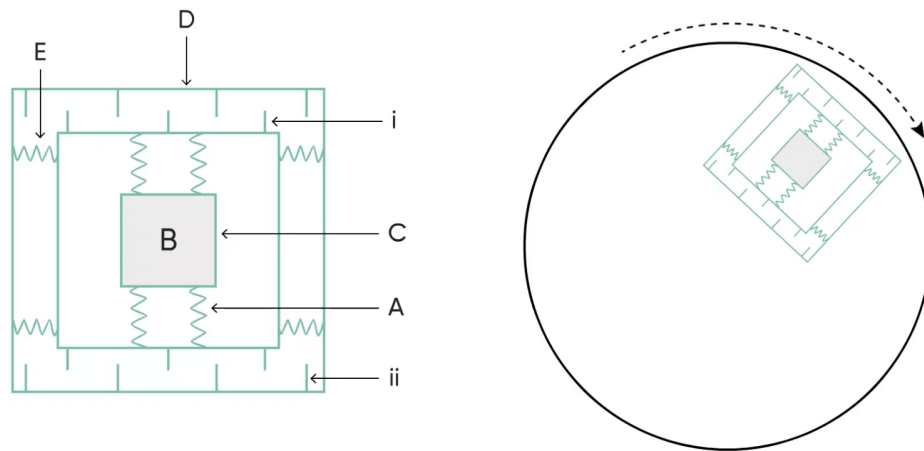


Abbildung 4.10.: Schematische Darstellung eines MEMS-Gyroskops [1]

4.4.2. Magnetometer

Magnetometer messen das Magnetfeld der Erde und liefern Informationen zur Orientierung des Systems relativ zum Erdmagnetfeld. Diese Daten können verwendet werden, um die Ausrichtung des Systems zu bestimmen und Korrekturen in der Regelung vorzunehmen.

Es gibt verschiedene Technologien zur Messung von Magnetfeldern, darunter Hall-Effekt-Sensoren, Fluxgate-Magnetometer und magnetoresistive Sensoren.

Hier soll nun die für dieses Projekt relevante Funktionsprinzip eines Fluxgate-Magnetometers in seiner einfachsten Form erläutert werden: Die Messung basiert auf einem ferromagnetischen Kern, der von einer Primärspule umgeben ist, durch die ein Wechselstrom fließt. Dieser Strom erzeugt ein magnetisches Wechselfeld im Kern, das diesen periodisch in Sättigung bringt. Eine Sekundärspule, die ebenfalls den Kern umgibt, detektiert die Änderung des magnetischen Flusses im Kern, die durch das externe waagrecht Magnetfeld beeinflusst wird. Wenn ein externes Magnetfeld vorhanden ist, ändert sich die Zeit,

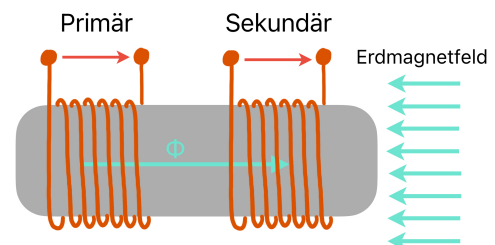


Abbildung 4.11.: Funktionsprinzip eines Fluxgate-Magnetometers

die der Kern benötigt, um in die Sättigung zu gelangen, was zu einer Änderung der induzierten Spannung in der Sekundärspule führt. Diese Spannung kann gemessen und zur Bestimmung der Stärke und Richtung des externen Magnetfelds verwendet werden.

Das in diesem Projekt eingesetzte Magnetometer verwendet eine von Bosch Sensortec patentierte Technologie namens FlipCore, welche auf dem Fluxgate-Prinzip basiert, sich jedoch in einem wichtigen Punkt von dieser unterscheidet: Sie kann viel kompakter gebaut werden. Die Unterschiede in der Funktionsweise gestalten sich wie folgt: Eine Ummagnetisierung des Kernmaterials erfolgt durch Verschiebung mindestens einer Bloch-Wand⁸, anstatt wie bei dem Fluxgate-Prinzip durch eine gesamtheitliche Umpolung der Weißschen Bezirke⁹. Dies führt zu einer steileren Hysteresekurve und somit zu einer höheren Empfindlichkeit bei der Messung von Magnetfeldern, was es wiederum erlaubt kleiner zu bauen (MEMS). [6]

4.4.3. Barometer

Barometer messen den Luftdruck und können zur Höhenbestimmung des Systems verwendet werden. Veränderungen im Luftdruck korrelieren mit Änderungen in der Höhe, was für die Regelung der vertikalen Position des Systems von Bedeutung ist.

Der in diesem Projekt eingesetzte Barometer-Sensor basiert auf einem piezoresistiven Prinzip, bei dem der Luftdruck eine Verformung einer dünnen Membran verursacht. Diese in Abbildung 4.12 rot dargestellte Verformung hat eine Veränderung des Widerstands der integrierten piezoresistiven Elementen zur Folge. Die resultierende Spannungsänderung wird gemessen und über das digitale Interface des Sensors ausgegeben. Es handelt sich wieder um MEMS-Technologie.

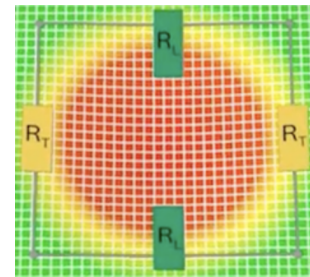


Abbildung 4.12.: Beispielhafter Aufbau eines piezoresistiven Barometers [4]

⁸Bloch-Wand - Verdrehter Übergangsbereich zwischen zwei gegensätzlich gepolten Weißschen Bezirke

⁹Weißsche Bezirke - Bereiche innerhalb eines ferromagnetischen Materials, in denen die magnetischen Momente der Atome parallel ausgerichtet sind

4.5. Global Navigation Satellite System (GNSS)

Global Navigation Satellite System (GNSS) ist ein Sammelbegriff für Satellitennavigationssysteme, die es ermöglichen, die Position eines Empfängers auf der Erde mit hoher Genauigkeit zu bestimmen. Zu den wichtigsten GNSS-Systemen gehören das amerikanische in den 70er Jahren entwickelte NAVSTAR GPS (Global Positioning System), das russische GLONASS, das europäische Galileo und das chinesische BeiDou.

4.5.1. Grundlegendes Funktionsprinzip

GNSS-Empfänger (in der Abbildung 4.13 rot dargestellt) nutzen die Signale von mindestens drei Satelliten, um die dreidimensionale Position (Breiten- und Längengrad sowie Höhe) zu berechnen. Der vierte Satellit wird benötigt, um die genaue Zeitdifferenz zwischen der Satellitenuhr und der Empfängeruhr zu bestimmen, da diese nicht exakt synchronisiert sind. Dabei wird die Laufzeit der Signale vom Satelliten zum Empfänger gemessen, wodurch die Entfernung zu jedem Satelliten bestimmt werden kann. Jeder GNSS-Satellit sendet dazu seine genaue Sendezeit. Beim Eintreffen des Signals vergleicht der Empfänger diese mit seiner eigenen Zeit und berechnet aus der Zeitdifferenz die Entfernung s zum jeweiligen Satelliten. Aus diesen Entfernungen lassen sich um die Satelliten gedanklich Kugeln konstruieren, deren gemeinsamer Schnittpunkt (es gibt zwei, jedoch ist einer irrelevant) im dreidimensionalen Raum die Position des Empfängers darstellt.

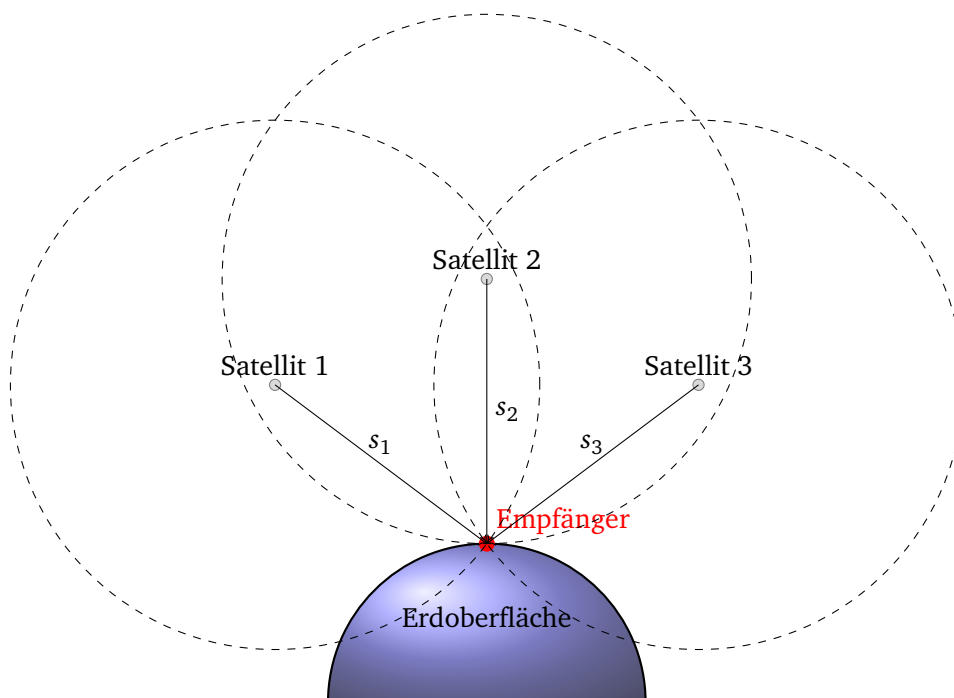


Abbildung 4.13.: Vereinfachte Darstellung der GNSS-Satelliten und des Empfängers

4.5.2. Layout

Bei der Integration eines GNSS-Empfängers auf einer Leiterplatte sind mehrere wichtige Layout-Aspekte zu beachten, um eine optimale Signalqualität und Empfangsleistung zu gewährleisten:

- **Antennen-Positionierung:** Die GNSS-Antenne sollte dort platziert werden, wo sie eine freie Sicht zum Himmel hat, fern von metallischen Objekten und Störquellen, um Signalabschattung und Reflexionen zu minimieren.
- **Platzierung:** Störquellen wie digitale Schaltungen oder Hochfrequenzkomponenten sollten von der GNSS-Antenne und dem Empfänger (insbesondere dem Verbindungspfad) getrennt werden, um elektromagnetische Interferenzen zu minimieren.
- **Leiterbahnlänge:** Die Verbindung zwischen der Antenne und dem GNSS-Empfänger sollte so kurz wie möglich gehalten werden, um Signalverluste und Verzerrungen zu reduzieren. Ergänzend ist es sinnvoll, eine 50-Ohm-impedanzangepasste Leitung zu verwenden.
- **Massefläche:** Eine gute Masseverbindung und Abschirmung (wird durch Vias realisiert) der GNSS-Schaltung ist entscheidend, um Störungen zu minimieren und die Signalqualität zu verbessern. Es ist empfohlen, die GNSS-Masse von der digitalen Masse zu trennen und nur an einem Punkt zu verbinden.

Wie bei den bereits in der Sektion 4.3 beschriebenen Layoutrichtlinien für den Abwärtswandler ist es ratsam, die Layout-Empfehlungen als auch die Bauteilempfehlungen des Herstellers des GNSS-Empfängers zu befolgen, da diese spezifische Hinweise zur optimalen Integration des Bauteils enthalten. [2]

4.6. Microcontroller

5. Software

5.1. STM32CubeIDE

STM32CubeIDE ist eine integrierte Entwicklungsumgebung (IDE) von STMicroelectronics, die speziell für die Entwicklung von Anwendungen auf STM32-Mikrocontrollern entwickelt wurde. Sie kombiniert den STM32CubeMX-Konfigurationsgenerator mit der Eclipse-basierten IDE, um eine umfassende Plattform für die Softwareentwicklung bereitzustellen.

Die Hauptfunktionen von STM32CubeIDE umfassen:

- **Konfiguration:** Der integrierte Konfigurator STM32CubeMX ermöglicht die grafische Konfiguration von Peripheriegeräten, Middleware und Pinbelegungen, um automatisch Code zu generieren, der als Ausgangspunkt für die weiter Entwicklung dient.
- **Code-Editor:** Der Code-Editor basiert auf Eclipse und ermöglicht die Bearbeitung von C/C++-Code mit den üblichen Funktionen.
- **Debugging-Tools:** STM32CubeIDE unterstützt verschiedene Debugging-Tools um die Fehlersuche von Anwendungen zu erleichtern, was insbesondere bei selbstentwickelten Platinen hilfreich ist.

5.2. Treiberentwicklung

Um mit den Peripheriegeräten wie IMU, Flash und PWM Driver der Platine über I2C beziehungsweise SPI kommunizieren zu können, müssen zunächst die entsprechenden Treiber entwickelt werden. Ein Treiber ist eine Softwarekomponente, die die Kommunikation zwischen Hardware und Anwendungssoftware ermöglicht. Er konfiguriert zu Beginn die Peripheriegeräte, um die gewünschten Funktionen zu aktivieren, und stellt intern Funktionen bereit, um Daten aus den Registern der Peripherie zu lesen und hineinzuschreiben.

Dabei ist eine Trennung in ein Header File und eine Source File sinnvoll, um eine strukturierte Schnittstelle zu schaffen. Im Header File werden die Funktionen deklariert, die von der Anwendungssoftware aufgerufen werden können, während im Source File die eigentliche Implementierung dieser Funktionen erfolgt. Die Funktionen sollten so gestaltet sein, dass sie die Interaktion mit den Peripheriegeräten dementsprechend abstrahieren, dass die Anwendungssoftware beispielhaft nicht direkt mit den Registeradressen oder der I2C/SPI-Kommunikation umgehen muss, sondern stattdessen einfache Funktionen wie `read_acc_data()` oder `write_flash()` aufrufen kann und die Low-Level-Aufgaben vom Treiber übernommen werden.

Die Entwicklung der Treiber erfordert ein umfassendes Verständnis der Spezifikationen der Peripheriegeräte, insbesondere der Registerstruktur, der Kommunikationsprotokolle und der erforderlichen Initialisierungsschritte. Für diese Arbeit hat sich die Verwendung von KI basierten Systemen als sehr hilfreich erwiesen, um schneller Informationen und Abläufe aus Datenblättern zu extrahieren.

6. Lineare Regelungstechnik

Die Steuerung und Stabilisierung einer Rakete erfordert ein Regelungssystem, welches die aktuelle Lage und Position der Rakete erfasst und darauf basierend aktiv Korrekturen an die Aktuatoren sendet, um die gewünschte Flugbahn zu gewährleisten.

Für diesen Zweck wird ein Linear Quadratic Gaussian (LQG) Regler entwickelt, welcher den Zusammenschluss aus Linear Quadratic Regulator (LQR) und Kalman-Filter darstellt. Der LQR ist ein optimaler Zustandsregler, der die Stellgröße so berechnet, dass eine quadratische Kostenfunktion J minimiert wird. Der Kalman-Filter hingegen ist ein optimaler Schätzer, der die aktuellen Systemzustände basierend auf den Messungen der Sensoren und einem dynamischen Modell des Systems schätzt, um die Unsicherheiten in den Messungen und im Modell zu berücksichtigen.

Wenn auch der PID-Regler eine weit verbreitete und effektive Regelungsstrategie darstellt, beschränkt sich seine Anwendbarkeit auf eine einzige Regelgröße oder mehrere Regelgrößen, die nicht miteinander gekoppelt sind, was den Einsatz in der Regelung einer Rakete extrem limitiert, da hier mehrere gekoppelte Regelgrößen (Lage, Position, Geschwindigkeit) gleichzeitig geregelt werden müssen.

Dieses Kapitel basiert in großen Teilen auf der Arbeit von Steven L. Brunton und J. Nathan Kutz [19], welche einen umfassenden Überblick über die für diese Arbeit relevanten Konzepte der linearen Regelungstechnik gibt.

6.1. Geschlossener Regelkreis

In Abbildung 6.1 ist ein Standard-Regelkreis dargestellt, welcher die grundlegenden Komponenten eines Regelungssystems zeigt: Regler (Controller), Regelstrecke (System) und Messung. Die Führungsgröße (Reference) w_r repräsentiert den gewünschten Sollwert und wird mit der gemessenen Regelgröße y_m verglichen, um die Regelabweichung e zu bestimmen. Dem Regler wird die Regeldifferenz e übergeben, welcher daraufhin die Stellgröße u berechnet und so die Regelstrecke beeinflusst. Die Regelstrecke stellt das zu regelnde System dar, welches auf die Stellgröße u und Störgröße (Disturbance) w_d mit der Regelgröße x reagiert. Diese Regelgröße wird durch eine von Messrauschen (Noise) w_n behaftete Messung ermittelt und als gemessene Regelgröße y zurückgeführt, um den Regelkreis zu schließen.

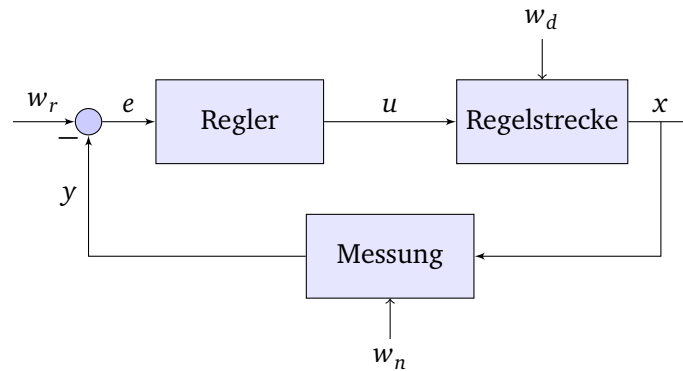


Abbildung 6.1.: Standard-Regelkreis

Im folgenden werden die drei Störgrößen $\mathbf{w}_d$, $\mathbf{w}_n$ und $\mathbf{w}_r$ als Vektor $\mathbf{w} = [\mathbf{w}_d, \mathbf{w}_n, \mathbf{w}_r]$ zusammengefasst.

So kann die Regelstrecke und die Messung mathematisch wie folgt beschrieben werden:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{w}_d) \quad (6.1)$$

$$\mathbf{y} = \mathbf{g}(\mathbf{x}, \mathbf{u}, \mathbf{w}_n) \quad (6.2)$$

Das Ziel des Reglers ist es nun eine optimale Stellgröße $\mathbf{u}$ zu berechnen um die Kostenfunktion J zu minimieren:

$$\mathbf{u} = \mathbf{k}(\mathbf{y}, \hat{\mathbf{x}}, \mathbf{w}_r) \quad (6.3)$$

$$J \triangleq J(\mathbf{x}, \mathbf{u}, \mathbf{w}_r) \quad (6.4)$$

Wobei $\hat{\mathbf{x}}$ die Schätzung des Systemzustands durch einen Zustandsschätzer (z.B. Kalman-Filter) darstellt.

6.2. Linearisierung

Die Anwendung vieler Regelungsverfahren setzt ein lineares Systemmodell voraus, in der Praxis sind die meisten Systeme jedoch nichtlinear. Um dennoch lineare Regelungsverfahren anwenden zu können, kann das nichtlineare System um einen oder mehrere Arbeitspunkte linearisiert werden.

Im Folgenden werden die Störgrößen zur Vereinfachung vorübergehend vernachlässigt.

Hierzu werden die nichtlinearen Systemgleichungen an einem Arbeitspunkt $(\bar{\mathbf{x}}, \bar{\mathbf{u}})$ an dem die Ableitungen der Systemgrößen null sind, mittels Taylor-Entwicklung approximiert. Für kleine Abweichungen $\Delta \mathbf{x}$ und $\Delta \mathbf{u}$ um den Arbeitspunkt gilt somit:

$$\mathbf{f}(\bar{\mathbf{x}} + \Delta \mathbf{x}, \bar{\mathbf{u}} + \Delta \mathbf{u}) = \mathbf{f}(\bar{\mathbf{x}}, \bar{\mathbf{u}}) + \underbrace{\frac{\partial \mathbf{f}}{\partial \mathbf{x}} \bigg|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})}}_A \cdot \Delta \mathbf{x} + \underbrace{\frac{\partial \mathbf{f}}{\partial \mathbf{u}} \bigg|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})}}_B \cdot \Delta \mathbf{u} + \dots \quad (6.5)$$

$$\mathbf{g}(\bar{\mathbf{x}} + \Delta \mathbf{x}, \bar{\mathbf{u}} + \Delta \mathbf{u}) = \mathbf{g}(\bar{\mathbf{x}}, \bar{\mathbf{u}}) + \underbrace{\frac{\partial \mathbf{g}}{\partial \mathbf{x}} \bigg|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})}}_C \cdot \Delta \mathbf{x} + \underbrace{\frac{\partial \mathbf{g}}{\partial \mathbf{u}} \bigg|_{(\bar{\mathbf{x}}, \bar{\mathbf{u}})}}_D \cdot \Delta \mathbf{u} + \dots \quad (6.6)$$

Nun können bei einer Verschiebung des Koordinatensystems auf den Arbeitspunkt $(\bar{x}, \bar{u})$ die Differentialgleichungen mit den Jacobi-Matrizen siehe 6.2.1 **A**, **B**, **C** und **D** wie folgt beschrieben werden:

$$\dot{\mathbf{x}} = \mathbf{A} \cdot \mathbf{x} + \mathbf{B} \cdot \mathbf{u} \quad (6.7)$$

$$\mathbf{y} = \mathbf{C} \cdot \mathbf{x} + \mathbf{D} \cdot \mathbf{u} \quad (6.8)$$

6.2.1. Jacobi Matrizen

Die Jacobi-Matrix ist eine Matrix, die die partiellen Ableitungen einer Matrix nach ihren Variablen enthält. Sie stellt die lokale Steigung einer nichtlinearen Funktion in mehreren Richtungen dar. Für eine Funktion $f(x)$ mit m Ausgabekomponenten (Zeilen) und n Eingabekomponenten (Spalten) ist die Jacobi-Matrix definiert als:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \quad (6.9)$$

6.3. Steuerbarkeit

Die Steuerbarkeit (Controllability) eines Systems beschreibt die Fähigkeit, den Systemzustand durch geeignete Wahl der Stellgröße $\mathbf{u}$ zu einem stabilen Endzustand zu steuern. Dies kann durch Rückführung der Stellgröße $\mathbf{u}$ auf den Systemzustand $\mathbf{x}$ über die Matrix **K** erreicht werden:

$$\mathbf{u} = -\mathbf{K} \cdot \mathbf{x} \quad (6.10)$$

durch einsetzen in die Systemgleichung ergibt sich:

$$\dot{\mathbf{x}} = \mathbf{A} \cdot \mathbf{x} + \mathbf{B} \cdot (-\mathbf{K} \cdot \mathbf{x}) = (\mathbf{A} - \mathbf{B} \cdot \mathbf{K}) \cdot \mathbf{x} \quad (6.11)$$

Ein stabiler Systemzustand ist gegeben, wenn alle Eigenwerte der Systemmatrix $(\mathbf{A} - \mathbf{B} \cdot \mathbf{K})$ negative Realteile haben.

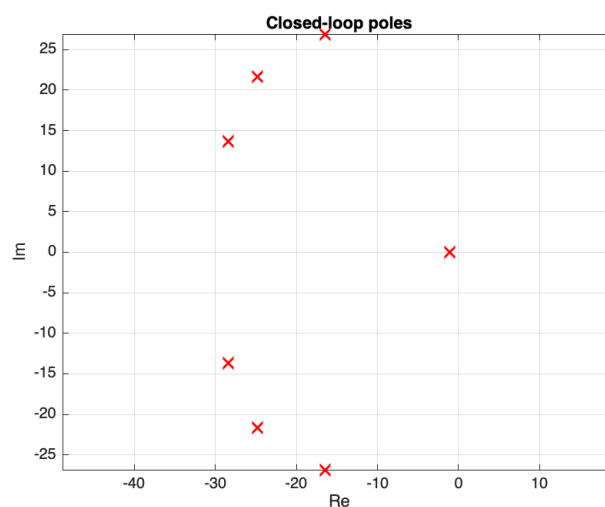


Abbildung 6.2.: Eigenwerte eines stabilen geschlossenen Regelkreises

In Matlab können die Eigenwerte einer Matrix mit der Funktion `eig()` berechnet werden. Ob ein System steuerbar ist, kann durch die Steuerbarkeitsmatrix $\mathcal{C} = [\mathbf{B}, \mathbf{A} \cdot \mathbf{B}, \mathbf{A}^2 \cdot \mathbf{B}, \dots, \mathbf{A}^n \cdot \mathbf{B}]$ überprüft werden, wenn diese Matrix vollen Rang sprich $\text{rank}(\mathcal{C}) = n$ hat, ist das System steuerbar. Hierzu kann in Matlab die Funktion `ctrb(A,B)` welche die Steuerbarkeitsmatrix berechnet und die Funktion `rank()` die den Rang von $\mathcal{C}$ überprüft verwendet werden. [19]

6.4. Beobachtbarkeit

Die Beobachtbarkeit (Observability) eines Systems beschreibt die Fähigkeit, den Systemzustand $\mathbf{x}$ aus den Messungen der Regelgröße $\mathbf{y}$ zu rekonstruieren. Mathematisch gesehen sind sich Steuerbarkeit und Beobachtbarkeit sehr ähnlich, was bereits darauf hindeutet, dass auch zwischen LQR und Kalman-Filter eine enge Verbindung besteht. Ein System ist genau dann beobachtbar, wenn die Beobachtbarkeitsmatrix $\mathcal{O} = [\mathbf{C}, \mathbf{A} \cdot \mathbf{C}, \mathbf{A}^2 \cdot \mathbf{C}, \dots, \mathbf{A}^n \cdot \mathbf{C}]$ vollen Rang n hat. In Matlab kann die Beobachtbarkeitsmatrix mit der Funktion `obsv(A,C)` berechnet werden. [19]

6.5. Linear Quadratic Regulator (LQR)

Zur Auslegung eines geeigneten Zustandsreglers wird ein Linear Quadratic Regulator (LQR) verwendet. Die Grundidee des LQR besteht darin, eine Zustandsrückführung zu bestimmen, die das System stabilisiert und gleichzeitig ein definiertes Optimierungskriterium erfüllt. Dabei wird ein Kompromiss zwischen einer möglichst schnellen Annäherung an den Sollzustand und einem möglichst geringen Stellaufwand gesucht. Der Name setzt sich aus den Begriffen *linear*, da der Regler linear ist $\mathbf{u} = -\mathbf{K}\mathbf{x}$, *"quadratic"*, da die Kostenfunktion J quadratisch ist, und *regulator* zusammen, da es sich um einen Regler handelt.

Unter der Voraussetzung, dass das System steuerbar ist und der vollständige Zustand $\mathbf{x}$ beobachtbar ist, kann eine optimale Rückführungsmatrix $\mathbf{K}$ bestimmt werden. Optimal bedeutet hierbei, dass die folgende Kostenfunktion minimiert wird:

$$J = \int_0^{\infty} (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) dt \quad (6.12)$$

Die Matrizen $\mathbf{Q}$ und $\mathbf{R}$ dienen als Gewichtungsmatrizen und legen fest, wie stark Zustandsabweichungen beziehungsweise Stellaufwand im Optimierungskriterium berücksichtigt werden. Während große Einträge in $\mathbf{Q}$ eine stärkere Bestrafung von Zustandsabweichungen bewirken, führen große Einträge in $\mathbf{R}$ zu einer stärkeren Einschränkung der Stellgrößen.

Im Gegensatz zur direkten Polvorgabe, bei der die Polstellen gezielt gewählt werden, ergibt sich beim LQR die Lage der geschlossenen Polstellen aus der Minimierung der Kostenfunktion. Dadurch entsteht ein sinnvoller Kompromiss zwischen Dynamik und Stellaufwand. [19]

6.5.1. Lösung der algebraischen Riccati-Gleichung

Die optimale Rückführungsmatrix $\mathbf{K}$ wird durch die Lösung der algebraischen Riccati-Gleichung bestimmt. In Matlab kann diese mit der Funktion `lqr(A,B,Q,R)` berechnet werden.

Im Folgenden soll die algebraische Riccati-Gleichung der Kostenfunktion J dem Erkenntnisgewinn

halber gelöst werden, um die optimale Rückführungsmatrix $\mathbf{K}$ zu bestimmen.

$$J = \int_0^{t_f} \underbrace{\frac{1}{2} (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u})}_{\text{Lagrangian } \mathcal{L}} dt + \underbrace{\frac{1}{2} \mathbf{x}_{t_f}^T \mathbf{Q}_f \mathbf{x}_{t_f}}_{\text{Terminal cost}} \quad (6.13)$$

In diesem Punkt wird die Kostenfunktion um eine Terminal Cost erweitert, welche die Kosten am Ende der Regelung zum Zeitpunkt t_f berücksichtigt. Außerdem wird die Kostenfunktion um den Faktor $\frac{1}{2}$ skaliert, um die Ableitungen zu vereinfachen.

$$J_{\text{aug}} = \int_0^{t_f} \frac{1}{2} (\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}) + \lambda^T (\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} - \dot{\mathbf{x}}) dt + \frac{1}{2} \mathbf{x}_{t_f}^T \mathbf{Q}_f \mathbf{x}_{t_f} \quad (6.14)$$

Auf Basis des Zusammenhangs $\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}$ sind $\mathbf{x}$ und $\mathbf{u}$ nicht unabhängig, weshalb die Kostenfunktion um die Lagrange-Multiplikatoren λ erweitert (augmented) wird, um diesen Zusammenhang zu berücksichtigen.

$$\delta J_{\text{aug}} = \int_0^{t_f} \left[\frac{\partial \mathcal{L}}{\partial \mathbf{x}} \delta \mathbf{x} + \frac{\partial \mathcal{L}}{\partial \mathbf{u}} \delta \mathbf{u} + \lambda^T \mathbf{A} \delta \mathbf{x} + \lambda^T \mathbf{B} \delta \mathbf{u} - \lambda^T \delta \dot{\mathbf{x}} \right] dt + \mathbf{Q}_f \mathbf{x}_{t_f} \delta \mathbf{x}_{t_f}. \quad (6.15)$$

Die partiellen Ableitungen der Lagrange-Funktion $\mathcal{L}$ nach $\mathbf{x}$ und $\mathbf{u}$ ergeben sich zu:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{x}^T \mathbf{Q} \quad \text{and} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{u}} = \mathbf{u}^T \mathbf{R}. \quad (6.16)$$

Der Term $\lambda^T \delta \dot{\mathbf{x}}$ kann durch partielle Integration umgeformt werden:

$$-\int_0^{t_f} \lambda^T \delta \dot{\mathbf{x}} dt = -\lambda_{t_f}^T \delta \mathbf{x}_{t_f} + \lambda_0^T \delta \mathbf{x}_0 + \int_0^{t_f} \dot{\lambda}^T \delta \mathbf{x} dt. \quad (6.17)$$

Dabei muss der Term $\lambda_0^T \delta \mathbf{x}_0$ nicht berücksichtigt werden, da die Anfangsbedingung $\mathbf{x}(0) = \mathbf{x}_0$ fest vorgegeben ist, und somit $\delta \mathbf{x}_0 = 0$ gilt.

Durch einsetzen ergibt sich die Variation der augmentierten Kostenfunktion zu:

$$\delta J_{\text{aug}} = \int_0^{t_f} (\mathbf{x}^T \mathbf{Q} + \lambda^T \mathbf{A} + \dot{\lambda}^T) \delta \mathbf{x} dt + \int_0^{t_f} (\mathbf{u}^T \mathbf{R} + \lambda^T \mathbf{B}) \delta \mathbf{u} dt + (\mathbf{x}_{t_f}^T \mathbf{Q}_f - \lambda_{t_f}^T) \delta \mathbf{x}_{t_f} \quad (6.18)$$

Zur Minimierung der Kostenfunktion muss die Variation δJ_{aug} für alle möglichen Variationen $\delta \mathbf{x}$, $\delta \mathbf{u}$ und $\delta \mathbf{x}_{t_f}$ gleich null sein. Dies führt zu den folgenden Gleichungen:

$$\mathbf{x}^T \mathbf{Q} + \lambda^T \mathbf{A} + \dot{\lambda}^T = 0 \quad (6.19)$$

$$\mathbf{u}^T \mathbf{R} + \lambda^T \mathbf{B} = 0 \quad (6.20)$$

$$\mathbf{x}_{t_f}^T \mathbf{Q}_f - \lambda_{t_f}^T = 0 \quad (6.21)$$

Diese Gleichungen können nun verwendet werden, um die optimale Rückführungsmatrix $\mathbf{K}$ zu bestimmen. Zunächst wird die zweite Gleichung nach $\mathbf{u}$ umgestellt:

$$\mathbf{u} = -\mathbf{R}^{-1} \mathbf{B}^T \lambda \quad (6.22)$$

Es wird angenommen, dass die Kostenvariable λ proportional zum Systemzustand x ist, mit der Proportionalitätskonstante P , welche eine symmetrische Matrix ist.

$$\lambda = \mathbf{P}\mathbf{x} \quad (6.23)$$

Durch einsetzen in die Gleichung für $\lambda = \mathbf{P}\mathbf{x}$ und $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ ergibt sich:

$$\dot{\mathbf{P}}\mathbf{x} + \mathbf{P}(\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}) + \mathbf{Q}\mathbf{x} + \mathbf{A}^T \mathbf{P}\mathbf{x} = 0 \quad (6.24)$$

Durch Einsetzen von $\mathbf{u} = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}\mathbf{x}$ ergibt sich die folgende Gleichung:

$$\dot{\mathbf{P}}\mathbf{x} + \mathbf{P}\mathbf{A}\mathbf{x} + \mathbf{A}^T \mathbf{P}\mathbf{x} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P}\mathbf{x} + \mathbf{Q}\mathbf{x} = 0 \quad (6.25)$$

Diese Gleichung muss für alle $\mathbf{x}$ erfüllt sein und kann daher auch als Matrixgleichung geschrieben werden. Unter Vernachlässigung der Terminal Cost und für $t \rightarrow \infty$ verschwindet der Term $\dot{\mathbf{P}}$, und man erhält die algebraische Riccati-Gleichung:

$$\mathbf{P}\mathbf{A} + \mathbf{A}^T \mathbf{P} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} + \mathbf{Q} = 0. \quad (6.26)$$

Durch lösen dieser Gleichung nach $\mathbf{P}$ und einsetzen in die Gleichung für $\mathbf{u}$ ergibt sich die optimale Rückführungsmatrix $\mathbf{K}$ zu:

$$\mathbf{K} = \mathbf{R}^{-1} \mathbf{B}^T \mathbf{P} \quad (6.27)$$

Ein intuitiver Ansatz ist die Frage wie schlimm der Zustand $\mathbf{P}$ aktuell ist, dieser wird auf die Matrix $\mathbf{B}$ projiziert, um zu bestimmen wie stark die Stellgröße $\mathbf{u}$ auf diesen Zustand einwirken kann, und schließlich wird mit der Inversen von $\mathbf{R}$ gewichtet, um den Stellaufwand zu berücksichtigen. [19]

6.6. Kalman-Filter

Das Kalman-Filter ist ein mathematisches Verfahren zur iterativen Schätzung des Zustands eines linearen Systems aus einer Reihe von verrauschten Messungen von indirekten Systemgrößen. Der große Vorteil im Vergleich zu anderen Filtermethoden liegt in der Fähigkeit, in Echtzeit zu arbeiten, was die Anwendung in Inertialnavigationssystemen in Flugzeugen und Raketen ermöglicht. Es wurde ursprünglich von Rudolf E. Kálmán, Richard S. Bucy und Ruslan L. Stratonovich in den 1960er Jahren entwickelt und fand bereits kurz darauf in den Apollo-Missionen der NASA Anwendung zur Navigation, dieser schnelle Sprung von der mathematischen Forschung zur ingenieurstechnischen Anwendung ist außergewöhnlich und zeigt die Effektivität des Verfahrens.

Um den Schätzwert des Systems zu beschreiben werden im Gegensatz zu einfacheren Filtermethoden wie dem gleitenden Mittelwert mehrdimensionale Normalverteilungen verwendet. So können auch die Zusammenhänge von Schätzfehlern verschiedener Systemgrößen berücksichtigt werden um in jedem Zeitschritt eine optimale Schätzung zu erhalten.

Eine Anwendung eines Kalman-Filters macht dann Sinn, wenn erstens eine zugrundeliegende Systemdynamik (physikalische Gesetze) modelliert werden kann und somit eine Vorhersage des Systemzustands möglich ist und zweitens eine direkte Messung der Systemgrößen nicht möglich oder mit Unsicherheiten behaftet ist. Das Kalman-Filter kombiniert diese beiden Informationsquellen optimal um so eine genauere Vorhersage des Systemzustands zu erhalten.

Im Folgenden soll der Kalman-Filter in seiner in der Literatur am häufigsten beschriebenen Form vorgestellt werden: in diskreter Zeit und für lineare Systeme. Der Kalman-Filter wird deshalb detaillierter beschrieben, da eine intuitivere Betrachtung im Vergleich zum Ablauf eines LQR-Controllers

möglich ist. Später wird im Rahmen des Kapitels 10.3 auf die spiegelbildartige Verbindung zwischen LQR und Kalman-Filter eingegangen, um die kaum zu übersehende Ähnlichkeit der beiden Verfahren zu verdeutlichen. [17] [5] [27]

6.6.1. Zustandsraummodellierung

Das Kalman-Filter basiert auf einer Zustandsraummodellierung des Systems, dabei wird der Systemzustand zu einem Zeitpunkt k durch einen mehrdimensionalen Vektor $\mathbf{x}_k$ (Zustandsgleichung) beschrieben, welcher sich aus den Systemgrößen zusammensetzt. Die Dynamik des Systems wird durch die Zustandsübergangsmatrix $\mathbf{F}_k$ beschrieben, welche den Einfluss des vorherigen Zustands $\mathbf{x}_{k-1}$ auf den aktuellen Zustand $\mathbf{x}_k$ modelliert. Um deterministische Steuerungseinflüsse zu berücksichtigen, wird der Steuervektor $\mathbf{u}_k$ mit der Steuerungsmatrix $\mathbf{B}_k$ multipliziert und zum Systemzustand addiert. Zusätzlich wird Prozessrauschen berücksichtigt, welches die Unsicherheiten im Systemmodell als normalverteilte Zufallsvariable $\mathbf{w}_k$ beschreibt.

$$\mathbf{x}_k = \mathbf{F}_k \cdot \mathbf{x}_{k-1} + \mathbf{B}_k \cdot \mathbf{u}_k + \mathbf{w}_k \quad (6.28)$$

Beispielhaft hier angeführt eine eindimensionale Bewegung mit konstanter Geschwindigkeit ohne Prozessrauschen, bei der die Position x_k und die Geschwindigkeit $\dot{x}_k$ des Systems den Systemzustand bilden:

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ \dot{x}_k \end{bmatrix} \quad (6.29)$$

Die Zustandsübergangsmatrix $\mathbf{F}_k$ für dieses System lautet:

$$\mathbf{F}_k = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \quad (6.30)$$

wobei Δt die Zeitdifferenz zwischen den Zeitschritten $k - 1$ und k ist. In diesem Fall wird kein Steuervektor verwendet, also ist $\mathbf{B}_k \cdot \mathbf{u}_k = 0$.

Man könnte die Zustandsgleichung nun wie folgt anschreiben:

$$\begin{bmatrix} x_k \\ \dot{x}_k \end{bmatrix} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_{k-1} \\ \dot{x}_{k-1} \end{bmatrix} = \begin{bmatrix} x_{k-1} + \dot{x}_{k-1} \cdot \Delta t \\ \dot{x}_{k-1} \end{bmatrix} \quad (6.31)$$

Die Beobachtungen der realen Systemgrößen werden durch den Messvektor $\mathbf{z}_k$ beschrieben, welcher sich aus den gemessenen Größen zusammensetzt. Die als linear angenommene Verzerrung zwischen dem Systemzustand und den Messungen wird durch die Beobachtungsmatrix $\mathbf{H}_k$ modelliert, welche den Einfluss des Systemzustands auf die Messungen beschreibt. Zusätzlich wird Messrauschen berücksichtigt, welches die Unsicherheiten in den Messungen als normalverteilte Zufallsvariable $\mathbf{v}_k$ beschreibt.

$$\mathbf{z}_k = \mathbf{H}_k \cdot \mathbf{x}_k + \mathbf{v}_k \quad (6.32)$$

Die Zustandsgleichungen und die Messgleichungen bilden die Grundlage für den Filteralgorithmus und werden als Zustandsraummodellierung bezeichnet.

6.6.2. Vorhersageschritt

Im Vorhersageschritt wird nach der zuvor aufgestellten Zustandsgleichung der vorhergesagte Systemzustand $\mathbf{x}_{k|k-1}$ und die vorhergesagte Kovarianzmatrix¹ $\mathbf{P}_{k|k-1}$ berechnet. Diese Vorhersagen basieren auf dem vorherigen geschätzten Zustand $\mathbf{x}_{k-1}$ und da der Erwartungswert des Prozessrauschens $\mathbf{w}_k$ null ist, wird dieses im Vorhersageschritt nicht berücksichtigt.

$$\mathbf{x}_{k|k-1} = \mathbf{F}_k \cdot \mathbf{x}_{k-1} + \mathbf{B}_k \cdot \mathbf{u}_k \quad (6.33)$$

In diesem Schritt wird ebenfalls die Kovarianzmatrix $\mathbf{P}_{k|k-1}$ der Schätzfehler aktualisiert, welche die Unsicherheiten in der Schätzung des Systemzustands beschreibt. Dabei wird auf Basis der vorherigen Kovarianzmatrix $\mathbf{P}_{k-1}$ und der Zustandsgleichung sowie der Kovarianzmatrix $\mathbf{Q}_k$ des Prozessrauschens $\mathbf{w}_k$ die neue Kovarianzmatrix berechnet.

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \cdot \mathbf{P}_{k-1} \cdot \mathbf{F}_k^T + \mathbf{Q}_k \quad (6.34)$$

6.6.3. Korrekturschritt

Im Korrekturschritt wird die Vorhersage des Systemzustands $\mathbf{x}_{k|k-1}$ anhand der aktuellen Messungen $\mathbf{z}_k$ angepasst, um eine genauere Schätzung zu erhalten. Dazu wird zunächst der Innovationsvektor $\mathbf{y}_k$ berechnet, welcher die Differenz zwischen den tatsächlichen Messungen und den vorhergesagten Messungen darstellt.

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}_k \cdot \mathbf{x}_{k|k-1} \quad (6.35)$$

Anschließend stellt sich die Frage wie stark die Messdaten gewichtet werden sollen, um die Vorhersage zu korrigieren. Diese Gewichtung wird durch die Kalman-Gain-Matrix $\mathbf{K}_k$ beschrieben, welche auf Basis der vorhergesagten Kovarianzmatrix $\mathbf{P}_{k|k-1}$, der Beobachtungsmatrix $\mathbf{H}_k$ und der Kovarianzmatrix $\mathbf{R}_k$ des Messrauschens $\mathbf{v}_k$ berechnet wird.

$$\mathbf{K}_k = \frac{\mathbf{P}_{k|k-1} \cdot \mathbf{H}_k^T}{\mathbf{H}_k \cdot \mathbf{P}_{k|k-1} \cdot \mathbf{H}_k^T + \mathbf{R}_k} \quad (6.36)$$

Mit der Kalman-Gain-Matrix wird nun die Vorhersage des Systemzustands dynamisch korrigiert, um die aktualisierte Schätzung des Systemzustands $\mathbf{x}_k$ zu erhalten.

$$\mathbf{x}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k \cdot \mathbf{y}_k \quad (6.37)$$

Abschließend wird die Kovarianzmatrix der Schätzfehler aktualisiert, um die Unsicherheiten in der aktualisierten Schätzung des Systemzustands zu beschreiben. Das Ziel des Filters ist es, diese Unsicherheiten zu minimieren.

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}_k) \cdot \mathbf{P}_{k|k-1} \quad (6.38)$$

6.6.4. Erweiterter Kalman-Filter

Der erweiterte Kalman-Filter (EKF) ist eine Erweiterung des klassischen Kalman-Filters, die es ermöglicht, nichtlineare Systeme zu modellieren und zu schätzen. Während der klassische Kalman-Filter

¹Kovarianzmatrix - mehrdimensionale Zufallsvariable

auf linearen Zustands- und Messgleichungen basiert, verwendet der EKF eine lokale lineare Approximation dieser Gleichungen durch das Euler-Verfahren.

Die Systemmodellierung erfolgt analog zum klassischen Kalman-Filter, wobei nichtlineare Zustandsübergangs- und Beobachtungsfunktionen mittels einer Linearisierung erster Ordnung approximiert werden. Hierzu werden die Jacobi-Matrizen der nichtlinearen Funktionen an der aktuellen Zustandsabschätzung ausgewertet und anstelle der linearen Matrizen $\mathbf{F}_k$ und $\mathbf{H}_k$ verwendet.

Das nichtlineare Zustandsmodell ist gegeben durch

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \boldsymbol{\omega}_{k-1}), \quad (6.39)$$

wobei $\boldsymbol{\omega}_{k-1}$ das Prozessrauschen beschreibt. Durch Linearisierung um die a-priori-Zustandsschätzung (Vorhersage) $\hat{\mathbf{x}}_{k-1}$ ergibt sich

$$\mathbf{x}_k \approx f(\mathbf{x}_{k|k-1}, \mathbf{0}) + \mathbf{F}_k(\mathbf{x}_{k-1} - \mathbf{x}_{k|k-1}) + \mathbf{W}_k \boldsymbol{\omega}_{k-1}, \quad (6.40)$$

mit den Jacobi-Matrizen

$$\mathbf{F}_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}_{k|k-1}}, \quad \mathbf{W}_k = \left. \frac{\partial f}{\partial \boldsymbol{\omega}} \right|_{\mathbf{x}_{k|k-1}} \quad (6.41)$$

Das nichtlineare Beobachtungsmodell (Messvektor) lautet

$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k), \quad (6.42)$$

wobei $\mathbf{v}_k$ das Messrauschen bezeichnet.

Die Linearisierung um die a-priori-Zustandsschätzung $\mathbf{x}_{k|k-1}$ führt zu

$$\mathbf{z}_k \approx h(\mathbf{x}_{k|k-1}, \mathbf{0}) + \mathbf{H}_k(\mathbf{x}_k - \mathbf{x}_{k|k-1}) + \mathbf{V}_k \mathbf{v}_k, \quad (6.43)$$

mit den Jacobi-Matrizen

$$\mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\mathbf{x}_{k|k-1}}, \quad \mathbf{V}_k = \left. \frac{\partial h}{\partial \mathbf{v}} \right|_{\mathbf{x}_{k|k-1}}. \quad (6.44)$$

[3] Für den Vorherageschritt des EKF ergibt sich somit:

$$\mathbf{x}_{k|k-1} = f(\mathbf{x}_{k-1}, \mathbf{u}_k) \quad (6.45)$$

$$\mathbf{P}_{k|k-1} = \mathbf{A} \cdot \mathbf{P}_{k-1} \cdot \mathbf{A}^T + \mathbf{W} \cdot \mathbf{Q}_k \cdot \mathbf{W}^T \quad (6.46)$$

Für den Korrekturschritt des EKF ergibt sich:

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H}_k \cdot \mathbf{x}_{k|k-1} \quad (6.47)$$

$$\mathbf{K}_k = \frac{\mathbf{P}_{k|k-1} \cdot \mathbf{H}_k^T}{\mathbf{H}_k \cdot \mathbf{P}_{k|k-1} \cdot \mathbf{H}_k^T + \mathbf{V} \cdot \mathbf{R}_k \cdot \mathbf{V}^T} \quad (6.48)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}_k) \cdot \mathbf{P}_{k|k-1} \quad (6.49)$$

$$\hat{\mathbf{x}}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k \cdot \mathbf{y}_k \quad (6.50)$$

Der Ablauf des EKF ist quasi identisch zum klassischen Kalman-Filter. [3][12]

6.6.5. Euler-Verfahren

Das Euler-Verfahren ist ein numerisches Verfahren zur Lösung von gewöhnlichen Differentialgleichungen. Es basiert auf der Idee, die Ableitung einer Funktion an einem bestimmten Punkt zu verwenden, um den Funktionswert an einem nahegelegenen Punkt zu approximieren.

$$y_{k+1} = y_k + h \cdot \dot{y}(t) = y_k + h \cdot f(t_k, y_k) \quad (6.51)$$

wobei y_k der Funktionswert zum Zeitpunkt t_k , h die Schrittweite und $f(t_k, y_k)$ die Ableitung der Funktion zum Zeitpunkt t_k ist, welcher durch $t_k = t_0 + k \cdot h$ definiert ist. k ist eine positive ganze Zahl, welche den Zeitschritt angibt.

6.7. Linear Quadratic Gaussian

An diesem Punkt soll die Verbindung zwischen Linear Quadratic Regulator und Kalman-Filter auch Linear Quadratic Estimator genannt vorgestellt werden: Der Linear Quadratic Gaussian (LQG) Regler kombiniert die Vorteile beider Verfahren, indem er die optimale Zustandsrückführung des LQR mit der optimalen Zustandsabschätzung des Kalman-Filters zu einem optimalen Full State Feedback Regler zusammenführt.

Dabei können die beiden optimalen Verfahren unabhängig voneinander ausgelegt werden und anschließend zusammengeführt werden, bleiben jedoch nach wie vor optimal.

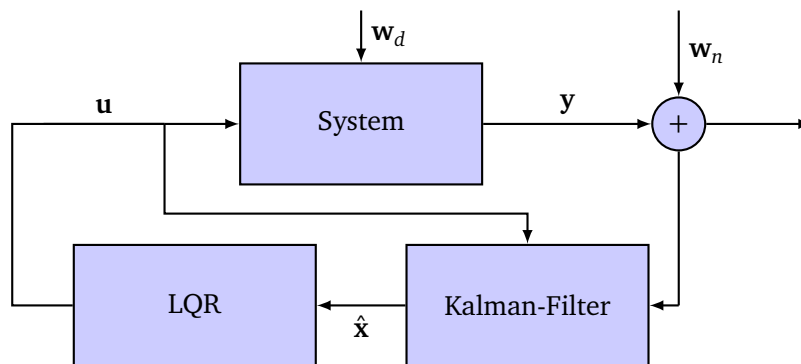


Abbildung 6.3.: LQG control structure

In der Abbildung 6.3 ist die Struktur eines LQG-Reglers dargestellt. Das System empfängt die Steuergröße u und produziert die Ausgangsgröße y . Störgrößen w_d beeinflussen das System, während Messrauschen w_n die gemessenen Signale y stören. Der Kalman-Filter verarbeitet die gemessenen Signale, um eine Schätzung $\hat{x}$ des Systemzustands zu liefern, welche dann an den LQR weitergegeben wird. Der LQR-Block berechnet die optimale Steuergröße basierend auf der geschätzten Zustandsinformation $\hat{x}$.

Das System wird dabei durch folgende bereits bekannte Gleichungen beschrieben:

$$\dot{x} = A \cdot x + B \cdot u + w_d \quad (6.52)$$

$$y = C \cdot x + w_n \quad (6.53)$$

Der LQG-Regler berechnet die Steuergröße u auf Basis der geschätzten Zustandsinformation $\hat{x}$ durch

die folgende Rückführung:

$$\mathbf{u} = -\mathbf{K}_r \cdot \hat{\mathbf{x}} \quad (6.54)$$

Der Kalman-Filter (hier in seiner kontinuierlichen Form auch Kalman-Bucy Filter genannt) berechnet die Schätzung $\hat{\mathbf{x}}$ des Systemzustands basierend auf den gemessenen Signalen $\mathbf{y}$:

$$\hat{\mathbf{x}} = -\mathbf{K}_f \cdot \mathbf{y} \quad (6.55)$$

Die Rückführungsmatrix $\mathbf{K}_r$ des LQR und die Kalman-Gain-Matrix $\mathbf{K}_f$ des Kalman-Filters werden unabhängig voneinander berechnet, wobei $\mathbf{K}_r$ auf der Minimierung der Kostenfunktion des LQR basiert und $\mathbf{K}_f$ auf der Minimierung der Kovarianzmatrix des Kalman-Filters basiert.

Wie in den detaillierten Beschreibungen der beiden Verfahren siehe Kapitel 6.5 und 6.6 beschrieben, sind sich die Berechnungsschritte der Rückführungsmatrix $\mathbf{K}_r$ des LQR und der Kalman-Gain-Matrix $\mathbf{K}_f$ des Kalman-Filters sehr ähnlich, und jeweils auf eine algebraische Riccati-Gleichung zurückzuführen.

Optimal control and estimation are mathematical dual problems, as are controllability and observability. [19]

Die vollständige Systemdynamik des LQG-Reglers kann somit durch die folgende Gleichung beschrieben werden:

$$\dot{\hat{\mathbf{x}}} = \mathbf{A} \cdot \hat{\mathbf{x}} + \mathbf{B} \cdot \mathbf{u} + \mathbf{K}_f (\mathbf{y} - \mathbf{C} \cdot \hat{\mathbf{x}}) \quad (6.56)$$

$$\mathbf{u} = -\mathbf{K}_r \cdot \hat{\mathbf{x}} \quad (6.57)$$

Wenn die Matrix $\mathbf{K}_f$ in der ersten Gleichung groß ist, vertraut der Schätzer stark auf die Messungen, wenn diese jedoch klein ist, verlässt sich dieser stärker auf die Systemdynamik. Dies hängt davon ab wie stark das Messrauschen $\mathbf{w}_n$ im Vergleich zum Prozessrauschen $\mathbf{w}_d$ ist, und somit wie zuverlässig die Messungen sind. [19]kn

7. Aerodynamische Grundlagen

Aerodynamik ist die Wissenschaft von strömenden Gasen, besonders von strömender Luft. Aerodynamik ist ein zentraler Punkt bei der Entwicklung von Flugkörpern. Raketen unterscheiden sich zwar von herkömmlichen Flugkörpern wie Flugzeugen im Aussehen und in aerodynamischen Eigenschaften, werden aber von den gleichen physikalischen Gesetzen und Kräften beeinflusst.

7.1. Die 4 Außenkräfte auf eine Rakete

Kräfte sind Vektorgrößen, die sowohl über eine Richtung als auch über einen Betrag verfügen. Bei der Beschreibung der Wirkung dieser Kräfte muss beides immer berücksichtigt werden. Im Flug wirken auf die Rakete gleichzeitig alle vier Kräfte, wie auf der Abbildung 7.1 zu sehen ist.

7.1.1. Nicht-aerodynamische äußere Kräfte

Die Gewichtskraft hängt von der Masse aller Teile der Rakete ab. Die Gewichtskraft wirkt immer in Richtung des Erdmittelpunkts durch den Schwerpunkt(CG) der Rakete, der mit einem gelben Punkt auf der Abbildung gekennzeichnet ist. Die Größe der Schubkraft hängt vom Massenstrom durch den Motor und von der Geschwindigkeit und dem Druck am Austritt der Düse ab. Die Schubkraft wirkt entlang der Längsachse der Rakete somit auch durch den eigenen Schwerpunkt. In einigen Raketensteuersystemen die Thrust-Vectoring implementieren, wird die Düse abgewinkelt, um die Richtung der Schubkraft und somit auch die Richtung der Flugbahn zu ändern.

Die Gewichtskraft und die Schubkraft sind wie folgt definiert:

$$\vec{F}_{weight} = m \cdot \vec{g} \quad (7.1)$$

$$\vec{F}_{thrust} = \dot{m} \cdot \vec{v}_e + (p_e - p_0) \cdot A_e \quad (7.2)$$

Mit m als Masse der Rakete, $\vec{g}$ als Die Erdbeschleunigung, $\dot{m}$ als Massenstrom durch den Motor, $\vec{v}_e$ als die Austrittsgeschwindigkeit der Gase aus der Düse, p_e als Druck am Düsenaustritt, p_0 als Umgebungsdruck und A_e als Querschnittsfläche am Düsenaustritt.

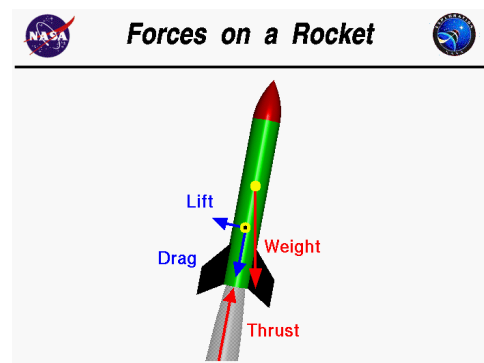


Abbildung 7.1.: Die 4 wichtigsten Kräfte auf einen Raketenflugkörper [10]

Aerodynamische Kräfte

Die Größe der aerodynamischen Kräfte hängt sowohl von der Größe und Form der Rakete, als auch von der Geschwindigkeit und der Eigenschaften der Atmosphäre ab. Diese Kräfte wirken durch den Druckmittelpunkt (CP) der Rakete, welcher mit einem Gelb-schwarzen Punkt auf der Abbildung gekennzeichnet ist. Aerodynamische Kräfte sind ein zentraler Punkt bei der Entwicklung von Modellraketen, für große Raketen sind diese möglicherweise weniger relevant, da sie in der Regel nur eine kurze Zeit in der Atmosphäre verbringen.

Diese zwei aerodynamischen Kräfte sind die Widerstandskraft und die Auftriebskraft. Diese sind wie folgt definiert:

$$\vec{F}_{drag} = C_d \cdot \frac{\rho \cdot A \cdot \vec{v}^2}{2} \quad (7.3)$$

$$\vec{F}_{lift} = C_l \cdot \frac{\rho \cdot A \cdot \vec{v}^2}{2} \quad (7.4)$$

Mit C_d als Widerstandsbeiwert, C_l als Auftriebsbeiwert, ρ als Luftdichte, A als Querschnittsfläche der aerodynamischen Features der Rakete und $\vec{v}$ als Geschwindigkeit der Rakete relativ zur Luft. Die Widerstandskraft und die Auftriebskraft hängen wie folgt zusammen:

$$\frac{\vec{F}_{drag}}{\vec{F}_{lift}} = \frac{C_d}{C_l} \quad (7.5)$$

7.1.2. Äußere Faktoren

Während des gesamten Fluges verändern sich die Kräfte andauernd, sowohl in ihrem Betrag als auch in ihrer Richtung. Wenn man die Kräfte unter Berücksichtigung ihrer Richtung addiert, erhält man eine externe resultierende Kraft auf die Rakete. Die durch diese Kraft verursachte Bewegung der Rakete kann durch Newtonsche Bewegungsgesetze beschrieben werden:

Erstes Newtonsches Gesetz

Ein Kräftefreier Körper bleibt in Ruhe oder bewegt sich mit konstanter Geschwindigkeit.

Zweites Newtonsches Gesetz

Kraft ist gleich Masse mal Beschleunigung.

$$\vec{F} = m \cdot \vec{a} \quad (7.6)$$

Drittes Newtonsches Gesetz

Kraft ist gleich Gegenkraft. Für eine Kraft von einem Körper auf einen anderen Körper wird eine Gegenkraft ausgeübt, die gleich groß ist und in entgegengesetzter Richtung wirkt.

$$\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A} \quad (7.7)$$

7.2. Aerodynamische Eigenschaften

Die aerodynamischen Kräfte auf eine Rakete hängen direkt von der Form der Rakete ab. Entscheidende aerodynamische Eigenschaften sind die Nase der Rakete und die Flügel. Das Profil dieser Eigenschaften hängt vor allem davon ab, ob die Rakete die Schallgeschwindigkeit überschreitet oder nicht.

7.2.1. Überschallflug und Unterschallflug

Überschallflug

Bei Überschreitung der Schallgeschwindigkeit ($Mach > 1$) bewegt sich die Rakete schneller durch die Luft, als diese sich mit der Rakete mitbewegen kann. Durch diesen enormen Druckanstieg vor der Rakete entsteht eine Schockwelle, die sich in einer kegelförmigen Struktur hinter der Rakete ausbreitet, wie auf der Abbildung 7.2 zu sehen ist. In diesem Fall sind die aerodynamischen Eigenschaften so zu formen, dass die Schockwelle so weit nach hinten gelenkt wird, wie möglich, um den Luftwiderstand unter diesen Bedingungen so gering wie möglich zu halten. Dadurch entstehen die typischen spitzen Nasen und scharfen Flügelprofile von Überschallraketen.

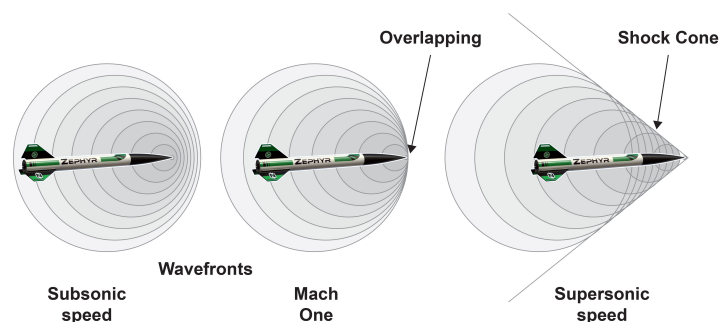


Abbildung 7.2.: Machscher Kegel [26]

Unterschallflug

Überschreitet die Rakete jedoch an keinem Zeitpunkt während des Fluges die Schallgeschwindigkeit, können die aerodynamischen Eigenschaften nach normalen aerodynamischen Konzepten geformt werden, um den Luftwiderstand zu minimieren und (falls erwünscht) den Auftrieb zu maximieren.

7.2.2. Raketenspitze

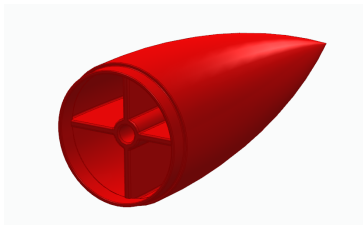


Abbildung 7.3.: Raketenspitze

Bei "ALBERT" hat die Raketenspitze die Form einer tangentialen Ovale. Das ist die am meisten verbreitete Form für die Spitze von Modellraketen, da sie bei kleinen Mach-Zahlen einen geringen Luftwiderstand bietet und gleichzeitig einfach zu bauen ist. Die Spitze ist innen hohl, um die Masse zu reduzieren, hat aber innen eine Verstärkung, um die strukturelle Integrität zu gewährleisten und eine ausreichend stabile Verbindung zum Seil des Fallschirms zu bieten. Abbildung 7.3 zeigt die Form der Raketenspitze von "ALBERT". Beim Aktivieren des Bergungssystems trennt sich die Spitze von der Rakete, daher wird diese mit einem Schochseil mit dem Rest der Rakete

und dem Fallschirm verbunden.

7.2.3. Raketenflügel

Raketenflügel, auch als Flossen oder Finnen bezeichnet, sind aerodynamische Oberflächen, die Normal zur Längsachse der Rakete befestigt werden. Diese dienen dazu, die Rakete während des Fluges zu stabilisieren und bei Bedarf Auftrieb zu erzeugen.

Anzahl der Flügel

Die Anzahl der Flügel an einer Rakete variiert, aber die meisten Modellraketen haben entweder 3 oder 4 Flügel, außer die Rakete verfügt sowohl über frontale als auch über hintere Flügel. Es werden meistens 4 Flügel verwendet, aber wenn der Luftwiderstand ganz gering gehalten werden soll, können auch nur 3 Flügel in Frage kommen.

Position der Flügel

Flügel können an verschiedenen Positionen entlang vom Raketengehäuse angebracht werden, am verbreitetsten sind jedoch Raketen mit entweder nur hinteren Flügeln oder mit sowohl frontalen als auch hinteren Flügeln. Bei aktiven Steuersystemen bieten bewegliche frontale Flügel den Vorteil der besseren Manövrierfähigkeit, sorgen aber für weniger Stabilität und benötigen komplexere Regelungstechnik.

Feste und bewegliche Flügel

Die Flügel einer Rakete können entweder fest oder beweglich sein, je nachdem ob die Rakete über ein aktives Steuersystem verfügt, oder ob die Flügel ausschließlich zur passiven Stabilisierung dienen.

Flügelform

Ein wichtiger Punkt beim Formen der Flügel ist die Auswahl der Form. Bei Modellraketen stehen grundsätzlich wie auf der Abbildung 7.4 zu erkennen ist 8 Formen zur Auswahl mit den jeweiligen Vor- und Nachteilen in 7.1.

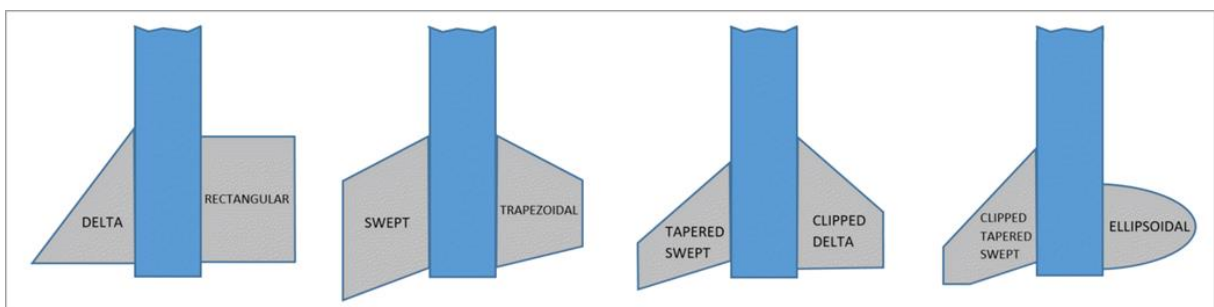


Abbildung 7.4.: Verschiedene Flügelformen [18]

Tabelle 7.1.: Vor- und Nachteile verschiedener Flügelformen

Form	Vorteile	Nachteile
Delta	Flatterrobust, gut für hohe Geschwindigkeiten	Spitze Kante anfällig für Beschädigungen
Rechteckig	Einfach zu bauen, strukturell robust	Kann bei Landung beschädigt werden, nicht optimal für Finnensteuerung
Gepfeilt	Flatterrobust, strukturell robust	Kann bei Landung beschädigt werden, nicht optimal für Finnensteuerung
Trapezförmig	Strukturell robust, kann gestützt werden um den Druckpunkt nach vorne zu verlagern	Weniger aerodynamisch als gepfeilte Formen
Trapezförmig gepfeilt	Gut für hohe Geschwindigkeiten, kann gestützt werden um den Druckpunkt nach vorne zu verlagern	Anfällig für Flattern und Beschädigungen bei Landung
Gestütztes Delta	Strukturell robust, gute Wahl für Finnensteuerung	Kann bei der Landung beschädigt werden
Gestütztes gepfeiltes Trapez	Eine Verbesserung des einfach gepfeilten Trapezes	Anfällig für Flattern und Beschädigungen bei Landung
Elliptisch	Aerodynamisch effizient, robust, flattersicher	Schwer herzustellen, Berechnung des Druckpunkts kompliziert

Flügelprofil

Das Profil der Raketenflügel hängt am meisten davon ab, ob die Rakete die Schallgeschwindigkeit überschreitet oder nicht. Wenn Überschallflug vorliegt, muss das Profil so geformt werden, dass die vordere Kante der Flügel scharf ist, um die Schockwelle nach hinten zu lenken und um weniger Luft zu komprimieren. Bei Unterschallflug werden Flügel in Form von einem symmetrischen Aerofoil Profil geformt, um gleichzeitig den Luftwiderstand zu minimieren und bei aktiven Steuersystemen den benötigten Auftrieb zu erzeugen. Auf der Abbildung 7.5 sind Beispiele von Flügelprofilen zu sehen, sowohl für Unterschall- aus auch für Überschallflug. Es ist auch möglich ein unsymmetrisches Profil zu verwenden, um die Rakete zum drehen zu bringen, was mit Stabilität bei Raketen ohne aktivem Steuersystem helfen kann.

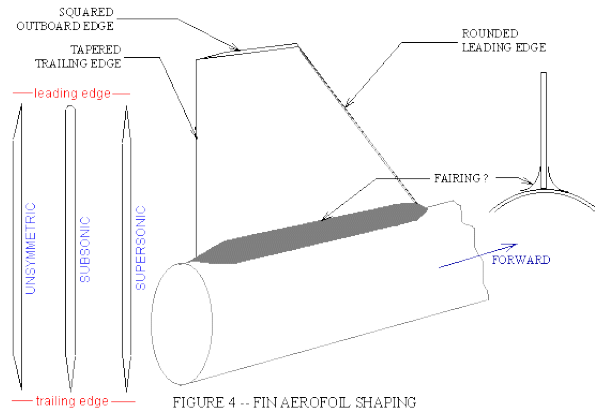


Abbildung 7.5.: Flügelprofile [18]

7.2.4. NACA Flügelprofile

Für Aerofoil Profile werden oft die NACA Profile verwendet. NACA Profile sind standardisierte Profile, die von der National Advisory Committee for Aeronautics (NACA) entwickelt wurden, welche später in die NASA überging.

Für symmetrische Flügelprofile für Unterschallflug werden meistens die vierstelligen NACA Profile verwendet. Die vierstellige NACA Serie umfasst 78 im Windkanal getestete Profile. Diese Profile wurden recht willkürlich ausgewählt, aber manche dieser Profile werden noch heute von Flugzeugherstellern verwendet. Das Konstruktprinzip der NACA Profile beruht auf Kreisen, die auf der Profilmittellinie gezeichnet werden.

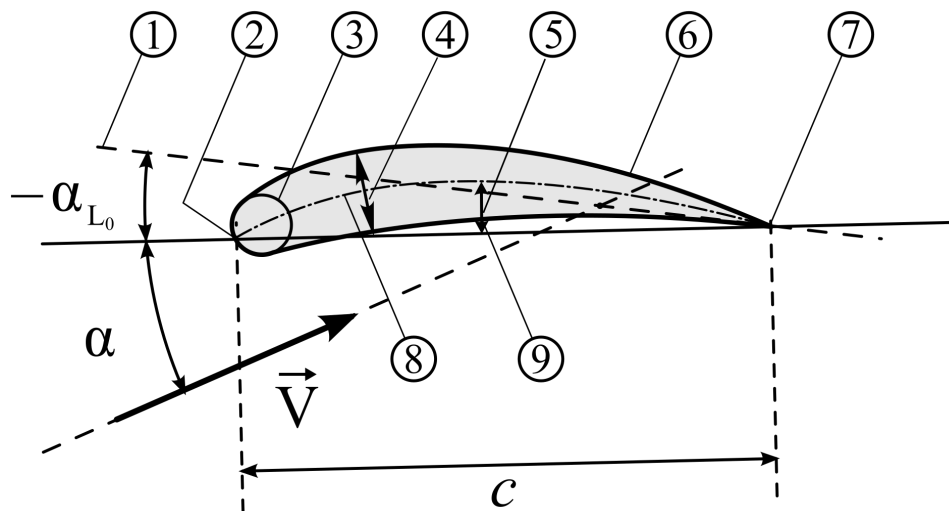
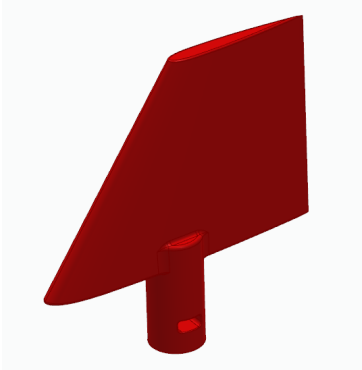


Abbildung 7.6.: Geometrie der NACA Profile [28]

Auf der Abbildung 7.6 ist die Geometrie der NACA Profile mit den folgenden Parametern zu sehen:

1. Nullauftriebslinie
2. Flügelnase
3. Krümmungsradius der Flügelnase
4. Wölbung maximale Profildicke
5. Wölbung
6. obere Profilseite
7. Profilhinterkante
8. Skelettlinie
9. untere Profilseite

7.2.5. Flügeldesign



Die Flügel von “ALBERT” haben die Form eines gestutzten Deltas, da diese Form sowohl strukturell robust ist, als auch eine gute Wahl für Finnensteuerung darstellt. Beim Profil der Flügel von “ALBERT” wurde das NACA0012 Profil gewählt, da dieses Profil symmetrisch ist, was keinen Auftrieb bei einem Anstellwinkel von 0° erzeugt, die Auftriebskennlinie des Profils für kleine Anstellwinkel relativ linear ist, was die Regelung der Flügel einfacher macht. Die vierstelligen NACA Profile existieren schon sehr lange, daher gibt es gute Daten über die aerodynamischen Eigenschaften, was das überprüfen von Simulationsergebnissen und Versuchen erleichtert. Abbildung 7.7 zeigt das CAD Modell der Flügel.

Abbildung 7.7.: Flügel CAD Modell

8. Strömungssimulation und Messung im Windkanal

8.1. CFD Simulation

8.1.1. Grundlagen von CFD

Computational Fluid Dynamics (CFD) ist ein Zweig der Fluidmechanik, der numerische Methoden und Algorithmen verwendet, um die Strömung von Flüssigkeiten und Gasen zu analysieren und vorherzusagen. CFD ermöglicht es Ingenieuren und Wissenschaftlern, komplexe Strömungsphänomene zu simulieren, die in der realen Welt auftreten, ohne physische Experimente durchführen zu müssen. CFD wird in einer Vielzahl von Anwendungen eingesetzt, darunter die Luft- und Raumfahrt, die Automobilindustrie, die Energieerzeugung und die Umweltwissenschaften.

8.1.2. Ansys Fluent

Die CFD Simulation der Rakete "ALBERT" wurde mit Student Version von Ansys Fluent durchgeführt, einem der führenden Softwarepakete für Computational Fluid Dynamics. Die Simulation läuft in mehreren Schritten ab.

Geometrie

Die zu simulierende Geometrie wird von der CAD Software in Ansys Workbench als .step Datei importiert und anschließend angepasst. Es müssen Grenzen definiert werden und die Geometrie anschließend von den Grenzen subtrahiert werden, damit ein einzelner fester Körper entsteht.

Meshing

Nachdem die Geometrie erfolgreich definiert wurde, muss diese in ein Mesh umgewandelt werden, wo die Oberflächen in viele, je nach Komplexität der Oberfläche, kleine oder große Polygone aufgeteilt werden. Für CFD werden meistens dreieckige Polygone verwendet, dies hängt aber natürlich von der Form der Geometrie ab.

Einstellen der Simulation

Bevor Ansys Fluent die tatsächliche Simulation durchführt, müssen mehrere Sachen definiert werden.

Als erstes werden bestimmte Eigenschaften des Solvers, darunter ob die Berechnung druckbasiert oder dichtebasiert verlaufen soll und ob die Strömungsgeschwindigkeit absolut oder relativ formuliert wird.

Gleich als nächstes folgt die Modellauswahl, diese hängt davon ab, was man berechnen will und welche Gleichungen Fluent im Hintergrund bei der Simulation rechnet. Darunter:

- Multiphase: Wenn mehrere Zustände gleichzeitig vorhanden sind (z.B. Wasser + Luft)
- Energy: Temperatur wird berechnet, Erwärmung Kühlung etc.
- Viscous Laminar/Turbulent: Bestimmt wie "unruhig" die Strömung verläuft
- Radiation: Wärmestrahlung wird bei der Simulation berücksichtigt
- Heat Exchanger: Dient zur vereinfachten Modellierung von Wärmetauschen
- Species: Mehrere Stoffe in einem Fluid (z.B. Luft + CO₂)
- Discrete Phase: Einzelne Partikel wie Staub oder Tropfen in der Luft werden berücksichtigt
- Solidification & Melting: Das Material kann schmelzen/gefrieren
- Acoustics: Zur Lärmanalys, berücksichtigt Geräusche/Druckwellen
- Structure: Wenn der feste Körper sich zusätzlich durch die Strömung bewegt
- Electric Potential: Elektrische Felder sind in der Strömungsanalyse zu berücksichtigen

Die Materialien des festen Körpers und des Fluids werden anschließend festgelegt. Als nächstes werden die Grenzbedingungen der Simulation definiert. Darunter der Einlass der Strömung, wo die Strömung austritt, das durchgehende Volumen durch das die Strömung fließt, und Wände (die äußeren Grenzen und der in der feste Körper in der Simulation). Diese Grenzbedingungen können noch einzeln eingestellt werden, so wie die Geschwindigkeit und die Turbulenzintensität beim Strömungseinlass.

Ein weiterer zentraler Punkt sind die Referenzwerte. Diese sind:

- Fläche: Referenzfläche für die Strömung, ist für die Berechnung von Kraft- und Momentkoeffizienten wichtig
- Tiefe: Referenztiefe für 2D Simulationen
- Dichte: Referenzwert der Dichte zur Berechnung vom dynamischen Druck
- Enthalpie: Referenzwert für die globale Differenz der Energiedichte
- Länge: Charakteristische Länge, Referenzpunkt für Momentkoeffizienten
- Druck: Referenzpunkt für Druckbezogene Berechnungen, Moment- und Druckkoeffizienten
- Temperatur: Temperaturreferenzpunkt für thermische Berechnungen
- Geschwindigkeit: Referenzgeschwindigkeit zur Berechnung vom dynamischen Druck
- Viskosität: Dient zur Berechnung von Reynold's Zahl an Grenzen
- Verhältnis der spezifischen Wärmekapazitäten: Referenzwert für Effizienzberechnungen
- γ - Wert für die Berechnung der Wärmeübergangskoeffizienten

Danach kommen Einstellungen für die Lösungsmethoden.

Auswertung der Simulation

8.1.3. CFD Berechnung der Kräfte/Koeffizienten

Um die Regelung vom Steuersystem der Rakete zu realisieren, sind Kraft-/Momentkoeffizienten und dessen Ableitungen nötig. CFD ermöglicht es, diese bei verschiedenen Anstellwinkeln der Flügel zu erfassen, ohne aufwendige und teure Messungen durchführen zu müssen oder ungenaue, sehr annähernde Berechnungen mit der Hand zu machen.

Teil III.

Praktische Umsetzung

9. Flightcomputer KOMADORI

9.1. Anforderungsanalyse

In der Planungsphase des Flightcomputers ist es entscheidend, die Anforderungen an das System klar zu definieren, um sicherzustellen, dass der entwickelte Flightcomputer alle zukünftigen Anforderungen erfüllt.

Die Anforderungen wurden in aktive und passive Anforderungen unterteilt. Aktive Anforderungen beziehen sich auf die Funktionen, die der Flightcomputer erfüllen muss, wie z.B. die Fähigkeit, Sensordaten zu erfassen, die Steuerungsalgorithmen schnell genug auszuführen und die Kommunikation mit anderen Systemen zu ermöglichen. Passive Anforderungen beziehen sich auf die Eigenschaften des Flightcomputers und dessen Bauteile, wie den Formfaktor und die Zuverlässigkeit in stark beschleunigten Umgebungen mit Vibrationen.

Aktive Anforderungen:

- Erfassung folgender Sensordaten zur Bestimmung von Fluglage, Geschwindigkeit, Höhe sowie Position:
 - Beschleunigung mittels 3-Achsen-Beschleunigungssensor
 - Winkelgeschwindigkeit mittels 3-Achsen-Gyroskop
 - Magnetfeldmessung zur Ausrichtungsbestimmung mittels 3-Achsen-Magnetometer
 - Luftdruckmessung zur Höhenbestimmung mittels Barometer
 - Positionsbestimmung mittels GNSS-Modul
- Bereitstellung von PWM-Ausgängen zur Ansteuerung der Servoantriebe der Finnensteuerung.
- Bereitstellung von Pyro-Kanälen zur Zündung des Fallschirmauswurfmechanismus.
- Telemetrieübertragung über LoRa zur Datenkommunikation mit der Bodenstation während des Fluges.
- Speicherung der Flugdaten auf einem SD-Kartenmodul zur nachträglichen Analyse.
- Bereitstellung von Kommunikationsschnittstellen (SPI, I²C und UART) zur Integration externer Sensoren und Aktuatoren.
- Versorgung über einen 2S-LiPo-Akku mit einem Spannungsbereich von 7,4 V bis 8,4 V.
- Möglichkeit zur Programmierung sowie zum Datenlogging über eine USB-C-Schnittstelle.
- Unterstützung von Debugging und Fehlersuche über eine SWD-Schnittstelle (Serial Wire Debug) mittels ST-Link.
- Einhaltung einer maximalen Latenzzeit von unter 10 ms zwischen Messwerterfassung und Steuerungsaktion, um eine ausreichend schnelle Reaktion auf Flugbedingungen sicherzustellen.

Passive Anforderungen:

- Kompakter Formfaktor mit maximalen Abmessungen von 110 mm × 78 mm zur Integration in die Rakete.
- Auswahl geeigneter Sensoren für Beschleunigungen von bis zu 15 g, um den mechanischen Belastungen während des Betriebs standzuhalten und valide Messwerte zu gewährleisten.
- Verwendung vibrationssicherer und zuverlässiger Steckverbinder zur Gewährleistung stabiler Verbindungen zu externen Sensoren, Aktuatoren sowie der Stromversorgung.
- Implementierung von ESD-Schutzmaßnahmen zum Schutz vor elektrostatischer Entladung.
- Ausführung der USB-C-Schnittstelle mit geeigneten Schutzmaßnahmen zum Schutz angeschlossener Endgeräte vor elektrischen Fehlzuständen, Überspannungen und Fehlströmen.

9.2. Systemarchitektur

Auf Basis der Anforderungen an die Funktionen und Eigenschaften des Flightcomputers wird im folgenden die Systemarchitektur ausgearbeitet (siehe Abbildung 9.1)um die Interaktion der verschiedenen Komponenten untereinander im Bezug auf ihre Funktionen zu strukturieren und so eine Grundlage für die weiteren Entwicklungsschritte zu schaffen.

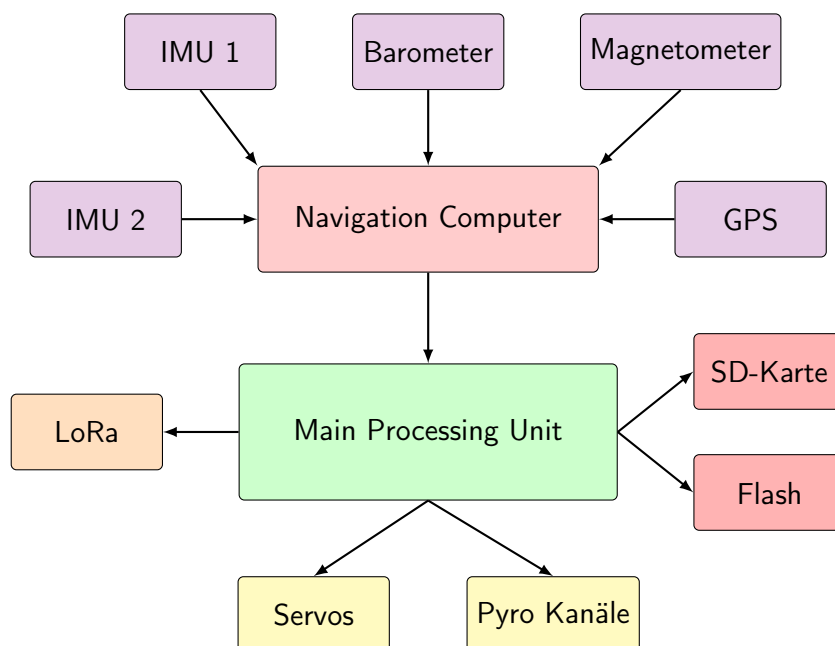


Abbildung 9.1.: Systemarchitektur des Flightcomputers

Die Hardwarearchitektur des Flightcomputers teilt sich in zwei Hauptkomponenten auf: Navigation Processing Unit (NAV) mit den Sensoren und Main Processing Unit (MPU) mit der Aktuatorik, den Datenspeichern sowie der Telemetrie (LoRa). Die NAV ist für die Erfassung der Sensordaten und die Berechnung der Fluglage, Geschwindigkeit sowie Position verantwortlich. Er empfängt Daten von den Sensoren (IMU 1, IMU 2, Barometer, Magnetometer und GPS) und verarbeitet diese mittels Sensor Fusion (siehe Kapitel 6.6.4).

Wohingegen die MPU die zentrale Steuereinheit des Flightcomputers darstellt, welche die Steue-

rungsalgorithmen ausführt, die Aktuatoren (Servos und Pyro-Kanäle) ansteuert, die Flugdaten auf der SD-Karte und im Flash-Speicher speichert sowie die Telemetriedaten über LoRa überträgt. Die MPU empfängt die Zustandsschätzung vom NAV und verarbeitet diese mittels LQR Regelung (siehe Kapitel ??), um die erforderlichen Steuerbefehle für die Servos zu berechnen.

Die Trennung der Aufgabengebiete dient primär der Echtzeitsicherheit und funktionalen Entkoppelung. Der NAV verarbeitet hochfrequente Sensordaten (insbesondere IMU) und schätzt den aktuellen Zustand aus Basis der Sensordaten mittels Extended Kalman Filter. Diese Berechnungen erfordern konstante Latenzen und dürfen nicht durch blockierende Operationen wie Speicherzugriffe oder Funkkommunikation beeinflusst werden. Durch die Auslagerung von potenziell nichtdeterministischen Aufgaben auf eine separate MPU wird sichergestellt, dass die Zustandsschätzung auch bei hoher Systemlast stabil bleibt. Darüber hinaus reduziert die funktionale Trennung die Softwarekomplexität.

Um Flugdaten sicher zu speichern werden diese während des Fluges im Flash-Speicher gespeichert und erst nach der Landung auf die SD-Karte übertragen. Dies verhindert Datenverluste da unter starken Vibrationen ein Datentransfer über die Kontakte der SD-Karte nicht garantiert werden kann.

9.3. Komponentenauswahl

Im folgenden soll die Auswahl der wichtigsten Komponenten des Flightcomputers erläutert werden und die Gründe für die Auswahl dargelegt werden. Für die spätere Treiberentwicklung gilt es bereits bei der Komponentenauswahl darauf zu achten, dass umfangreiche Dokumentationen und optimalerweise Referenztreiber vorliegen, da dies den Zeitaufwand später deutlich verkürzt. Desweiteren ist es essentiell darauf zu achten, dass die ausgewählten Komponenten bei dem gewählten Platinenhersteller verfügbar sind.

9.3.1. Sensoren

IMU1 - BMI088

Das BMI088 ist speziell für hochdynamische Anwendungen mit starken Vibrationen konzipiert und eignet sich daher ideal für den Einsatz in Raketen. Es beinhaltet einen 3-Achsen-MEMS-Beschleunigungssensor mit einem Messbereich von bis zu $\pm 24\text{ g}$ und einen 3-Achsen-Gyroskop mit einem Messbereich von bis zu $\pm 2000^\circ/\text{s}$. Diese Spezifikationen decken den Arbeitsbereich des Flightcomputers mit ausreichend Sicherheitsreserven ab und verhindern bei starken Beschleunigungen am Start das Abschneiden von Signalen außerhalb des Messbereichs.

IMU2 - LSM6DSLTR

Aufgrund der immensen Wichtigkeit der IMU-Daten für die Lagebestimmung des Systems, wird eine Inertial Measurement Unit als Backup verwendet. Der LSM6DSLTR bietet mit einem Messbereich von bis zu $\pm 16\text{ g}$ für den Beschleunigungssensor und $\pm 2000^\circ/\text{s}$ für das Gyroskop ausreichend Reserven und kann in den Sensor Fusion Algorithmus integriert werden, um die Robustheit der Zustandsschätzung zu erhöhen.

Barometer - BMP390

Der Drucksensor wird für die Höhenbestimmung verwendet. Mit einem Arbeitsbereich von 300-1250 hPa und einer Genauigkeit von 0.50 hPa erfüllt der BMP390 die Anforderungen problemlos und kann als zusätzlicher Stützwert für die Zustandsschätzung sowie als verlässliche Entscheidungsgrundlage für den Fallschirmauswurf dienen.

Magnetometer - BMM150

Das Magnetometer wird zur Bestimmung der Ausrichtung der Rakete im Bezug auf den Referenzrahmen der Erde verwendet. Dabei ist es wichtig, zu beachten, dass die Messungen durch magnetische Störfelder auf der Platine beeinflusst und somit verfälscht werden können. Das BMM150 bietet einen Messbereich von $\pm 1300 \mu\text{T}$ und einer Auflösung von $0.3 \mu\text{T}$.

9.3.2. Prozessoren

Für die Auswahl der beiden Prozessoren des Flightcomputers – der Navigation Processing Unit (NAV) sowie der Main Processing Unit (MPU) – wurde besonderer Wert auf eine umfassende technische Dokumentation, die zuverlässige Verfügbarkeit über den Leiterplattenfertiger JLCPCB sowie eine breite communitybasierte Nutzung gelegt.

Zum Zeitpunkt des Projektbeginns beschränkte sich die schulische Ausbildung im Bereich der Mikrocontroller-Programmierung ausschließlich auf die Verwendung der Arduino-Plattform. Eine vertiefte Auseinandersetzung mit hardwarenaher Embedded-Programmierung, eigenständigem Leiterplattendesign oder der Entwicklung komplexerer Systeme war im bisherigen Unterricht nicht vorgesehen und musste sich in Eigeninitiative angeeignet werden.

Aus den oben genannten Gründen wird für die Navigation Processing Unit (NAV) der STM32F405RGT6 von STMicroelectronics ausgewählt, da dieser Mikrocontroller eine leistungsstarke Arm Cortex-M4 CPU mit einer Taktfrequenz von bis zu 168 MHz bietet, was ausreichend Rechenleistung für die Echtzeitverarbeitung der Zustandsschätzung gewährleistet sowie genügend Schnittstellen zur Anbindung von Sensoren bereitstellt.

Für die Main Processing Unit (MPU) wird der STM32H753VIT6 ausgewählt, da dieser mit einer ARM Cortex-M7 CPU mit einer Taktfrequenz von bis zu 400 MHz eine deutlich höhere Rechenleistung bietet, die für die Regelung, die Verwaltung der Peripherie und die Telemetrieübertragung in Echtzeit mehr als ausreichend ist und darüber hinaus zusätzliche Pins für externe Peripherie bietet.

Der zentrale Vorteil der STM32-Plattform liegt in der stark ausgeprägten Community-Unterstützung. Neben der offiziellen Dokumentation stehen zahlreiche technisch hochwertige, communitygetriebene Lernressourcen zur Verfügung, welche die eigenständige Einarbeitung in komplexere Themenbereiche um ein vielfaches erleichtern.

9.3.3. Weitere Peripheriekomponenten

Servos - BMS-125WV+

Für die Ansteuerung der Finnen sind Servos notwendig, welche die erforderlichen Kräfte und Geschwindigkeiten bereitstellen können, zugleich aber auch kompakt genug sind um in den Rumpf zu passen. Die BMS-125WV+ Servos bieten mit einem Stellmoment von 7,1 kg/cm sowie einer Geschwindigkeit von $0.09 \text{ s}/60^\circ$ bei 8.4 V ausreichend Leistung für die Finnensteuerung und sind mit

Abmessungen von 23.2 mm × 10 mm × 23 mm ideal für die Integration in die Rakete. Die Servos werden per Pulsweitenmodulation (PWM) über den PWM-Controller angesteuert.

PWM-Controller - PCA9685PW,118

Um acht Servos anzusteuern, ist die Implementierung eines PWM-Controllers sinnvoll, da sich so die Anzahl der belegten Pins der MPU reduziert und die Ansteuerung der Servos vereinfacht wird. Die Steuerbefehle werden dabei per I²C-Schnittstelle von der MPU übermittelt. Der PCA9685PW,118 bietet 16 Kanäle mit 12-Bit-Auflösung und ist mit einem Betriebsspannungsbereich von 2.3 V bis 5.5 V kompatibel mit der Logikspannung der MPU.

Pyro-Kanäle-Mosfets - BSC028N06LS3

Um den Fallschirm auszuwerfen, werden Pyro-Kanäle benötigt, welche die pyrotechnischen Ladungen zünden. Die BSC028N06LS3 N-Kanal-MOSFETs bieten mit einem maximalen Drain-Strom von 28 A und einer maximalen Drain-Source-Spannung von 60 V mehr als genug Leistung für die Zündung und können direkt über Logic Level sprich die MPU angesteuert werden.

Buck Converter - TPS54308

Um die Komponenten des Flightcomputers mit der benötigten Spannung von 3.3 V zu versorgen, wird ein Buck Converter benötigt, welcher die Eingangsspannung von 7.4-8.4 V des 2S-LiPo-Akkus auf die benötigte Spannung herunterregelt. Der TPS54308 bietet mit einem Eingangsspannungsbereich von 5.5 V bis 36 V und einem Ausgangsstrom von bis zu 3 A ausreichend Leistung für die Versorgung aller Komponenten des Flightcomputers.

Auf die Auswahl der Komponenten für die SD-Karte, den Flash-Speicher, des LoRa-Moduls, des GPS-Moduls sowie des PWM-Controllers wird an dieser Stelle nicht weiter eingegangen, da diese auf Basis der Verfügbarkeit bei JLCPCB, den bereits beschriebenen Anforderungen sowie der vorhandenen Dokumentation ausgewählt wurden und eine detaillierte Erläuterung dem Kapitel keinerlei Mehrwert bieten würde.

9.4. Schaltungsentwurf

Im folgenden wird der Entwicklungsprozess und der Aufbau der Schaltung näher erläutert. Dabei wird auf die Versorgungskonzept, die Sensorik sowie die Prozessoren genauer eingegangen. Für die vollständigen Schaltpläne wird auf den Anhang verwiesen (siehe Anhang ??), da diese aufgrund der Komplexität und des Umfangs an dieser Stelle nicht sinnvoll dargestellt werden können.

9.4.1. Sensoren

Den wichtigsten Teil der Sensorik stellt die IMU1 (BMI088) dar, welche die Hauptquelle für die Daten der Zustandsschätzung ist. Für eine möglichst hohe Datenrate und damit eine möglichst genaue Zustandsschätzung wird die IMU1 über die SPI-Schnittstelle an die Navigation Processing Unit angebunden, da diese im Vergleich zu I²C eine höhere Datenrate ermöglicht. Die IMU2 (LSM6DSLTR) sowie das Magnetometer (BMM150) werden über die I²C-Schnittstelle angebunden. Im folgenden

soll genauer auf die Anbindung der IMU1 eingegangen werden, da diese repräsentativ für alle Sensoren ist.

Die beiden Sensoren der IMU1 (Beschleunigungssensor und Gyroskop) teilen sich die gleiche SPI-Schnittstelle (CLK, MOSI, MISO), sind jedoch als zwei separate Sensoren implementieren und müssen aufgrund dessen mit einzelnen CS-Leitungen versehen werden. Zur rechentechnisch ressourcenschonenden Anbindung werden zwei Data Ready Interrupts (DRDY) verwendet, welche die NAV-Unit darüber informieren, dass neue Daten des jeweiligen Sensors bereit zur Auslese sind. Um den Sensor in SPI Mode zu betreiben muss der PS Pin auf GND gezogen werden. Auf den Versorgungsleitungen werden zusätzlich 100 nF Keramikkondensatoren als Decoupling-Kondensatoren eingesetzt (siehe Abbildung 9.2). Die auf dem Bild zusehenden Widerstände dienen der Zustandsdefinition beim Einschalten, sind jedoch in keinerlei Hinsicht kritisch und können auch weggelassen werden.

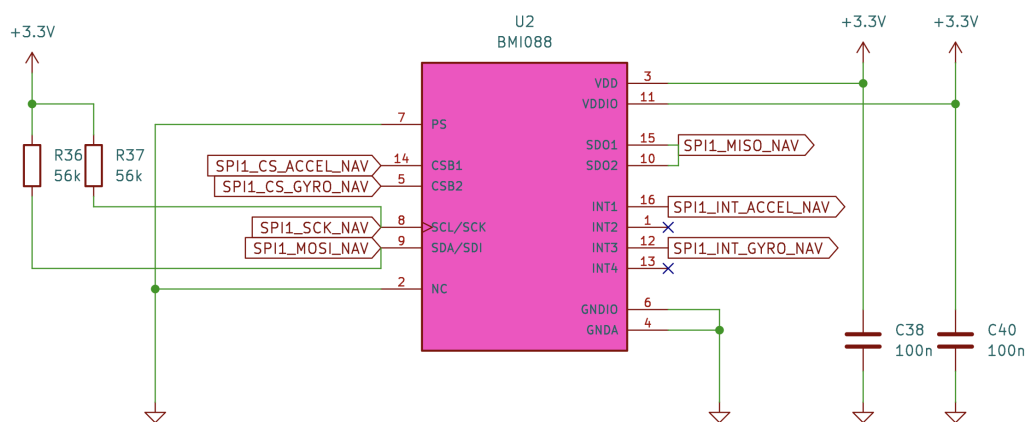


Abbildung 9.2.: Schaltplan des BMI088

9.4.2. Prozessoren

Die Anbindung der beiden Prozessoren setzt eine Reihe von passiven Bauteilen voraus, um eine stabile Funktion zu gewährleisten. Im Folgenden soll anhand der Anbindung der MPU (STM32H753VIT6) genauer auf diese eingegangen werden, da die Anbindung der NAV (STM32F405RGT6) im Grunde identisch ist. [20] [21]

Der wichtigste Punkt im Bezug auf die Anbindung des Prozessors ist eine stabile Spannungsversorgung von 3.3 V der verschiedenen Spannungsschienen. Diese teilen sich wie folgt auf:

- VDD: Versorgungsspannung der internen Logik und In- und Outputs des Prozessors
- VDDA: Versorgungsspannung der analogen Komponenten des Prozessors (ADC, DAC, etc.)
- VREF+ : Interne Referenzspannung
- VBAT: Versorgungsspannung über externe Batterie

VDD wird direkt an die 3.3 V Schiene angeschlossen, wobei zwischen jedem der fünf VDD-Pins und Masse ein 100 nF Keramikkondensator als Decoupling-Kondensator eingesetzt wird, um die Stabilität der Versorgung zu gewährleisten, zusätzlich zu den 100 nF Keramikkondensatoren wird ein 10 µF Kondensator eingesetzt um größere Ströme zu kompensieren. Um die durch die digitalen Schaltvorgänge auf der 3.3 V Schiene entstehenden hochfrequenten Störungen zu minimieren, werden die VDDA und VREF+ Pins über einen PI-Filter von der VDD Schiene entkoppelt, welcher aus zwei 1 µF Kondensator, einem 10 nF Keramikkondensator sowie einem Ferrite Bead (Induktivität) mit einer Impedanz von 100 Ω besteht (siehe Abbildung 9.3). Für VREF+ wird zusätzlich ein RC Tiefpass

eingefügt, um eine möglichst konsistente Referenzspannung zu gewährleisten. Hier stellt der Widerstand kein Problem dar, da kein nennenswerter Strom fließt. Da keine externe Batterie verwendet wird, kann der VBAT Pin als VDD Pin behandelt werden.

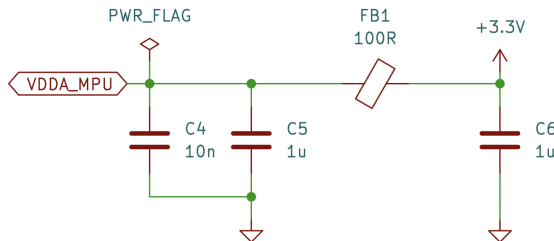


Abbildung 9.3.: PI-Filter zur Entkopplung der VDDA und VREF+ Pins von der 3.3 V Schiene

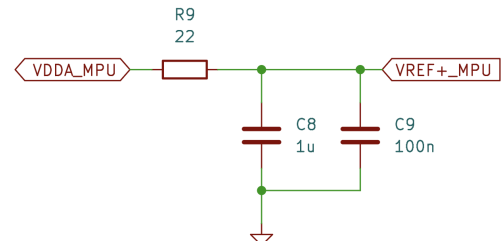


Abbildung 9.4.: RC-Tiefpass zur Stabilisierung der Referenzspannung am VREF+ Pin

Auch wenn der STM32H753VIT6 über eine interne Clock-Quelle verfügt, wird eine externer Quarz (High Speed External Crystal) mit einer Frequenz von 16 MHz verwendet, um eine höhere Genauigkeit der Taktfrequenz zu erreichen. Der Quarz wird über zwei 10 pF Lastkapazitäten mit Masse verbunden, um die Schwingung zuverlässig zu starten und auf der vorgesehenen Frequenz zu halten (siehe Abbildung 9.5). Die Werte für die Lastkapazitäten basieren auf folgender Formel:

$$C_L = \frac{C_{L1} \cdot C_{L2}}{C_{L1} + C_{L2}} + C_{stray} \quad (9.1)$$

wobei C_L die gesamte Lastkapazität des Quarzes mit 9 pF [7], C_{L1} und C_{L2} die beiden Lastkapazitäten sowie C_{stray} die parasitäre Kapazität mit angenommenen 4 pF ist. [15]

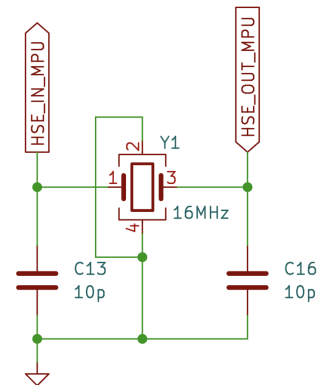


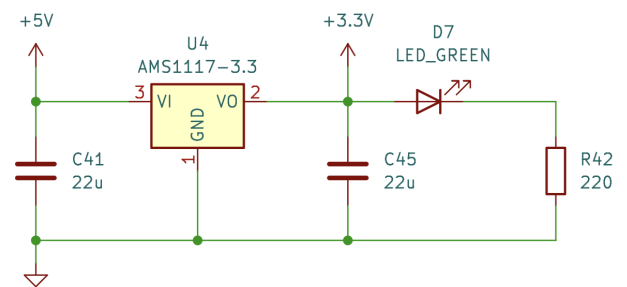
Abbildung 9.5.: Quarzschwingkreis zur Taktfrequenzgenerierung

9.4.3. Spannungsversorgung

Die Versorgung des Flightcomputer während des gesamten Fluges erfolgt über einen 2S-LiPo-Akku mit einem Spannungsbereich von 7.4 V (vollständig entladen) bis 8.4 V (vollständig geladen), während die Betriebsspannung der Komponenten 3.3 V beträgt. Um nun die benötigte Betriebsspannung von 3.3V aus der Eingangsspannung zu erzeugen, wird ein Abwärtswandler (siehe Abschnitt 4.3) verwendet, welcher die Spannung herunterregelt. Die Entscheidung fiel auf einen Abwärtswandler, da dieser im Bezug auf die Effizienz deutlich besser ist als ein linearer Spannungsregler, welcher die überschüssige Energie in Form von Wärme an die Umgebung abgibt.

Bei der Planung des Buck Converters ist es essentiell den Bauteilanforderungen des Herstellers insbesondere bei der Induktivität und den Kondensatoren nachzukommen, da diese die Stabilität der Schaltung sowie die Qualität der Ausgangsspannung maßgeblich beeinflussen. Aufgrund dessen wurde der WEBENCH POWER DESIGNER von Texas Instruments verwendet, um die Schaltung zu dimensionieren und die passenden Komponenten auszuwählen. In der Abbildung 9.6 ist der Schaltplan des Buck Converters mit Ein- und Ausgangskondensatoren, Induktivität, Feedback Spannungsteiler zur Ausgangsspannungseinstellung sowie einem Testpunkt und einer Lötbrücke zur späteren Funkti-

Um in der Entwicklungsphase nicht abhängig von der Akkuspannung zu sein wird die Möglichkeit geschaffen, den Flightcomputer auch über eine externe Spannungsversorgung (etwas Laptop) mit 5 V zu versorgen, welche über die USB-C-Schnittstelle angeschlossen werden kann. In diesem Fall wird die Eingangsspannung von 5 V über einen LDO (Low Dropout Regulator) auf die benötigten 3.3 V herunterregelt, da zu keiner Zeit eine hohe Stromaufnahme erfolgt und die Spannungsdifferenz gering ist. An den Ein- und Ausgängen des LDOs sind zwei 22 μ F Kondensatoren zur Stabilisierung der Spannung verschaltet (siehe Abbildung 9.7) außerdem wird eine LED zur Anzeige des Betriebszustands verwendet. Um im späteren Betrieb die Möglichkeit zu haben, die Akkus zu überwachen, wird außerdem ein Strom & Spannungsmessmodul integriert, welches die Messwerte über eine I²C-Schnittstelle an die Mikrocontroller überträgt.



9.5. Leiterplattendesign

The image shows two hand-drawn diagrams of PCB layouts, labeled 'Fran' and 'Back'.

Fran: This diagram shows a rectangular board with various components labeled in boxes. The components include: 'Saver on' (with a note 'Saver on' and a small box), 'GPS', 'NAV', 'MPU', '3.3V', 'RV', 'SERVOS' (with a note 'SERVOS' and a small box), 'H-MOT', 'P/MO', and 'I2C'. There are also several small circles representing holes or pins.

Back: This diagram shows a rectangular board with a few components labeled in boxes. The components include: 'Flux', 'SD', 'LOLA', and a small box labeled 'I2C'. There are also several small circles representing holes or pins.

Abbildung 9.8.: Erste Skizze der Leiterplatte

Die Dimensionen der Platine unterliegen den Limitationen des Raketenrumpfs mit einem Innendurchmesser von 80 mm und betragen 73 mm × 98 mm. Im Bezug auf das Stack-up wird ein 4-lagiges Design mit einer Gesamtdicke von 1.6 mm gewählt, da dies für die Anforderungen im Bezug auf Signalintegrität und Platzbedarf ausreichend ist und gleichzeitig die Kosten im Rahmen hält. Die Anordnung der Lagen erfolgt wie folgt:

- Top Layer: Signale, Bauteile
- Inner Layer 1: Masse
- Inner Layer 2: Versorgungsspannung (3.3 V)
- Bottom Layer: Signale, Bauteile

Obwohl die Bottom-Layer nicht direkt an eine zusammenhängende Ground-Plane angrenzt, ist dies für die dort verwendeten Bauteile akzeptabel, da die zugehörigen Netze keine allzu schnellen Flanken bzw. hohen Stromtransienten aufweisen. Die Rückstromführung bleibt damit unkritisch und es sind keine signifikanten Auswirkungen auf Signalqualität oder EMV zu erwarten.

Hier soll nun die Platine des Flightcomputers KOMADORI und die wichtigsten Designentscheidungen möglichst strukturiert und verständlich erläutert werden.

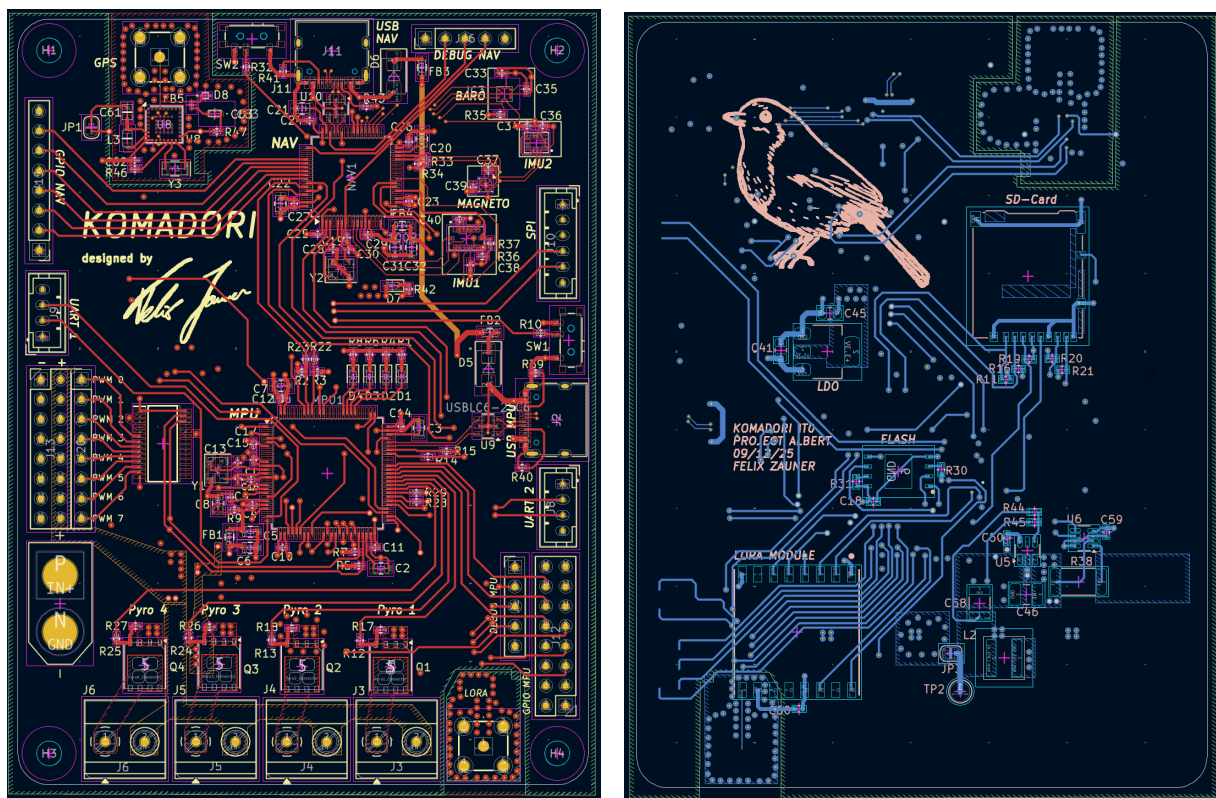
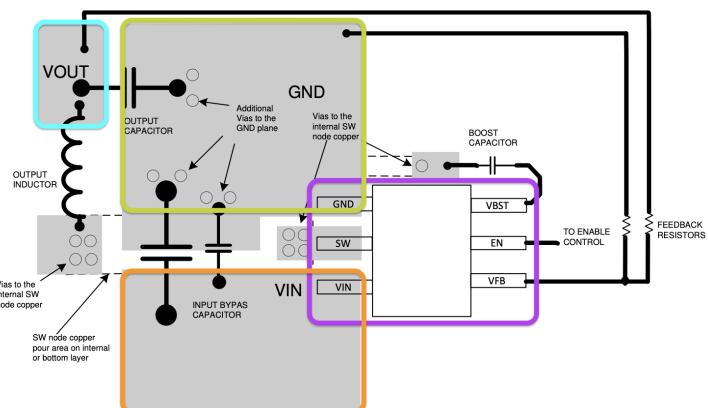
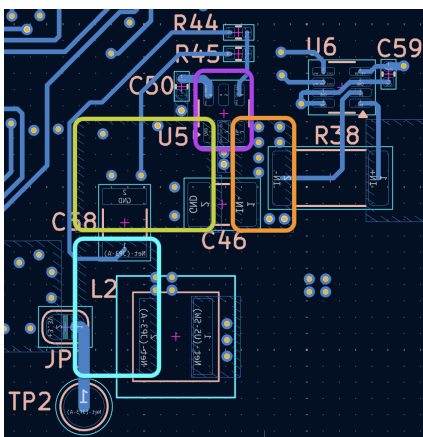
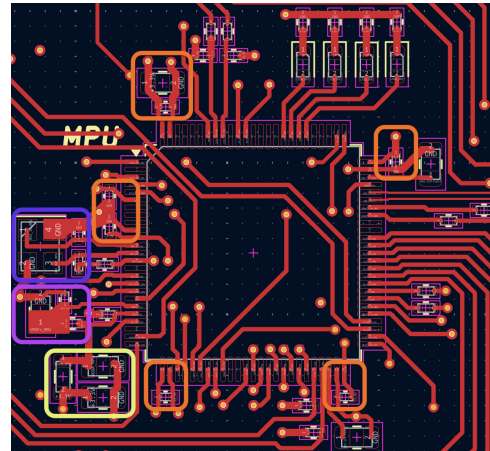


Abbildung 9.9.: Layout der Leiterplatte des Flightcomputers KOMADORI (Vorder- und Rückseite)



9.5.3. GNSS Modul

Im Datenblatt des verwendeten GPS-Moduls ZOE-M8Q siehe [25] werden mehrer Layoutempfehlungen gegeben, um eine optimale Funktionalität zu gewährleisten. Dabei ist die räumliche Trennung des GPS-Moduls von digitalen Komponenten und insbesondere der Abstand zur Antenne von großer Bedeutung um Störungen zu minimieren. Desweiteren ist eine gute Masseanbindung, eine durchgehende Masse Fläche sowie eine Abschirmung durch einen GND-Via Ring des GPS-Moduls essentiell.

In der Abbildung 9.12 ist das Layout des GPS-Moduls dargestellt. Das GPS-Modul befindet sich in der linken oberen Ecke auf der Vorderseite der Leiterplatte, mit einer direkt darunterliegenden durchgehenden Massefläche und ist damit sowohl auf der Vorder- als auch auf der Rückseite von digitalen Störungen räumlich getrennt. Um die Störanfälligkeit weiter zu minimieren, ist das GPS-Modul von einem Ring aus GND-Vias umgeben, welcher als Abschirmung dient. Die per SMA-Stecker angeschlossene Antenne wird über einen $50\,\Omega$ Microstrip mit einer Breite von $0.12\,\text{mm}$ mit dem RF-Input des GPS-Moduls verbunden. Wobei die Breite des Microstrips mit einem Impedanzrechner von JLCPCB basierend auf den Stack-up Spezifikationen der Leiterplatte berechnet wird. Bei den an den Microstrip angeschlossenen Bauteilen handelt es sich um die Speisung für die Aktivantenne. Es soll an dieser Stelle nicht näher auf die Beschaltung des GPS-Moduls eingegangen werden, da diese auf Basis des Datenblatts und der Bauteilvorschläge des Herstellers erfolgt und eine detaillierte Erläuterung den Rahmen sprengen würde.

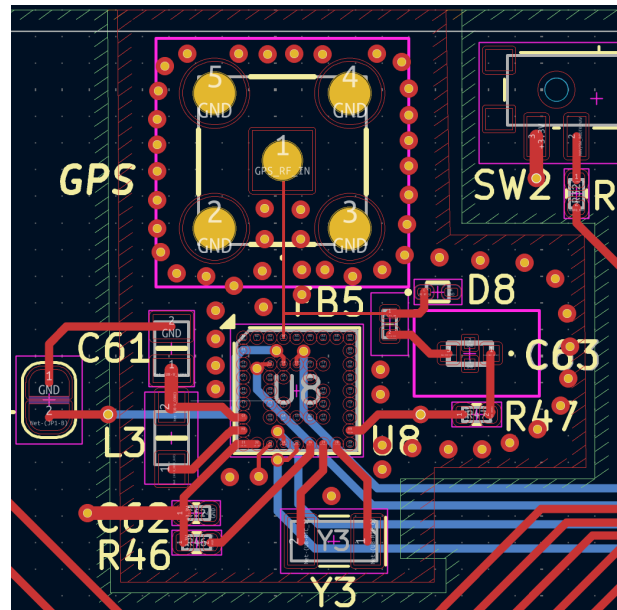


Abbildung 9.12.: Layout des GPS-Moduls mit den zugehörigen Designrichtlinien

9.5.4. Servos und Pyro-Kanäle

Die Servos werden direkt über die Eingangsspannung des 2S-LiPo-Akkus ($7.4\text{--}8.4\,\text{V}$) versorgt, da sie bei höherer Versorgungsspannung eine größere Stellgeschwindigkeit erreichen. Ebenso werden die Pyro-Kanäle direkt über die Akkuspannung betrieben, da beim Zünden der pyrotechnischen Ladungen kurzzeitig hohe Ströme auftreten. Durch die direkte Versorgung aus dem Akku wird verhindert, dass diese Stromspitzen zu Spannungseinbrüchen auf der $3.3\,\text{V}$ -Versorgungsschiene der Steuerelektronik führen.

Um die höheren Ströme der Servos und Pyro-Kanäle zu bewältigen, werden die entsprechenden Leiterbahnen möglichst breit ausgeführt, desweiteren werden diese auf der dritten Lage der Leiterplatte geführt.

10. Steuerungssystem

10.1. Anforderungsanalyse

10.2. Zustandsraummodellierung

Ein wesentlicher Teil der Entwicklung der Regelung für die Rakete ist die Erstellung eines Zustandsraummodells, das die Dynamik des Systems ausreichend akkurat beschreibt. Im Bezug auf die Rakete umfasst dies die Modellierung der translationalen und rotatorischen Bewegungen, die durch die Kräfte und Momente beeinflusst werden, die auf die Rakete wirken.

Die im folgenden aufgestellten Bewegungsgleichung basieren in großen Teilen auf den Gleichungen, die in [9] beschrieben werden.

10.2.1. Translationale Bewegungsgleichungen

$$\dot{u} = \frac{F_{A_{x_b}} + F_{P_{x_b}} + F_{g_{x_b}}}{m} - (qw - rv) \quad [\text{m/s}^2] \quad (10.1)$$

$$\dot{v} = \frac{F_{A_{y_b}} + F_{P_{y_b}} + F_{g_{y_b}}}{m} - (ru - pw) \quad [\text{m/s}^2] \quad (10.2)$$

$$\dot{w} = \frac{F_{A_{z_b}} + F_{P_{z_b}} + F_{g_{z_b}}}{m} - (pv - qu) \quad [\text{m/s}^2] \quad (10.3)$$

wobei $F_{A_{x_b}}$, $F_{A_{y_b}}$ und $F_{A_{z_b}}$ die Komponenten der aerodynamischen Kräfte in Körperachsen darstellen, $F_{P_{x_b}}$, $F_{P_{y_b}}$ und $F_{P_{z_b}}$ die Komponenten der Schubkraft und $F_{g_{x_b}}$, $F_{g_{y_b}}$ und $F_{g_{z_b}}$ die Komponenten der Gravitationskraft.

Der Subtrahend in den Gleichungen berücksichtigt die Drehbewegung des Body Frames im Navigation Frame, wobei p , q und r die Winkelgeschwindigkeiten um die Körperachsen darstellen. Sie stammen aus dem Kreuzprodukt der Winkelgeschwindigkeiten mit den Geschwindigkeiten.

$$\boldsymbol{\omega} \times \mathbf{v} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} qw - rv \\ ru - pw \\ pv - qu \end{bmatrix} \quad [\text{m/s}^2] \quad (10.4)$$

Die aerodynamischen Kräfte können wie folgt berechnet werden:

$$F_{A_{x_b}} = -\frac{1}{2}\rho S C_A |V|^2 \quad [\text{N}] \quad (10.5)$$

$$F_{A_{y_b}} = \frac{1}{2}\rho S C_{N_y} |V|^2 \quad [\text{N}] \quad (10.6)$$

$$F_{A_{z_b}} = \frac{1}{2}\rho S C_{N_z} |V|^2 \quad [\text{N}] \quad (10.7)$$

wobei ρ die Luftdichte, S die Referenzfläche (Stirnfläche), C_A der axiale Widerstandsbeiwert (Luftwiderstand) und $|V|$ der Betrag des Geschwindigkeitsvektors der Rakete ist. Der Auftriebsbeiwert, C_{N_y} und C_{N_z} sind die Normalbeiwertkomponenten in den y - und z -Richtung welche sich wie folgt aus C_N berechnen lassen:

$$C_{N_y} = C_N \frac{-v}{\sqrt{v^2 + w^2}} \quad (10.8)$$

$$C_{N_z} = C_N \frac{-w}{\sqrt{v^2 + w^2}} \quad (10.9)$$

Die Schubkraftkomponenten können wie folgt berechnet werden:

$$F_{P_{x_b}} = F_{pref} + (p_{ref} - p_a A_e) \quad (10.10)$$

$$F_{P_{y_b}} = 0 \quad (10.11)$$

$$F_{P_{z_b}} = 0 \quad (10.12)$$

wobei F_{pref} die Schubkraft ist, die sich aus dem Massestrom und der Austrittsgeschwindigkeit berechnet (siehe 7.1.1).

Für die Gravitationskraft gilt im Navigation Frame:

$$F_{g_{x_n}} = mg \quad (10.13)$$

$$F_{g_{y_n}} = 0 \quad (10.14)$$

$$F_{g_{z_n}} = 0 \quad (10.15)$$

wobei g die Erdbeschleunigung ist. Um die Gravitationskraft in den Körperachsen zu berechnen, muss sie mit der Transformationsmatrix $T_{n \rightarrow b}$ von Navigation Frame zu Body Frame transformiert werden:

$$\mathbf{T}_{n \rightarrow b} = \begin{bmatrix} \cos \theta \cos \psi & \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi \\ \cos \theta \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{bmatrix} \quad (10.16)$$

wobei ϕ , θ und ψ die Roll-, Nick- und Gierwinkel der Rakete sind. Die Gravitationskraft in den Körperachsen kann dann wie folgt berechnet werden:

$$\begin{bmatrix} F_{g_{x_b}} \\ F_{g_{y_b}} \\ F_{g_{z_b}} \end{bmatrix} = \mathbf{T}_{n \rightarrow b} \begin{bmatrix} F_{g_{x_n}} \\ F_{g_{y_n}} \\ F_{g_{z_n}} \end{bmatrix} = \mathbf{T}_{n \rightarrow b} \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} \quad [\text{N}] \quad (10.17)$$

Da über den Zeitraum der Masseverlust durch den Treibstoffverbrauch berücksichtigt werden muss, wird die Masse m als Funktion der Zeit modelliert.

$$m = m_0 - \frac{1}{I_{sp}} \int_0^t F_{p_{ref}} dt \quad [\text{kg}] \quad (10.18)$$

Dabei ist $F_{p_{ref}}$ die Referenzschubkraft in Newton, I_{sp} der spezifische Impuls des Treibstoffs in Ns/kg, m_0 die Raketenmasse zum Zeitpunkt $t = 0$ in Kilogramm, und t die simulierte Zeit in Sekunden.

10.2.2. Rotatorische Bewegungsgleichungen

Die Winkelgeschwindigkeiten um die Körperachsen können wie folgt berechnet werden:

$$\begin{aligned} \dot{p} &= \frac{L_A + L_p - q r (I_z - I_y)}{I_x} \quad [\text{rad/s}^2] \\ \dot{q} &= \frac{M_A + M_p - r p (I_x - I_z)}{I_y} \quad [\text{rad/s}^2] \\ \dot{r} &= \frac{N_A + N_p - p q (I_y - I_x)}{I_z} \quad [\text{rad/s}^2] \end{aligned}$$

wobei L_A , M_A und N_A die aerodynamischen Momente in Körperachsen darstellen, L_p , M_p und N_p die Schubmomente und I_x , I_y und I_z die Trägheitsmomente der Rakete um die Körperachsen. Die Schreibweise der aerodynamischen Momente im Body Frame als

$$M_B = \begin{bmatrix} L \\ M \\ N \end{bmatrix} \quad [\text{Nm}] \quad (10.19)$$

ist historisch bedingt und wird in der Literatur häufig verwendet, um die Momente um die drei Körperachsen zu beschreiben. L für das Moment um die x-Achse (Rollmoment), M für das Moment um die y-Achse (Nickmoment) und N für das Moment um die z-Achse (Giermoment).

Für die aerodynamischen Momente gilt:

$$L_A = \frac{1}{2} \rho S C_{l_i} |V|^2 d \quad [\text{Nm}] \quad (10.20)$$

$$M_A = \frac{1}{2} \rho S C_{m_i} |V|^2 d \quad [\text{Nm}] \quad (10.21)$$

$$N_A = \frac{1}{2} \rho S C_{n_i} |V|^2 d \quad [\text{Nm}] \quad (10.22)$$

wobei d die aerodynamische Referenzlänge ist, und C_{l_i} , C_{m_i} und C_{n_i} die Roll-, Nick- und Giermoment-

beiwertkomponenten sind, welche sich wie folgt berechnen lassen:

$$C_l = C_{l\delta} \delta_r + \frac{d}{2|V|} (C_{lp} p) \quad (10.23)$$

$$C_m = C_{m_{ref}} - C_{N_z} \frac{x_{cm} - x_{ref}}{d} + \frac{d}{2|V|} (C_{mq} + C_{m_\alpha}) q \quad (10.24)$$

$$C_n = C_{n_{ref}} + C_{N_y} \frac{x_{cm} - x_{ref}}{d} + \frac{d}{2|V|} (C_{nr} + C_{n_\beta}) r \quad (10.25)$$

Dabei ist C_{lp} die Roll-Dämpfungsableitung bezüglich der Rollrate p in rad^{-1} . $C_{l\delta}$ bezeichnet die Steigung der Kurve des Rollmomentbeiwerts C_l in Abhängigkeit der Finnenauslenkung in rad^{-1} . $C_{m_{ref}}$ ist der Nickmomentbeiwert bezogen auf die Referenzmomentstation. C_{mq} ist die Nick-Dämpfungsableitung bezüglich der Nickrate q in rad^{-1} . C_{m_α} ist die Nick-Dämpfungsableitung bezüglich der Änderungsrate des Anstellwinkels $\dot{\alpha}$ in rad^{-1} . C_{N_y} ist der Beiwert der Normalkraftkomponente in y_b -Richtung. C_{N_z} ist der Beiwert der Normalkraftkomponente in z_b -Richtung. C_{nr} ist die Gier-Dämpfungsableitung bezüglich der Gierrate r in rad^{-1} . $C_{n_{ref}}$ ist der Giermomentbeiwert bezogen auf die Referenzmomentstation. C_{n_β} ist die Gier-Dämpfungsableitung bezüglich der Änderungsrate des Schiebewinkels $\dot{\beta}$ in rad^{-1} . x_{cm} ist der Abstand von der Raketenspitze zum Schwerpunkt in Metern.

Dabei ist x_{ref} der Abstand von der Raketenspitze zur Referenzmomentstation in Metern. δ_r ist die effektive Ruderauslenkung, die ein Rollmoment verursacht, angegeben in Radiant.

$$C_{m_{ref}} = C_{m_\alpha} \alpha + C_{m_\delta} \delta_\eta \quad (10.26)$$

$$C_{n_{ref}} = C_{n_\beta} \beta + C_{n_\delta} \delta_\zeta \quad (10.27)$$

$C_{m_{ref}}$ ist der Nickmomentbeiwert bezogen auf die Referenzmomentstation. C_{m_α} bezeichnet die Steigung der Kurve des Nickmomentbeiwerts C_m in Abhängigkeit vom Anstellwinkel α in rad^{-1} . C_{m_δ} ist die Steigung der Kurve des Nickmomentbeiwerts C_m in Abhängigkeit von der Ruderauslenkung für Nickbewegungen in rad^{-1} . C_{n_β} ist die Steigung der Kurve des Giermomentbeiwerts C_n in Abhängigkeit vom Schiebewinkel β in rad^{-1} . C_{n_δ} ist die Steigung der Kurve des Giermomentbeiwerts C_n in Abhängigkeit von der effektiven Ruderauslenkung für Gierbewegungen in rad^{-1} . Dabei ist α der Anstellwinkel (Angle of Attack) in Radiant, β der Schiebewinkel (Sideslip Angle) in Radiant, $\delta_\eta = \delta_1 = \delta_3$ die effektive Ruderauslenkung, die ein Nickmoment verursacht, in Radiant, und $\delta_\zeta = \delta_4 = \delta_2$ die effektive Ruderauslenkung, die ein Giermoment verursacht, in Radiant siehe Abbildung 10.1.

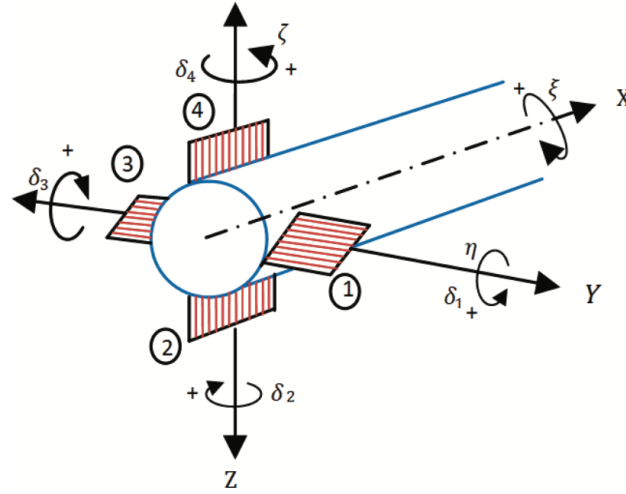


Abbildung 10.1.: Modell eines Rakete mit 6 Freiheitsgraden [9]

Der Anstellwinkel α (und gesamte Anstellwinkel α_t), der Schiebewinkel β sowie der aerodynamische Rollwinkel ϕ zur Bestimmung der aerodynamischen Koeffizienten ergeben sich zu:

$$\alpha = \tan^{-1}\left(\frac{w}{u}\right) \quad \text{oder} \quad \alpha_t = \tan^{-1}\left(\frac{\sqrt{v^2 + w^2}}{u}\right) \quad [\text{rad}] \quad (10.28)$$

$$\beta = \tan^{-1}\left(\frac{v}{V_M}\right) \quad [\text{rad}] \quad (10.29)$$

$$\phi = \tan^{-1}\left(\frac{v}{w}\right) \quad [\text{rad}] \quad (10.30)$$

10.2.3. Ausrichtung (Attitude)

Es gibt zwei übliche Methoden, um die Ausrichtung eines Flugkörpers zu beschreiben: die Verwendung von Euler-Winkeln oder die Verwendung von Quaternionen. Für ausgereifere Steuerungsalgorithmen, insbesondere solche, die eine kontinuierliche und singularitätsfreie Darstellung der Ausrichtung erfordern, sind Quaternionen oft die bevorzugte Wahl. Da wir uns in diesem Projekt ausschließlich auf kleine Winkeländerungen konzentrieren und somit eine Division durch $\cos(90^\circ) = 0$ und die daraus folgende Instabilität ausschließen können, bieten Euler-Winkel eine einfachere und intuitivere Möglichkeit, die Ausrichtung zu beschreiben. In diesem Fall werden die Euler-Winkel ϕ (Roll), θ (Nick) und ψ (Gier) verwendet, um die Ausrichtung der Rakete im Navigation Frame zu beschreiben.

$$\dot{\phi} = p + (q \sin \phi + r \cos \phi) \tan \theta \quad [\text{rad/s}] \quad (10.31)$$

$$\dot{\theta} = q \cos \phi - r \sin \phi \quad [\text{rad/s}] \quad (10.32)$$

$$\dot{\psi} = \frac{q \sin \phi + r \cos \phi}{\cos \theta} \quad [\text{rad/s}] \quad (10.33)$$

10.2.4. Systemdynamik

Die gesamte nichtlineare Systemdynamik mit 6 Freiheitsgraden (6dof, 3 translationale und 3 rotatorische) kann mit den folgenden bereits oben behandelten Gleichungen beschrieben werden:

$$\mathbf{x} = \begin{bmatrix} u \\ v \\ w \\ p \\ q \\ r \\ \phi \\ \theta \\ \psi \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} \delta_\eta \\ \delta_\zeta \\ \delta_r \end{bmatrix}, \quad \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \frac{F_{A_{x_b}} + F_{P_{x_b}} + F_{g_{x_b}}}{m} - (qw - rv) \\ \frac{F_{A_{y_b}} + F_{P_{y_b}} + F_{g_{y_b}}}{m} - (ru - pw) \\ \frac{F_{A_{z_b}} + F_{P_{z_b}} + F_{g_{z_b}}}{m} - (pv - qu) \\ \frac{L_A + L_p - qr(I_z - I_y)}{I_x} \\ \frac{M_A + M_p - rp(I_x - I_z)}{I_y} \\ \frac{N_A + N_p - pq(I_y - I_x)}{I_z} \\ p + (q \sin \phi + r \cos \phi) \tan \theta \\ q \cos \phi - r \sin \phi \\ \frac{q \sin \phi + r \cos \phi}{\cos \theta} \end{bmatrix}. \quad (10.34)$$

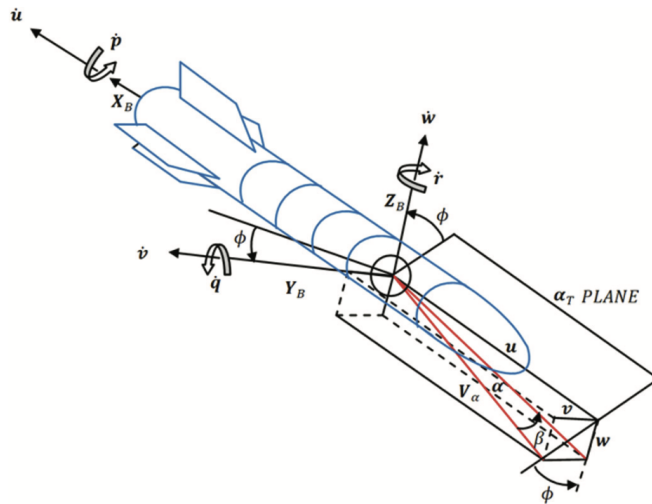


Abbildung 10.2.: Modell einer Rakete mit 6 Freiheitsgraden [9]

Für die Entwicklung eines Linear Quadratic Gaussian (LQG) Reglers ist es notwendig, das System an einem passenden Arbeitspunkt zu linearisieren.

10.2.5. Bestimmung des stationären Arbeitspunkts

Es gilt einen Zustand $\bar{\mathbf{x}}$ und Eingänge $\bar{\mathbf{u}}$ zu finden, sodass $f(\bar{\mathbf{x}}, \bar{\mathbf{u}}) = 0$. Für eine Rakete wäre ein stationärer Arbeitspunkt ein Zustand, in dem die Rakete sich mit konstanter Geschwindigkeit und konstanter Ausrichtung bewegt. Das bedeutet, dass die Ableitungen von Geschwindigkeit, Winkelgeschwindigkeit und Euler-Winkeln alle null sein sollten, was zu den folgenden Bedingungen führt:

$$f(\bar{\mathbf{x}}, \bar{\mathbf{u}}) = [\dot{u} \quad \dot{v} \quad \dot{w} \quad \dot{p} \quad \dot{q} \quad \dot{r} \quad \dot{\phi} \quad \dot{\theta} \quad \dot{\psi}]^T = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^T \quad (10.35)$$

An diesem Punkt tritt jedoch ein Problem auf, da die Beschleunigung in x Richtung aufgrund der nicht steuerbaren Schubkraft in der gesamten Phase des Starts nicht null sein wird, was bedeutet, dass sich die Rakete nicht in einem stationären Arbeitspunkt befindet.

Um trotzdem eine Linearisierung durchführen zu können und die Entwicklung eines LQG Reglers zu ermöglichen, werden mehrere Arbeitspunkte mit verschiedenen Geschwindigkeiten u in x Richtung definiert, um die Dynamik der Rakete in verschiedenen Flugphasen zu beschreiben. Über Interpolation zwischen diesen Arbeitspunkten kann dann die Dynamik der Rakete über den gesamten Flug hinweg beschrieben werden und die Regler im Laufe des Fluges dynamisch angepasst werden.

Für n quasi Arbeitspunkte $\bar{\mathbf{x}}_n$ gilt:

$$\bar{\mathbf{x}}_n = [u_n \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^T \quad (10.36)$$

Wobei für die Eingangsgrößen u im Vertikalflug gilt:

$$\bar{\mathbf{u}}_n = [\delta_\eta \quad \delta_\zeta \quad \delta_r]^T = [0 \quad 0 \quad 0]^T \quad (10.37)$$

10.2.6. Linearisierung

Da während der Startphase kein stationärer Arbeitspunkt existiert, erfolgt Linearisierung des Systems um den Arbeitspunkt $\bar{\mathbf{x}}_n$ durch Berechnung der Jacobimatrix der Funktion $\mathbf{f}(\mathbf{x}, \mathbf{u})$ für n Arbeitspunkte. Dies führt zu den folgenden linearen von der Geschwindigkeit $\bar{\mathbf{u}}_n$ abhängigen Systemmatrizen $\mathbf{A}_n$ und $\mathbf{B}_n$ für jeden Arbeitspunkt:

$$\mathbf{A}_n = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{x}=\bar{\mathbf{x}}, \mathbf{u}=\bar{\mathbf{u}}_n} \quad \text{und} \quad \mathbf{B}_n = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\mathbf{x}=\bar{\mathbf{x}}, \mathbf{u}=\bar{\mathbf{u}}_n} \quad (10.38)$$

Damit kann die lineare Systemdynamik um die Arbeitspunkte beschrieben werden als:

$$\Delta \dot{\mathbf{x}} = \mathbf{A}_n \Delta \mathbf{x} + \mathbf{B}_n \Delta \mathbf{u} \quad (10.39)$$

Mit $\Delta \mathbf{x} = \mathbf{x} - \bar{\mathbf{x}}_n$ und $\Delta \mathbf{u} = \mathbf{u} - \bar{\mathbf{u}}_n$ als Abweichungen von dem jeweiligen Arbeitspunkt. Im folgenden werden die Arbeitspunkte als Nullpunkte der linearen Systeme betrachtet, sodass die lineare Systemdynamik um die Arbeitspunkte beschrieben werden kann als:

$$\dot{\mathbf{x}} = \mathbf{A}_n \mathbf{x} + \mathbf{B}_n \mathbf{u} \quad (10.40)$$

Zur Bestimmung der Systemmatrizen $\mathbf{A}_n$ und $\mathbf{B}_n$ wird die *Symbolic Toolbox* von MATLAB verwendet. Zunächst werden die Jacobi-Matrizen des nichtlinearen Zustandsraummodells $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ symbolisch berechnet. Durch Auswertung dieser Ableitungen an den festgelegten Arbeitspunkten (Einsetzen der jeweiligen Zustands- und Eingangsgrößen sowie der Modellparameter) ergeben sich die numerischen

Systemmatrizen. Die folgenden Matrizen A_n und B_n beschreiben die lineare Dynamik der Rakete um die jeweiligen Arbeitspunkte mit verschiedenen Geschwindigkeiten u in x Richtung.

10.3. Linear Quadratic Gaussian

10.3.1. Steuerbarkeit

Um sich im ersten Schritt ein Bild über die Regelstrecke zu machen, werden zuerst die Eigenwerte der Systemmatrix A_n für die verschiedenen Arbeitspunkte berechnet. Alle Eigenwerte bis auf einen haben negative Realteile, was darauf hindeutet, dass das System um die Arbeitspunkte stabil ist. Der eine Eigenwert mit positivem Realteil ist durch die Geschwindigkeit u in x Richtung bedingt.

$$\begin{bmatrix} -0.1531 \\ -1.4844 + 23.1239i \\ -1.4844 - 23.1239i \\ -0.1958 \\ -24.6607 \\ 21.6927 \\ -0.1966 \\ 0.0000 \\ 0.0000 \end{bmatrix} \quad (10.42)$$

Nachdem die Eigenwerte berechnet wurden, wird die Steuerbarkeit des Systems überprüft. Das System ist steuerbar, wenn die Steuerbarkeitsmatrix $\mathcal{C} = [B_n, A_n B_n, A_n^2 B_n, \dots, A_n^{n-1} B_n]$ vollen Rang hat. Dies wird mit `ctrlrank = rank(ctrb(A,B))` Funktion in MATLAB überprüft. Als Rückgabewert erhält man für den Wert 8, was bedeutet, dass das System nicht vollständig steuerbar ist, da die Systemdimension 9 beträgt, was auch völlig logisch ist, wenn man bedenkt, dass die Geschwindigkeit u maßgeblich von der Schubkraft bestimmt wird, die nicht als Steuergröße zur Verfügung steht.

In Folge dieser Erkenntnis wird das System um die nicht steuerbare Geschwindigkeit u reduziert, wobei die Größe trotzdem in der Zustandsschätzung berücksichtigt wird, da die anderen Systemgrößen von der Geschwindigkeit u abhängen.

10.3.2. Ermittlung der K Matrix

10.3.3. Kalman Filter

$$A_n = \begin{pmatrix} -\frac{49u_b}{16000} - \frac{49v_b}{12000} - \frac{49w_b}{12000} & r - \frac{49u_b}{12000} & -q - \frac{49u_b}{12000} & 0 & -w_b & v_b & 0 & -\frac{981 \cos(\psi) \sin(\theta)}{100} \\ \frac{49v_b}{960} - r - \frac{49u_b}{4800} \left(\delta_\zeta + \frac{10w_b}{u_b} \right) & \sigma_4 & p & w_b & 0 & -u_b & \frac{981 \sin(\phi) \sin(\psi)}{100} + \frac{981 \cos(\phi) \cos(\psi) \sin(\theta)}{100} & -\frac{981 \cos(\psi) \sin(\theta)}{100} \\ q + \frac{49w_b}{960} - \frac{49u_b}{4800} \left(\delta_\eta + \frac{10w_b}{u_b} \right) & -p & \sigma_4 & -v_b & u_b & 0 & \frac{981 \cos(\phi) \sin(\psi)}{100} - \frac{981 \cos(\psi) \sin(\phi) \sin(\theta)}{100} & \frac{981 \cos(\phi) \cos(\psi) \sin(\theta)}{100} \\ \frac{49\delta_\zeta u_b}{400} - \frac{49p}{800} & 0 & 0 & -\frac{49u_b}{800} & 0 & 0 & 0 & 0 \\ \frac{343w_b}{1600} - \frac{49q}{4000} + \frac{147\delta_\eta u_b}{1600} & 0 & \frac{343u_b}{1600} & \frac{3r}{4} & \sigma_3 & \frac{3p}{4} & 0 & 0 \\ \frac{343v_b}{1600} - \frac{49r}{4000} + \frac{147\delta_\zeta u_b}{1600} & \frac{343u_b}{1600} & 0 & -\frac{3q}{4} & -\frac{3p}{4} & \sigma_3 & 0 & 0 \\ 0 & 0 & 0 & 1 & \frac{\sin(\phi) \sin(\theta)}{\cos(\theta)} & \frac{\cos(\phi) \sin(\theta)}{\cos(\theta)} & -r \cos(\phi) - q \sin(\phi) & r \cos(\phi) + q \sin(\phi) \\ 0 & 0 & 0 & 0 & \frac{\cos(\phi)}{\cos(\theta)} & -\frac{\sin(\phi)}{\cos(\theta)} & \frac{\sin(\theta) \sigma_1}{\cos(\theta)} & 0 \\ 0 & 0 & 0 & 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} & \frac{\sigma_1}{\cos(\theta)} & \frac{\sin(\theta) \sigma_2}{\cos(\theta)^2} \end{pmatrix}$$

where

$$\sigma_1 = q \cos(\phi) - r \sin(\phi)$$

$$\sigma_2 = r \cos(\phi) + q \sin(\phi)$$

$$\sigma_3 = -\frac{49u_b}{4000}$$

$$\sigma_4 = -\frac{49u_b}{960}$$

(10.41)

11. Implementierung in Software

11.1. Treiber

An dieser Stelle soll der Treiber für das IMU1 BMI088 repräsentativ für die Treiber aller Sensoren beschrieben werden. Alle Treiber sind in C implementiert und teilen sich in ein Header File, in dem die Schnittstelle definiert ist, und ein Source File, in dem die eigentlichen Funktionen implementiert sind.

Die Entwicklung der Treiber für die übrigen Sensoren sowie für den PWM-Controller, den Flash-Speicher und die SD-Karte erfolgt sowohl bei SPI- als auch bei I²C-Kommunikation nach dem gleichen grundlegenden Prinzip. Dabei werden lediglich die jeweiligen Registeradressen und Konfigurationswerte entsprechend den Anforderungen der einzelnen Bauteile angepasst.

Der Treiber für das GPS-Modul unterscheidet sich hingegen deutlich von den übrigen Treibern, da die Kommunikation über ein komplexeres Protokoll erfolgt und zusätzliche Funktionen zur Datenverarbeitung erforderlich sind. Aufgrund dieser erhöhten Komplexität wird der GPS-Treiber im Rahmen dieser Arbeit nicht näher behandelt.

11.1.1. Header File

Im Header File wird zuerst ein Include Guard definiert, um Mehrfacheinbindungen zu verhindern. Der Treiber basiert auf den Funktionen des HAL Treibers (Hardware Abstraction Layer) von STM32, einer Abstraktionsschicht die Funktionen für die Kommunikation mit Peripheriehardware bereitstellt.

Codeblock 11.1: Include Guard

```
1 #ifndef BMI088_H
2 #define BMI088_H
3
4 #include "stm32f4xx_hal.h"
5
6 //... rest of the header file content ...
7
8 #endif
```

Anschließend werden die Registeradressen des BMI088 definiert, so müssen diese bei Verwendung nicht ausgeschrieben werden, was die Lesbarkeit deutlich verbessert.

Zur Verwaltung der Sensordaten als auch der Kalibrierungsdaten wird eine Struktur definiert, die alle relevanten Informationen enthält. Diese Struktur ermöglicht es, die Daten organisiert und leicht zugänglich zu halten.

Codeblock 11.2: Sensor Struktur

```
1 typedef struct {
2     /* SPI HANDLE */
3     SPI_HandleTypeDef *spiHandle;
```

```
4
5     /* Acceleration data (X,Y,Z) in m/s^2 */
6     float acc_mps2[3];
7
8     /* Gyroscope data (X,Y,Z) in rad/s */
9     float gyr_rdp[3];
10 }BMI088;
```

Folgende Funktionen werden im Header File deklariert, um die Schnittstelle des Treibers zu definieren:

Codeblock 11.3: Funktionendeklarationen

```
1  uint8_t BMI088_Init(BMI088 *dev, SPI_HandleTypeDef *spiHandle);
2
3  // Read Data
4  HAL_StatusTypeDef BMI088_ReadAcceleration(BMI088 *dev);
5  HAL_StatusTypeDef BMI088_ReadAngularRate(BMI088 *dev);
6
7  // Low level register access
8  HAL_StatusTypeDef BMI088_ReadRegister(BMI088 *dev, uint8_t reg,
9                                         uint8_t *data, uint8_t select);
10 HAL_StatusTypeDef BMI088_ReadRegisters(BMI088 *dev, uint8_t reg,
11                                         uint8_t *data, uint8_t length, uint8_t select);
12 HAL_StatusTypeDef BMI088_WriteRegister(BMI088 *dev, uint8_t reg,
13                                         uint8_t value, uint8_t select);
```

11.1.2. Source File

Die Implementierung der Funktionen im Source-File wird hier beschrieben. Zunächst muss das Header-File des Treibers selbst inkludiert werden, außerdem das Main-Header-File, um Zugriff auf die HAL-Funktionen zu erhalten.

Hier soll zunächst auf die Funktion BMI088_Init genauer eingegangen werden, jedoch basiert diese auf den Funktionen BMI088_ReadRegister, BMI088_ReadRegisters und BMI088_WriteRegister, welche die grundlegende SPI-Kommunikation mit dem Sensor ermöglichen, um Register zu lesen und zu beschreiben. Diese werden im Anschluss beschrieben, um den Funktionsablauf so verständlich und kohärent wie möglich zu darzustellen.

BMI088_Init

In der Funktion BMI088_Init wird die SPI-Schnittstelle initialisiert und die Sensoren konfiguriert, um die gewünschten Messwerte zu liefern. Dies beinhaltet das Schreiben von Konfigurationswerten in die entsprechenden Register des Sensors, um beispielsweise die Messrate, den Messbereich und die Filtereinstellungen festzulegen.

Codeblock 11.4: Struktur initialisieren

```
1  /* Set struct parameters */
2  dev->spiHandle = spiHandle;
```

Zuerst wird der SPI-Handle in der Sensorstruktur gespeichert, damit er in den anderen Funktionen für die Kommunikation mit dem Sensor verwendet werden kann. Außerdem werden die Variablen

für die Chip Select Pins für das Accelerometer und das Gyroskop definiert, um die Kommunikation mit den jeweiligen Sensoren zu ermöglichen.

Da der Accelerometerteil des BMI088 standardmäßig im I²C Modus startet, muss dieser zunächst mittels einer Dummy-Read-Operation in den SPI Modus versetzt werden, bevor die Register konfiguriert werden können.

Codeblock 11.5: SPI Modus aktivieren

```
1 /* Dummy Read */
2 BMI088_ReadRegister(dev, BMI088_ACC_CHIP_ID, &regData,
  BMI088_ACC_SELECT);
```

Außerdem werden im Rahmen der Initialisierung die Register BMI088_ACC_CHIP_ID und BMI088_GYRO_CHIP_ID gelesen, um zu überprüfen, ob die SPI Kommunikation funktioniert und richtig konfiguriert ist.

Codeblock 11.6: Chip ID überprüfen

```
1 /* Check Device ID of the accelerometer */
2 status = BMI088_ReadRegister(dev, BMI088_ACC_CHIP_ID, &regData,
  BMI088_ACC_SELECT);
3 errNum += ( status != HAL_OK);
4
5 if ( regData != BMI088_ACC_CHIP_ID_VALUE){
6
7     return HAL_ERROR;
8
9 }
```

Anschließend werden die Register des Sensors mit den gewünschten Konfigurationswerten beschrieben.

Für das Accelerometer müssen folgende Register konfiguriert werden:

- ACC_CONF: Stellt ODR auf 800 Hz und den Filtermodus OSR4 ein.
- ACC_RANGE: Setzt den Messbereich des Accelerometers auf ± 24 g.
- ACC_PWR_CTRL: Aktiviert den Beschleunigungssensor im Normalbetrieb.
- INT1_IO_CTRL: Konfiguriert INT1 als aktiv-high Push-Pull-Ausgang.
- INT_MAP_DATA: Legt den Data-Ready-Interrupt des Accelerometers auf INT1.
- ACC_PWR_CONF: Setzt den Accelerometer in den aktiven Modus.

Für das Gyroskop müssen folgende Register konfiguriert werden:

- GYRO_LPM1: Setzt das Gyroskop in den Normalmodus.
- GYRO_BANDWIDTH: Konfiguriert Bandbreite und ODR auf 116 Hz bzw. 1000 Hz.
- GYRO_INT_CTRL: Aktiviert den Data-Ready-Interrupt des Gyroskops.
- INT3_INT4_IO_CONF: Konfiguriert INT3 als aktiv-high Push-Pull-Ausgang.
- INT3_INT4_IO_MAP: Legt den Data-Ready-Interrupt des Gyroskops auf INT3.
- GYRO_RANGE: Setzt den Messbereich des Gyroskops auf ± 500 °/s.

Hier ist zu Demonstrationszwecken die Einstellung für den Messbereich von ± 24 g angeführt:

Codeblock 11.7: Sensor konfigurieren

```

1 /* Accelerometer range (+-24G = 0x03) */
2 regData = 0x03;
3 status = BMI088_WriteRegister(dev, BMI088_ACC_RANGE, regData,
    BMI088_ACC_SELECT);
4 errNum += ( status != HAL_OK);

```

Wobei status den Rückgabewert der Funktion BMI088_WriteRegister speichert, um zu überprüfen, ob die Register erfolgreich beschrieben wurden, wenn dies nicht der Fall ist, wird die Variable errNum inkrementiert, um die Anzahl der Fehler zu zählen. Am Ende der Initialisierungsfunktion wird die Anzahl der Fehler zurückgegeben, um anzuzeigen, ob die Initialisierung erfolgreich war oder ob Probleme aufgetreten sind. Dies ist eine sehr einfache Fehlerbehandlung, die jedoch hilfreich sein kann, um Probleme bei der Sensorinitialisierung zu erkennen.

BMI088_ReadAcceleration und BMI088_ReadAngularRate

Die Funktionen BMI088_ReadAcceleration und BMI088_ReadAngularRate lesen die 8 Bit Rohdaten von den entsprechenden Sensorregistern, fügen diese zu 16 Bit Werten zusammen und konvertieren sie in $\frac{m}{s^2}$ für Beschleunigung und $\frac{rad}{s}$ für Winkelgeschwindigkeit, bevor sie in der Sensorstruktur gespeichert werden. Hier ist die Funktion BMI088_ReadAcceleration als Beispiel angeführt:

Codeblock 11.8: Beschleunigungsdaten lesen

```

1 /* legere accelerationis notitia */
2 HAL_StatusTypeDef BMI088_ReadAcceleration(BMI088 *dev){
3
4     int16_t pnRawData[3];
5     uint8_t regData[6];
6     uint8_t i = 0;
7
8     BMI088_ReadRegisters(dev, BMI088_ACC_DATA, regData, 6,
        BMI088_ACC_SELECT);
9
10    for(i=0; i<3; i++)
11    {
12        pnRawData[i]=(((uint16_t)regData[2*i+1]) << 8) + (
        uint16_t)regData[2*i]);
13    }
14
15    dev->acc_mps2[0] = (float)(pnRawData[2] * 0.00718505824f);
16    dev->acc_mps2[1] = (float)(pnRawData[1] * 0.00718505824f);
17    dev->acc_mps2[2] = -(float)(pnRawData[0] * 0.00718505824f);
18
19    return HAL_OK;
20 }

```

11.1.3. BMI088_ReadRegisters

Es ist wichtig die Rohdaten in einem Stück zu auszulesen (Burst Read), um sicherzustellen, dass die Daten konsistent sind und nicht durch eine Unterbrechung der Kommunikation während des Lesens unbrauchbar werden. Aufgrund dessen wird zusätzlich zur Funktion BMI088_ReadRegister, die nur ein einzelnes Register liest, auch die Funktion BMI088_ReadRegisters implementiert, die es

ermöglicht, mehrere Register in einem einzigen SPI-Transaktionszyklus zu lesen. Hier ist die Implementierung der Funktion BMI088_ReadRegisters angeführt:

Codeblock 11.9: Register lesen

```

1 HAL_StatusTypeDef BMI088_ReadRegisters(BMI088 *dev, uint8_t reg,
    uint8_t *data, uint8_t length, uint8_t select){
2     uint8_t res = HAL_OK;
3     uint8_t addr = reg | 0x80; /* MSB = 1 -> Read */
4     uint8_t dummy;
5
6     GPIO_TypeDef *cs_port;
7     uint16_t cs_pin;
8
9     /* Choose the correct CS */
10    if (select == BMI088_ACC_SELECT)
11    {
12        cs_port = dev->acc_cs_port;
13        cs_pin = dev->acc_cs_pin;
14    }
15    else
16    {
17        cs_port = dev->gyr_cs_port;
18        cs_pin = dev->gyr_cs_pin;
19    }
20
21    HAL_GPIO_WritePin(cs_port, cs_pin, GPIO_PIN_RESET);
22
23    if (HAL_SPI_Transmit(dev->spiHandle, &addr, 1, HAL_MAX_DELAY)
    != HAL_OK){
24        res = HAL_ERROR;
25    }
26    if(select == BMI088_ACC_SELECT){ /* Dummy Byte */
27        if (HAL_SPI_Receive(dev->spiHandle, &dummy, 1,
    HAL_MAX_DELAY) != HAL_OK){
28            res = HAL_ERROR;
29        }
30    }
31    if (HAL_SPI_Receive(dev->spiHandle, data, length, HAL_MAX_DELAY)
    != HAL_OK){
32        res = HAL_ERROR;
33    }
34
35    HAL_GPIO_WritePin(cs_port, cs_pin, GPIO_PIN_SET);
36    return res;
37 }

```

In dieser Funktion wird zunächst das Most Significant Bit (MSB) der auszulesenden Adresse auf 1 gesetzt, um anzuzeigen, dass es sich um einen Lesevorgang handelt. Anschließend wird je nach Zustand der Variable select der entsprechende Chip Select Pin ausgewählt und auf LOW gesetzt, um die Kommunikation mit dem Sensor zu starten. Danach wird die Adresse des ersten Registers gesendet, gefolgt von einem Dummy-Read, wenn das Accelerometer ausgewählt ist, da dieses im Gegensatz zum Gyroskop die Daten erst im zweiten und dritten Byte bereitzustellen. Schließlich werden die Daten über die HAL_SPI_Receive Funktion empfangen und der Chip Select Pin wieder

auf HIGH gesetzt, um die Kommunikation zu beenden. Die Funktion gibt den Status der Operation zurück, um anzuzeigen, ob der Lesevorgang erfolgreich war oder ob ein Fehler aufgetreten ist. Die Funktionen BMI088_ReadRegister und BMI088_WriteRegister sind im dargestellten Ablauf sehr ähnlich und sollen hier nicht weiter beschrieben werden.

12. CAD-Aufbau

12.1. Konzeptentwicklung

12.2. Flügelsteuermechanismus

Nach dem Entwurf der gesamten Raketengeometrie und der Festlegung der Form/Position der Flügel, wurde die Entwicklung eines Steuermechanismus für die Flügel begonnen. Nach ein Paar gescheiterten Versuchen, wurde ein Mechanismus entwickelt, bei dem sowohl die Motoren aufgrund von Platzmangel in das Gehäuse passten, als diese auch effizient das Drehmoment auf die Flügel übertragen konnten.

Aufgrund von Platzmangel im Heckteil der Rakete, konnten die die Servomotoren nicht direkt an die Flügel angebracht werden, sondern mussten über ein System mit 2 ineinander zeigenden Hebelarmen und einem Gelenkarm, der die zwei Hebelarme miteinander verbindet. Das Drehmoment der Servomotoren wird dabei 1 zu 1 übertragen, da beide Hebelarme die gleiche Länge einer Drehachse auf die andere haben. Abbildung 12.1 zeigt den Flügelsteuermechanismus von "ALBERT".

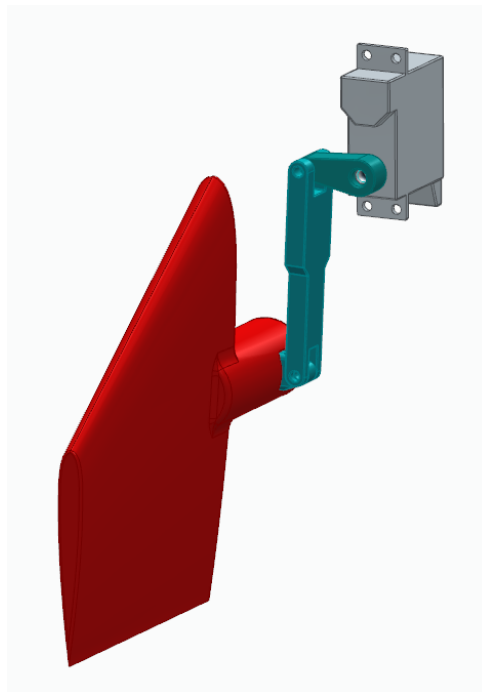


Abbildung 12.1.: Flügelsteuermechanismus

Um sicherzustellen, dass die Flügel stabil am Gehäuse befestigt sind, wurden diese mit 2 Kugellagern gelagert, die in das Gehäuse eingebaut wurden.

12.3. Innenstruktur und Materialwahl

Die Innenstruktur der Rakete wird auf 4 Carbonstäbe aufgebaut, die vom Heck bis zum Fallschirmfach verlaufen. Diese Stäbe sorgen für die notwendige Steifigkeit der Rakete und helfen dabei, die einzelnen Komponenten in nötiger Position zu halten. Die Stäbe mussten allerdings im Bereich des Flightcomputers unterbrochen werden, um Platz für diesen zu schaffen.

Die Wahl des Materials für die Rakete fiel auf 3D-gedrucktes PLA, da dieses Material leicht zu verarbeiten ist, eine ausreichende Festigkeit für die Anforderungen der Rakete bietet und kostengünstig ist. Allerdings um die Festigkeit der Flügel und dessen Steuermechanismus zu erhöhen, wurde das Greentec Pro Filament von Extrudr verwendet, da dieses Material genauso einfach wie PLA zu verarbeiten ist, aber eine deutlich höhere Festigkeit aufweist.

Für das Außengehäuse der Rakete wurde ein Kartorohr mit einem Außendurchmesser von 84 mm und einer Wandstärke von 2mm verwendet. Dieses ist eine kostengünstige und leicht verfügbare Option, die alle Anforderungen an die Festigkeit und Steifigkeit der Rakete ausreichend erfüllt.

12.4. Implementierung und Fertigung

Die Fertigung der Rakete erfolgte hauptsächlich durch 3D-Druck, wobei die meisten Komponenten mit dem Bambulab A1 mini gedruckt wurden. Die einzige Ausnahme war die Raketenspitze, die aufgrund ihrer Größe auf einem größeren Drucker gedruckt werden musste.

12.5. Herausforderungen und Lösungen

12.6. Fazit

13. Simulation der Aerodynamik

13.1. Simulation des Raketenkörpers in Ansys Fluent

Um alle nötigen Kraft-/Momentkoeffizienten für das Steuersystem zu berechnen, wurde der ganze Raketenkörper in Ansys Fluent mit Verschiedenen Anstellwinkeln der Flügel simuliert.

13.1.1. Definition der Referenzwerte und der simulierten/berechneten Werte

Um die richtige Berechnung von den Kraft-/Momentkoeffizienten und dessen Ableitungen müssen in Fluent bestimmte Referenzwerte gesetzt werden.

Definition der Referenzwerte für alle Simulationsdurchgänge

- $A = \frac{\pi \cdot D^2}{4}$: Referenzfläche (Frontfläche der Rakete) mit D als Durchmesser des Raketenkörpers
- $l = D$: Referenzlänge (Durchmesser des Raketenkörpers)

Definition der Winkel des Raketenkörpers und der Flügel

- α : Pitch Winkel des Raketenkörpers
- β : Yaw Winkel des Raketenkörpers
- δ_i : Anstellwinkel von Flügel i
- η : Anstellwinkel vom für Pitch verantwortliches Flügelpaar
- ζ : Anstellwinkeln vom für Yaw verantwortliches Flügelpaar

Aus der Simulation bezogene Kräfte/Beiwerte

- F_A : Axialkraft
- F_Y : Seitenkraft
- F_Z : Normalkraft
- C_l : Rollmomentbeiwert
- C_m : Pitchmomentbeiwert
- C_n : Yawmomentbeiwert

13.1.2. Simulationsdurchgänge

Um genug Messpunkte der verschiedenen Werte für genaue Ableitungen zu erreichen und um für alle simulierten Werte die besten Bedingungen zu schaffen, wo diese am genauesten von Fluent berechnet

werden können, wurden 11 Verschieden Fälle in der Simulation betrachtet. Durch diese 11 Durchgänge wurden sowohl alle möglichen Fälle bezüglich Pitch- und Yaw-Winkel abgedeckt, als auch Veränderungen im Anstellwinkel der Flügel. Da der Raketenkörper symmetrisch ist, sind die Beiwerte für Pitch und Yaw Stellungen identisch. Daher können die in der Simulation errechneten Werte für Pitch Änderung auch für Yaw-bezogene Beiwerte verwendet werden. Diese Werte für α, β, δ_i bzw. η und ζ wurden bei jedem Durchgang angenommen:

- Fall 1: $\alpha = 0^\circ, \beta = 0^\circ, \eta = 0^\circ, \zeta = 0^\circ$
- Fall 2: $\alpha = 0^\circ, \beta = 0^\circ, \eta = 2^\circ, \zeta = 0^\circ$
- Fall 3: $\alpha = 0^\circ, \beta = 0^\circ, \eta = 5^\circ, \zeta = 0^\circ$
- Fall 4: $\alpha = 2^\circ, \beta = 0^\circ, \eta = 0^\circ, \zeta = 0^\circ$
- Fall 5: $\alpha = 2^\circ, \beta = 0^\circ, \eta = 2^\circ, \zeta = 0^\circ$
- Fall 6: $\alpha = 2^\circ, \beta = 0^\circ, \eta = 5^\circ, \zeta = 0^\circ$
- Fall 7: $\alpha = 5^\circ, \beta = 0^\circ, \eta = 0^\circ, \zeta = 0^\circ$
- Fall 8: $\alpha = 5^\circ, \beta = 0^\circ, \eta = 2^\circ, \zeta = 0^\circ$
- Fall 9: $\alpha = 5^\circ, \beta = 0^\circ, \eta = 5^\circ, \zeta = 0^\circ$
- Fall 10: $\alpha = 5^\circ, \beta = 0^\circ, \delta_1 = 2^\circ, \delta_2 = 2^\circ, \delta_3 = 2^\circ, \delta_4 = 2^\circ$
- Fall 11: $\alpha = 0^\circ, \beta = 0^\circ, \delta_1 = 5^\circ, \delta_2 = 5^\circ, \delta_3 = 5^\circ, \delta_4 = 5^\circ$

13.2. Auswertung der Simulationsergebnisse

Nach dem Simulieren von allen 11 Fällen, wurden diese entsprechend ausgewertet und alle für die Regelung nötigen Beiwerte wurden errechnet. Im ersten Schritt wurden alle aus den Simulationsergebnissen bezogene Kräfte in entsprechende Beiwerte umgerechnet.

Berechnung der Kraftbeiwerte

Da es in der Student-Version von Ansys Fluent nicht möglich ist bei einem Kraft-Report dessen Beiwert zu berechnen, müssen die Axial-, Normal- und Seitkräfte mit der Hand auf die jeweiligen Beiwerte umgerechnet werden. Kraftbeiwerte werden grundsätzlich nach folgenden Prinzip berechnet:

$$C = \frac{F}{q \cdot A_{ref}} \quad (13.1)$$

Dabei ist C der resultierende Beiwert, F die simulierte Kraft, A_{ref} die Referenzfläche und q der dynamische Druck der wie folgt definiert ist:

$$q = \frac{1}{2} \rho v^2 \quad (13.2)$$

Simulationsergebnisse und Plots

Berechnungen der “Modellwerte” und Ableitungen

Als ersten Schritt wurden die Widerstandbeiwerte berechnet. Diese resultieren aus den folgenden Ableitungen:

$$C_A = C_{A0} + \frac{\Delta C_A}{\Delta \alpha} + \frac{\Delta C_A}{\Delta \beta} \quad (13.3)$$

Hier wird der Axialkraftbeiwert C_A aus dem Axialkraftbeiwert ohne jeglichen Anstellwinkeln C_{A0} , der Ableitung des Axialkraftbeiwertes nach Pitch Winkel $\frac{\Delta C_A}{\Delta \alpha}$ und der Ableitung nach Yaw Winkel $\frac{\Delta C_A}{\Delta \beta}$.

$$C_Z = \frac{\Delta C_Z}{\Delta \alpha} + \frac{\Delta C_Z}{\Delta \eta} \quad (13.4)$$

Hierbei resultiert der Normalkraftbeiwert C_Z aus dessen Ableitung nach Pitch Winkel $\frac{\Delta C_Z}{\Delta \alpha}$ und nach Anstellwinkel des für Pitch verantwortliches Flügelpaar $\frac{\Delta C_Z}{\Delta \eta}$.

$$C_Y = \frac{\Delta C_Y}{\Delta \beta} + \frac{\Delta C_Y}{\Delta \zeta} \quad (13.5)$$

Anschließend ergibt sich der Seitenkraftbeiwert C_Y aus dessen Ableitung nach Yaw Winkel $\frac{\Delta C_Y}{\Delta \beta}$ und nach dem Anstellwinkel des für Yaw verantwortliches Flügelpaar $\frac{\Delta C_Y}{\Delta \zeta}$.

Visualisierung der Strömung um den Raketenkörper

13.3. Messung eines Raketenflügels im Windkanal

13.4. Vergleich mit Messdaten

13.5. Herausforderungen und Lösungen

13.6. Fazit

Literatur

- [1] Advanced Navigation. *Inertial Measurement Unit (IMU) – An Introduction*. 2023. URL: <https://www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction/> (besucht am 30.12.2025).
- [2] Altium. *GPS-Antennen in Ihrem PCB-Design: Sie werden sich nie wieder verirren*. 2020. URL: <https://resources.altium.com/de/p/gps-antennas-in-your-pcb-design-you-won-t-get-lost-again> (besucht am 31.12.2025).
- [3] Birgit Moeller und Denis Williams. *Extended Kalmanfilter*. 2003. URL: https://www2.informatik.uni-halle.de/agprbio/AG/Lehre/ABV_WS03/material/extendedKalman.pdf (besucht am 09.01.2026).
- [4] Bosch Mobility. *EN | Bosch Working principle of a pressure sensor*. 2014. URL: https://www.youtube.com/watch?v=zXIeqeT_FC8 (besucht am 30.12.2025).
- [5] Dor Fuchs. *Wie ein bisschen Mathe bei der Mondlandung half (Das Kalman-Filter)*. 2019. URL: <https://www.youtube.com/watch?v=EBjca6tPu00> (besucht am 07.01.2026).
- [6] Robert Bosch GmbH. „Vorrichtung und Verfahren zur Messung von Magnetfeldern“. WO2012/126693A1. 2012. URL: <https://patentimages.storage.googleapis.com/f7/0c/82/ff4acea094b486/WO2012126693A1.pdf>.
- [7] JLCPCB. *Datasheet for YSX321SL Crystall Oscillator*. URL: <https://jlcpcb.com/api/file/downloadByFileSystemAccessId/8588879072148017152> (besucht am 06.03.2026).
- [8] jlcpcb. *Decoupling vs Bypass Capacitor : What's the Difference*. 2025. URL: <https://jlcpcb.com/blog/decoupling-vs-bypass-capacitor> (besucht am 31.12.2025).
- [9] Aliyu Bhar Kisabo und Aliyu Funmilayo Adebimpe. „State-Space Modeling of a Rocket for Optimal Control State-Space Modeling of a Rocket for Optimal Control System Design“. In: (2019). URL: <https://www.intechopen.com/chapters/64567>.
- [10] Nancy Hall. *Beginners Guide to Aeronautics*. 2023. URL: <https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/> (besucht am 15.01.2026).
- [11] Nikola T. *Standard 4- and 6-Layer PCB Stackups*. 2013. URL: <https://www.bitweenie.com/listings/standard-4-and-6-layer-pcb-stackups/> (besucht am 28.12.2025).
- [12] Phil's Lab. *(Sponsored) Extended Kalman Filter - Sensor Fusion 3 - Phil's Lab 37*. 2021. URL: <https://www.youtube.com/watch?v=hQUkiC5o0JI&t=17s> (besucht am 08.01.2026).
- [13] Phil's Lab. *(Sponsored) PCB Stack-Up and Build-Up - Phil's Lab 56*. 2022. URL: <https://www.youtube.com/watch?v=QA0EtfvCaMw&t=554s> (besucht am 29.12.2025).
- [14] Phil's Lab. *(Sponsored) Switching Regulator PCB Design - Phil's Lab 60*. 2022. URL: <https://www.youtube.com/watch?v=AmfLhT5SntE&t=582s> (besucht am 29.12.2025).
- [15] Phil's Lab. *KiCad 6 STM32 PCB Design Full Tutorial - Phil's Lab 65*. 2022. URL: https://www.youtube.com/watch?v=aVUqaB0IMh4&list=PLklY_EOG-raUnTVxycViqcf9D-y793ckX&index=13 (besucht am 06.03.2026).

- [16] Prof. Dr.-Ing. Heinz Schmidt-Walter. *Abwärtswandler*. URL: http://schmidt-walter-schaltnetzteile.de/snt/snt_deu/sntdeu2.pdf (besucht am 29.12.2025).
- [17] R. E. KALMAN. *A New Approach to Linear Filtering and Prediction Problems*. 1960. URL: <https://www.unitedthc.com/DSP/Kalman1960.pdf> (besucht am 07.01.2026).
- [18] Richard Nakka. *Richard Nakka's Experimental Rocketry Site*. 2026. URL: <https://www.nakka-rocketry.net/> (besucht am 18.02.2026).
- [19] Steven L. Brunton und J. Nathan Kutz. *Data Driven Science & Engineering*. 2017. URL: <https://databookuw.com/databook.pdf> (besucht am 15.01.2026).
- [20] STM Electronics. *Datenblatt für STM32F405RGT6 Microcontroller*. URL: <https://www.st.com/resource/en/datasheet/stm32f405rg.pdf> (besucht am 06.03.2026).
- [21] STM Electronics. *Datenblatt für STM32H753VIT6 Microcontroller*. URL: <https://www.st.com/resource/en/datasheet/stm32h753vi.pdf> (besucht am 06.03.2026).
- [22] Texas Instruments. *Datenblatt für TPS54308 Buck Converter*. URL: <https://www.ti.com/lit/ds/symlink/tps54308.pdf> (besucht am 08.03.2026).
- [23] Texas Instruments. *TPS55289 Buck Converter Datasheet*. 2022. URL: <https://www.ti.com/lit/ds/symlink/tps55289.pdf> (besucht am 29.12.2025).
- [24] Todd Toporski. *Switch Node Layout Considerations for EMC*. URL: https://www.monolithicpower.com/en/learning/resources/switch-node-layout-considerations-for-emc?srsltid=AfmBOoqRXddokKGt6qhy2Qypf72WE3ECnBd1sNlJD2nkrFNsOXMkb-D_ (besucht am 29.12.2025).
- [25] u-blox. *Datenblatt für ZOE-M8Q GNSS Receiver*. URL: https://content.u-blox.com/sites/default/files/ZOE-M8_HIM_UBX-16030136.pdf (besucht am 08.03.2026).
- [26] Walter Lohmueller. *Peak of Flight Newsletter: Designing and Building for SuperSonic Flight*. 1562. URL: <https://www.apogeerockets.com/Peak-of-Flight/Newsletter563> (besucht am 18.02.2026).
- [27] Wikipedia Contributors. *Kalman-Filter*. 2025. URL: https://de.wikipedia.org/wiki/Kalman-Filter#cite_note-1 (besucht am 07.01.2026).
- [28] Wikipedia Contributors. *NACA-Profile*. 2025. URL: <https://de.wikipedia.org/wiki/NACA-Profile> (besucht am 18.02.2026).
- [29] Zachariah Peterson. *Was ist die Rückstromführung in einem PCB?* 2021. URL: <https://resources.altium.com/de/p/what-return-current-path-pcb> (besucht am 29.12.2025).